

# **OpenRover: A 3D-printed open-source robotic platform for software and hardware development**

Pol Valero Arjona

*La Salle Campus Barcelona – Grau en Enginyeria Informàtica*

Barcelona, Juny 2025

**NOTICE:** Some pages of this report have been removed to reduce the length of the document and highlight the most interesting parts of the project.

# Abstract

Rovers are planetary surface exploration robots that space agencies like NASA send to Mars in order to explore and collect information that will be sent back to Earth.

This final degree project originates from a personal hobby project in which, using an existing 3D design of a Mars rover replica found online, the design has been adapted, improved, 3D printed and assembled at home. The final 3D printed result has the functional structure and appearance of the "Perseverance" rover, which was launched by NASA in July 2020, and landed on Mars in February 2021.

The main objective of the final degree project is to develop an open-source robotic platform that can enable anyone who tries building it to learn more about both the software and hardware aspects of DIY robotic projects, specially using and learning about widely adopted microcontrollers such as "Arduino" and "ESP32". As rovers are extremely sophisticated machines, they provide the perfect excuse to incorporate into the robotic platform several complex functionalities that replicate, at a smaller scale, the functionalities that real Mars rovers have (e.g., robotic arm, environmental sensors, 6-wheel drive).

Although many open-source robotic projects already exist (even some also related to Mars rover robots), they usually only use a single type of development board. In the developed OpenRover project, partly due to its complexity and wide scope, several development boards are used among its main functional modules (e.g., RaspberryPi for web connectivity, ESP32 for embedded screen, Arduino for motor control) and there are interactions between these modules. This adds more educational value to the project, allowing to see each development board optimal usecase, and differentiates OpenRover from other open-source DIY robotic projects.

Starting from the 3D printed structure of the rover, this project has incorporated into the rover all the hardware (e.g., motors, microcontrollers, sensors) as well as developed all the software (e.g., web dashboard for monitorization, creation of communication protocols between modules, generation of control signals for motors) that is necessary in order to operate the rover remotely. Additionally, a remote control module with a touchscreen has also been developed, which serves both as a means to operate the rover and to monitor its status in a graphical way.

The project has been named OpenRover, due to the "open source" nature of the project. OpenRover has been able to meet its main objective, as the created robotic platform and remote control module have incorporated several interesting functionalities using different development boards (Arduino, ESP32, RaspberryPi), making this project a useful resource for people wanting to learn more about the DIY robotics field, or for those who want to replicate and build upon the OpenRover robotic platform.

**Keywords** – Robotics, RaspberryPi, Arduino, ESP32, LVGL, Web dashboard, Embedded system



# Contents

|        |                                                   |    |
|--------|---------------------------------------------------|----|
| 1      | Introduction .....                                | 1  |
| 1.1    | Project motivation and origin .....               | 1  |
| 1.2    | Objectives .....                                  | 2  |
| 1.3    | Project modules overview .....                    | 3  |
| 1.4    | Methodology .....                                 | 4  |
| 2      | Requirements definition .....                     | 6  |
| 2.1    | Mechanical structure requirements .....           | 6  |
| 2.2    | Hardware requirements .....                       | 6  |
| 2.3    | Software requirements .....                       | 7  |
| 3      | Theoretical framework.....                        | 9  |
| 3.1    | Mars rover definition .....                       | 9  |
| 3.1.1  | Components of a Mars rover.....                   | 10 |
| 3.1.2  | Rocker-bogie mechanism .....                      | 10 |
| 3.1.3  | Ackermann steering .....                          | 12 |
| 3.2    | Open-source definition .....                      | 14 |
| 3.3    | Similar existing projects .....                   | 15 |
| 3.3.1  | Distinctive characteristics of OpenRover .....    | 17 |
| 3.4    | Hardware components analysis.....                 | 18 |
| 3.4.1  | Development boards .....                          | 18 |
| 3.4.2  | Touchscreens.....                                 | 21 |
| 3.4.3  | Cameras .....                                     | 22 |
| 3.4.4  | Batteries .....                                   | 22 |
| 3.4.5  | Wireless communication modules .....              | 22 |
| 3.4.6  | Environmental sensors .....                       | 23 |
| 3.4.7  | Motors .....                                      | 23 |
| 3.4.8  | Servomotors .....                                 | 24 |
| 3.4.9  | Stepper motors.....                               | 24 |
| 3.4.10 | Generic remote control.....                       | 25 |
| 3.4.11 | Wired communication protocols .....               | 26 |
| 3.5    | Software technologies and libraries analysis..... | 26 |
| 3.5.1  | Development environment .....                     | 27 |
| 3.5.2  | Operating systems .....                           | 28 |

|        |                                                     |    |
|--------|-----------------------------------------------------|----|
| 3.5.3  | Wireless communication .....                        | 28 |
| 3.5.4  | Wired communication .....                           | 29 |
| 3.5.5  | Touchscreen .....                                   | 29 |
| 3.5.6  | Camera .....                                        | 31 |
| 3.5.7  | Motors .....                                        | 31 |
| 3.5.8  | Servomotors .....                                   | 32 |
| 3.5.9  | Stepper motors.....                                 | 32 |
| 3.5.10 | Environmental sensors.....                          | 32 |
| 3.5.11 | Generic remote control.....                         | 33 |
| 3.5.12 | Website .....                                       | 33 |
| 4      | Practical framework .....                           | 35 |
| 4.1    | Solution proposition .....                          | 35 |
| 4.1.1  | Selected hardware components .....                  | 35 |
| 4.1.2  | Selected software technologies and libraries .....  | 36 |
| 4.2    | Hardware and software selection rationale .....     | 36 |
| 4.2.1  | Hardware components rationale .....                 | 36 |
| 4.2.2  | Software technologies and libraries rationale ..... | 39 |
| 4.3    | Mechanical structure implementation .....           | 42 |
| 4.4    | Hardware implementation .....                       | 43 |
| 4.4.1  | Central rover module .....                          | 44 |
| 4.4.2  | Remote control and touchscreen module.....          | 48 |
| 4.4.3  | Camera and webserver module .....                   | 51 |
| 4.5    | Software implementation.....                        | 54 |
| 4.5.1  | Central rover module .....                          | 54 |
| 4.5.2  | Remote control and touchscreen module.....          | 60 |
| 4.5.3  | Camera and webserver module .....                   | 70 |
| 4.5.4  | Handling of concurrent activities .....             | 80 |
| 4.5.5  | Communication protocol.....                         | 81 |
| 4.6    | Making the project open-source .....                | 83 |
| 5      | Fulfillment of requirements .....                   | 85 |
| 5.1    | Mechanical structure requirements .....             | 85 |
| 5.2    | Hardware requirements .....                         | 85 |
| 5.3    | Software requirements.....                          | 86 |
| 6      | Cost analysis .....                                 | 88 |

|     |                            |     |
|-----|----------------------------|-----|
| 6.1 | Temporal costs.....        | 88  |
| 6.2 | Economic costs .....       | 89  |
| 7   | Conclusions .....          | 90  |
| 8   | Future lines of work ..... | 92  |
| 9   | References.....            | 94  |
|     | Annex I.....               | 99  |
|     | Annex II.....              | 104 |

# List of figures

|                                                                                                                                             |    |
|---------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1. Modules diagram (own elaboration) .....                                                                                           | 4  |
| Figure 2. Perseverance twin rover testing on Earth [2] .....                                                                                | 9  |
| Figure 3. Rocker-bogie mechanism [5].....                                                                                                   | 11 |
| Figure 4. Ackermann steering turning point [7] .....                                                                                        | 12 |
| Figure 5. Ackermann steering rover variables [6] .....                                                                                      | 12 |
| Figure 6. Rover modified design and assembled mechanical structure (own elaboration of images and original design extracted from [9] )..... | 43 |
| Figure 7. Central rover module schematic (own elaboration) .....                                                                            | 45 |
| Figure 8. Central rover module chassis wiring (own elaboration) .....                                                                       | 46 |
| Figure 9. Rover power distribution PCBs (own elaboration) .....                                                                             | 47 |
| Figure 10. Initial rover component connections (own elaboration) .....                                                                      | 47 |
| Figure 11. Final rover component connections (own elaboration) .....                                                                        | 47 |
| Figure 12. Remote control and touchscreen module schematic (own elaboration) .....                                                          | 49 |
| Figure 13. Remote control and touchscreen prototyping board (own elaboration) .....                                                         | 50 |
| Figure 14. Remote control custom PCB (own elaboration) .....                                                                                | 51 |
| Figure 15. Camera and webserver module schematic (own elaboration).....                                                                     | 52 |
| Figure 16. Initial camera and webserver module connections (own elaboration) .....                                                          | 53 |
| Figure 17. Final camera and webserver module connections (own elaboration) .....                                                            | 53 |
| Figure 18. Key header and implementation files for central rover module (own elaboration) .                                                 | 55 |
| Figure 19. Central rover module data structures (own elaboration).....                                                                      | 57 |
| Figure 20. Central rover module main activity diagram (own elaboration) .....                                                               | 57 |
| Figure 21. Central rover module “Execute frame type functions” activity (own elaboration) ...                                               | 58 |
| Figure 22. Central rover module “Execute operation mode” activity (own elaboration) .....                                                   | 59 |
| Figure 23. Central rover module “Get status data frame” activity (own elaboration) .....                                                    | 59 |
| Figure 24. Embedded remote control UI icons in Figma (own elaboration).....                                                                 | 62 |
| Figure 25. SquareLine studio components with default theme and configuration (own elaboration).....                                         | 62 |
| Figure 26. Final remote controller screens design (own elaboration).....                                                                    | 63 |
| Figure 27. Key header and implementation files for ESP32S3 and touchscreen submodule (own elaboration).....                                 | 64 |
| Figure 28. Library headers and main.cpp file for Arduino Nano submodule (own elaboration) .....                                             | 66 |
| Figure 29. ESP32S3 and touchscreen submodule main activity diagram (own elaboration) .....                                                  | 66 |
| Figure 30. ESP32S3 and touchscreen submodule “Execute frame type functions” activity (own elaboration).....                                 | 67 |
| Figure 31. ESP32S3 and touchscreen submodule buttons and switches flow (own elaboration) .....                                              | 67 |
| Figure 32. Arduino Nano submodule main activity diagram (own elaboration) .....                                                             | 68 |
| Figure 33. Frame structure candidates (own elaboration) .....                                                                               | 69 |
| Figure 34. Key implementation files and libraries for camera and webserver module (own elaboration).....                                    | 70 |
| Figure 35. Camera and webserver module main activity diagram (own elaboration) .....                                                        | 71 |
| Figure 36. Camera and webserver module “Handle received frames thread” activity (own elaboration).....                                      | 72 |



|                                                                                                                                                        |     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 37. Camera and webserver module “Manage status data thread” activity (own elaboration).....                                                     | 72  |
| Figure 38. Camera and webserver module “Flask webserver threads” activity (own elaboration) .....                                                      | 73  |
| Figure 39. Web dashboard monitor screen (own elaboration) .....                                                                                        | 76  |
| Figure 40. Web dashboard in mobile browser (own elaboration).....                                                                                      | 77  |
| Figure 41. Web dashboard and embedded UI comparison (own elaboration) .....                                                                            | 77  |
| Figure 42. RaspberryPi startup bash script (own elaboration) .....                                                                                     | 78  |
| Figure 43. Tasks timing using non-blocking “millis()” template (own elaboration).....                                                                  | 81  |
| Figure 44. C++ (left) and Python (right) frame structure (own elaboration) .....                                                                       | 81  |
| Figure 45. Frame types definition C++ header file (own elaboration) .....                                                                              | 82  |
| Figure 46. Project’s time distribution pie chart (own elaboration) .....                                                                               | 88  |
| Figure 47. Project’s Gantt diagram (own elaboration) .....                                                                                             | 89  |
| Figure 48. Original DIY Perseverance Rover Replica design (own elaboration of images and original design extracted from [9] ).....                     | 99  |
| Figure 49. Original (left) and modified (right) Jakob Krantz arm design (own elaboration of images and original designs extracted from [9] [10] )..... | 99  |
| Figure 50. Modified DIY Perseverance Rover Replica design (own elaboration of images and original design extracted from [9] ).....                     | 100 |
| Figure 51. Organized set of OpenRover’s 3D printed pieces (own elaboration) .....                                                                      | 100 |
| Figure 52. OpenRover body assembly (own elaboration).....                                                                                              | 101 |
| Figure 53. Custom remote control cases design (own elaboration) .....                                                                                  | 102 |
| Figure 54. 3D-printed remote-control cases (own elaboration).....                                                                                      | 102 |
| Figure 55. Original (left) and modified (right) interior head design (own elaboration of images and original design extracted from [9] ).....          | 103 |
| Figure 56. Interior head components placement (own elaboration) .....                                                                                  | 103 |

# List of tables

Table 1. Arduino boards comparison (own elaboration with data extracted from [19] ) ..... 18

Table 2. ESP32 SoC comparison (own elaboration with data extracted from [20] )..... 19

Table 3. Teensy boards comparison (own elaboration with data extracted from [23] ) ..... 20

Table 4. RaspberryPi boards comparison (own elaboration with data extracted from [26] ) .... 20

Table 5. OpenRover components cost (own elaboration) ..... 105

# Acronyms

ADC: Analog to Digital Converter

AP: (WiFi) Access Point

BOM: Bill Of Materials

CSI: Camera Serial Interface

DIY: Do It Yourself

FFC: Flat Flexible Cable

FPV: First Person View

FPS: Frames Per Second

GPIO: Global Purpose Input Output

GUI: Graphical User Interface

I/O: Input Output

IDE: Integrated Development Environment

LORA: Long Range module

OOP: Object Oriented Programming

OS: Operating System

PCB: Printed Circuit Board

PWM: Pulse Width Modulation

RC: Remote Control

RTOS: Real Time Operating System

SoC: System on Chip

STL: Standard Triangle Language

VCS: Version Control System



# 1 Introduction

## 1.1 Project motivation and origin

This final degree project originates from a personal project of mine that started nearly 2 years ago, in the summer of 2023. For a long time, I have had a passion for tinkering with electronics and coding, and in my free time I have developed all kinds of DIY projects. Besides experimenting with electronics and programming, I have also enjoyed designing and 3D printing components for these at-home DIY hobby projects.

As my Computer Engineering degree was coming to an end, I felt the need to create a project that combined all my existing knowledge in 3D design, 3D printing and mechanical assembly with the more electronics and software related knowledge that I had acquired during my degree. That is why I started looking for a project that could be complex enough to be a challenge and where I could spend a lot of time not only developing the hardware aspect of the project, but the software aspect as well.

After extensive research on the internet, looking for already existing open-source projects and taking interesting insights from each of them, I found a project idea that met my needs. This project was called "DIY Mars Perseverance Rover Replica", and it was created by a mechatronics enthusiast who spent a lot of time making a very detailed and 3D-printable functional model of a Mars rover. He also created a website and some videos showing how to assemble the 3D printed model and how to put some essential hardware components and a microcontroller to make the model operational.

One might think that since this enthusiast did the 3D design, mechanical assembly guides, put some hardware and created code for a microcontroller, there would be no point in taking this project as a starting point for another project, as most of the work would already be done. However, after careful consideration, I ended up thinking that this was not necessarily the case. First of all, the project creator had made a very detailed and functional mechanical platform, but the electronic hardware and coding aspects of the project were far from complex. The operational functionalities that he ended up implementing were very limited, only being able to operate the rover with an existing generic remote control, and not adding any sort of sensors nor complex functionalities. He did implement a camera for the rover, but it was also very basic and only broadcasted low quality analog video using a FPV drone transmitting module. Secondly, the mechanical and 3D printed design left lots of room for expansion; for example, he had not implemented any sort of robotic arm (which real Mars rovers have) nor cable management system.

This is why I finally decided to take this existing project as a starting point for my personal project. It is important to mention that I only replicated the mechanical aspects of the project, such as the 3D design and assembly. All the electronic hardware and software of my rover project was created from scratch, using a completely different approach than the one the original creator of the rover model followed. Most of the hardware component similarities are from elements that were constrained by the existing 3D design (e.g., motors and servomotors) as there was no point in modifying the 3D design if these components were known models that

could be easily purchased, as it was the case. The remaining hardware component similarities, such as the motor driver model, are from electronic components that are widely used in similar robotic projects.

After some modifications to the original design, I started 3D printing the model on the summer of 2023. As the rover model was quite large (1/4 scale of a real rover) two entire months were required to 3D print all the pieces, with not many days where the 3D printer was not printing. Between the end of the summer of 2023 and the summer of 2024 I did the assembly of the mechanical parts of the rover and the initial wiring of the motors and servomotors.

As I was developing the mechanical aspects of the rover, I began to realize that the scope of the project was going to be very large and that there would be a lot of hours of work involved. That is when I began to think that this personal project could in fact be converted into the final project of my degree. After validating the project idea with one professor at La Salle URL, I ended up concluding that the hardware and software development parts of the project would be the main focus of my final degree project.

In my previous personal projects, I had frequently used "Arduino" microcontrollers that were very easy to use and met my needs, but for this project, I wanted to learn more about other boards that could also be used in more complex robotic projects, such as the "ESP32", "Teensy", and "Raspberry Pi" boards. After searching online, I saw that most DIY robotics projects only used a single type of development board, and that is why I decided that for my Mars rover robot, I would try to use several types of development boards, each one of them used in the module of the rover that best takes advantage of their specific characteristics. This would add more educational value to the project, allowing to see each development board optimal usecase, and would differentiate the project from other existing open-source DIY robotic projects.

## 1.2 Objectives

The main objective is to develop an open-source robotic platform build upon the 3D printed model of the Mars rover that was developed as a personal project. The project has been named OpenRover, due to the open-source nature of the platform. This platform will include all the necessary hardware and software to achieve the specific objectives listed below.

As rovers are extremely sophisticated machines, they provide the perfect excuse to build a complex robotic platform that can be continuously expanded by replicating more functionalities that real rovers have.

Other more specific objectives of the project are:

- Control the rover remotely via a custom remote control module with a touchscreen. This module has to be an add-on to an existing generic RC or to a custom RC.
- Create a web dashboard to monitor and control the rover remotely. Self-host the server of this web dashboard on the rover, creating its own WiFi network at startup, and allowing clients to connect and see the dashboard on their browsers. Incorporate a real time camera feed on the web dashboard, showing the view from the rover's head.

- Gather status data so that it can be displayed on the web dashboard and on the screen of the custom remote control.
- Incorporate into the platform a set of functionalities that replicate, at a smaller scale, the functionalities that real Mars rovers have, like a robotic arm, 4-wheel steering or the ability to perform 360° turns.
- Enable anyone who tries building the platform to learn more about the software and hardware aspects of DIY robotic projects, and to have the possibility of adding more software or hardware features to the platform.
- Use different types of development boards among the rover's main functional modules and have interactions between these different development boards. This adds more educational value to the project, allowing to see the optimal usecase for each type of development board. These development boards should be cheap and accessible to the general public.

## 1.3 Project modules overview

The rover has dozens of individual components, both at a hardware and a software level; the objective of this section is not to detail each one of them, but to show the overall picture of the system, with its main modules and components, and the data interactions between the modules.

The 3 main modules of the rover are:

- Central rover module: It has the development board that is inside the main body of the rover and that manages all moving elements (e.g., motors, servomotors) as well as the rover's sensors and sending/receiving the radio signal.
- Remote control and touchscreen module: It has the development board that is responsible for the touchscreen, reading the RC channel values and sending/receiving the radio signal.
- Camera and webserver module: It has the development board that is inside the head of the rover and that is able to use a camera, as well as generating a WiFi and hosting a website.

Each of these modules has several individual components inside, on the diagram shown in **Figure 1**, only the most important individual components of each module are shown. This diagram is very basic and only meant to facilitate the understanding of each of the module's purpose and its interactions.

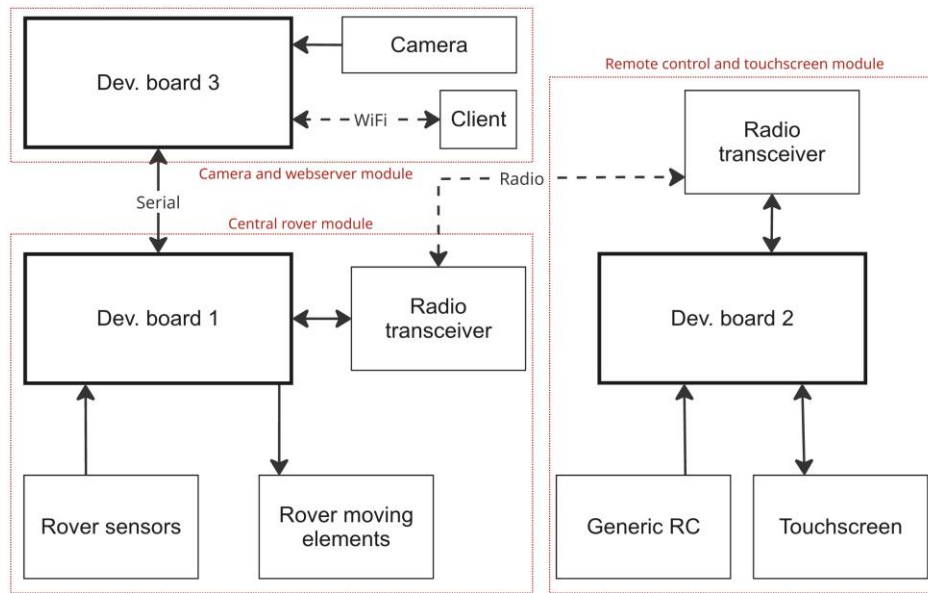


Figure 1. Modules diagram (own elaboration)

In **Figure 1**, the dashed arrows indicate wireless communication between the components, and the conventional arrows indicate wired communication. The directionality of the arrows is also important, conceptually indicating the origin and destination of the data that is transmitted. Most of the communication methods employed to connect the different components are not detailed in this diagram, as only an overview of the system is shown at this moment. The exact communication methods and protocols used (e.g., between development board 2 and the touchscreen), will be specified in the analysis section for each component that requires it.

## 1.4 Methodology

Regarding the methodology that has been used to develop this project, we have to differentiate between the theoretical and practical aspects of the project.

The first steps of development were part of the theoretical framework. In this stage, an investigation was conducted on how Mars rovers work and what some of their main functionalities are. This research was carried out to provide some context, since the aim of this project is to use the Mars rover concept simply as an excuse to develop a very complex robot. Therefore, the developed functionalities do not need to faithfully replicate the ones that real Mars rovers have.

Once this initial investigation was conducted, an exploration of existing open-source DIY robotic projects was done, in order to see what new and interesting concepts each one of them contributed (e.g., use of a custom 3D printed RC customized to the robot needs), but also what things did they lack (e.g., only one type of development board was used).

Thanks to this exploration, but also to many completely original ideas not seen in other projects, a list of objectives for the project was created. From these general objectives, a more specific list of requirements was elaborated; these requirements detail the different technical outcomes that are expected, and are very useful to see, at the end of the project, whether each requirement was fulfilled or not. The final part of the theoretical framework consisted in



analyzing specific hardware components and software technologies that had the potential of being included in the project.

In the practical framework, from the list of potential components and technologies, some were selected for the design and creation of the project's solution, and arguments are provided to justify why these options were the ones selected. Having selected the hardware components, it was then possible to buy all the necessary hardware components of the project.

With these hardware components at hand, the next step consisted in conducting several experiments using prototyping boards and creating test programs, in order to see the limitations and possible problems of the hardware and software. Once the testing was complete, if necessary, the hardware was soldered on a soldering board. Finally, with all the hardware securely placed and connected, new software functionalities that used the hardware were developed. As the main rover modules are fairly independent, the process described in this paragraph was done on a module-by-module basis, starting with the central rover module, continuing with the RC and touchscreen module, and ending with the camera and webserver module.

Finally, once the technical development was finished, some READMEs were created and the project was made open-source and published on GitHub.

## 2 Requirements definition

From the objectives of the project, the requisites can be defined and categorized into the mechanical, software and hardware aspects of the rover model.

### 2.1 Mechanical structure requirements

Even though the mechanical aspect of the rover model is not fully inside the scope of this final degree project, which focuses more on the hardware and software aspects, it is still interesting to point out some of the requirements that the mechanical platform should fulfill.

- 6-wheel drive platform.
- 4 steerable wheels.
- 4-axis foldable robotic arm with gripper.
- Tilttable and rotary head, with space for a development board and a camera.
- Rocker-bogie mechanism.
- 3D printed structure except the components subjected to high stress or that have common shapes (e.g., tubes).
- All-terrain capabilities, capable of withstanding a lot of mechanical stress.

### 2.2 Hardware requirements

The following list of hardware requirements conceptually details the main hardware components that are required for the rover, as well as some of their more important specifications.

- Custom remote control and touchscreen module that boots almost immediately, to allow a quick and easy operation and monitorization of the robot. The RC has to be able to monitor its battery status. The custom remote control module and screen has to be easily detachable, using wire connectors, allowing to return the generic RC or custom RC to its original form.
- Single battery that powers all the elements of the rover model and that has a good enough discharge rate.
- Radio module that allows bidirectional communication between the rover and the remote control, has low latency and good enough data transmission rate.
- Environmental sensors in the rover model to detect temperature, humidity, pressure and altitude. Battery sensor to monitor the rover's battery status.
- Development board that allows the use of a camera and that has an integrated WiFi module. The board must be able to broadcast a WiFi signal and host a simple website server.

- Motor drivers and motors that allow speed control. Servomotors with a 180° freedom range.
- Development board that can take care of all the real time operations on the rover model (e.g., motor and servomotor control). This board has to boot almost immediately.
- Cable management, especially on the more visible and exposed parts of the model. Use standard wiring connectors to facilitate assembly, disassembly and repairs (e.g., JST connectors).
- All development boards must support some sort of serial communication.

## 2.3 Software requirements

The following list of software requirements details the specific characteristics and functionalities that have to be achieved via software.

- Concurrency support. Both the rover model and its custom remote control have to be able to execute multiple tasks concurrently (e.g., send status data through radio while an automated servomotor movement is being performed).
- Different operation modes have to be implemented. 4 main operation modes can be selected from the RC touchscreen: conventional driving control, 360° turn control, robotic arm control, head control.
- The RC module has to read the generic or custom RC channel values (e.g., via a communication port in a generic RC, or using an ADC to read joystick values in a custom RC) and send a radio signal with these channel values, using the custom communication protocol of the rover. The module has to have at least 8 channels (4 for the joystick axis and 4 for the AUX channels) .
- The RC touchscreen has to be able to display the following status data: rover battery, RC battery, temperature, humidity, pressure, altitude, current RC values being sent, CPU temperature and workload of the board that hosts the website and has the camera, speed, wheel revolutions, traveled distance. For the temperature and humidity, graphs detailing their evolution over the past minute also have to be displayed.
- The RC touchscreen has to display a selection menu to change between the "Home/Control" and "Monitor" screens. These screens will have all the elements described in previous remote control requirements.
- The webserver of the monitoring dashboard, which displays the same status data as the RC touchscreen and the temperature and humidity graphs, must be very lightweight and self-hosted on a development board. The WiFi network and the web server automatically start as soon as the development board is booted. A camera feed panel appears in the web dashboard, and the feed can be started or stopped to save resources. Some type of control element (e.g., joysticks) also should appear in the web dashboard, allowing to manage some of the rover functionalities.

- Implement Ackermann steering equations to avoid tire slipping. With these equations, given a wanted turning radius for the rover, we calculate the optimal steering angle and speed of each wheel.
- Create a custom communication protocol between the development boards that has a low packet size, helping to increase packet transmission rate and improve concurrency. This custom protocol should be the same for radio and wired serial communications, allowing to easily forward to serial a received radio packet and vice versa.
- Implement remote control link loss detection, showing the RC link status on the touchscreen of the remote, and making sounds with a buzzer in the rover if it detects that the signal has been lost. This buzzer also has to make sounds if the rover's battery is low.
- Develop modular code, organizing the implementation of functionalities into separate files, each responsible for a specific aspect of the system.

## 3 Theoretical framework

Prior to being able to implement a solution, a thorough research phase must be done. Among other things, this phase includes research related to the way Mars rovers work and what some of their main functionalities are, as well as deep exploration of different software and hardware technologies that have the potential of being included in the project.

### 3.1 Mars rover definition

Mars rovers are planetary surface exploration robots that space agencies like NASA send to Mars to explore the planet and collect information that is sent back to Earth.

The latest Mars rovers, like the Perseverance [1] model that was sent in 2020, have the main objective of searching for signs of ancient microbial life, exploring the habitability of the planet and collecting geological samples for a potential future pickup mission.

Due to significant delay in the radio signal transmissions between Mars and Earth, rovers tend to have a high degree of autonomy, being able to explore previously designated areas on their own [1] .

Rovers have to endure conditions that are far more challenging than the ones on Earth, as they must be able to withstand high levels of radiation, extreme temperatures, high acceleration, fine dust particles... Once on their voyage or on their final destination, no reparation is possible, so they must be very resilient in order to remain functional for as long as possible.

Six wheeled platforms are often used by rovers to ensure good traction and optimal all-terrain capabilities. Moreover, an innovative and very interesting mechanism, called rocker-bogie [4] , is implemented in many of the rovers to ensure that they are able to climb over most of the obstacles that they may encounter on Mars.



Figure 2. Perseverance twin rover testing on Earth [2]

### 3.1.1 Components of a Mars rover

In order to provide some context and to replicate some of the functionalities that real Mars rovers have, it is interesting to research what their main components are and what purposes do they serve. The exact components that Mars rovers have differ between models. Focusing on the models sent in the last two expeditions, Curiosity and Perseverance [1], these are some of the core components that they have:

- **Body:** Also known as Warm Electronics Box (WEB). It has a rectangular cuboid shape, with a strong and shielded outer layer, that helps protecting the rover's computer and other electronics that are inside the body.
- **Wheels:** Six wheels are used for the platform, each of them having their own motor. Only four of the wheels, the two at the front and the two at the back, are steerable. This four-wheel steerable drive enables the rover to make better arcing turns and to have the capability of rotating a full 360 degrees on its center axis. The rover's wheels are part of the rocker-bogie mechanism, which will be explained in the next section.
- **Computing unit:** The computing elements are found inside the body. These elements are usually duplicated so that there is always a spare one in case of an error or to compare computing results. These computers tolerate extreme radiation and are responsible for sending commands and data to other components. The computers also help with monitoring the rover's temperature and power level, as well as storing all the necessary data that is to be sent back to earth when requested.
- **Head:** On top of the body, there is a mast that holds the rover's head, which has cameras used for taking pictures of the view from an elevated perspective. It also has other cameras used only for navigation purposes. The head can rotate and tilt to easily change the perspective without needing to move the whole rover.
- **Sensors:** Several sensors are used to monitor the environmental condition of Mars as well the condition of the computers, electronics, and power of the rover. The MEDA [3] (Mars Environmental Dynamics Analyzer) includes a barometer, humidity sensor, wind sensor and temperature sensors.
- **Robotic arm:** It is foldable and has several joints, offering flexibility. The tip of the arm has sensors, cameras, and tools to extract geological samples.

### 3.1.2 Rocker-bogie mechanism

One of the key features of the Curiosity and Perseverance Mars rovers, as well as some previous models, is the rocker-bogie mechanism they employ. Replicating this rocker-bogie mechanism was one of the mechanical requirements of the project, so it was interesting to do some research about this mechanism.

Rocker-bogie [4] is a suspension mechanism developed by NASA in 1988 for its rover model Sojourner. Thanks to its capabilities of maintaining stability while traversing rough terrains, this

mechanism has been NASA's favorite and has also been implemented in the more recent rover models like Curiosity and Perseverance. The mechanism is formed by two main components, the rocker and the bogie; each "leg" of the rover has these two components.

The rocker component is connected to the rover's body and to the other rocker using a differential. These rockers rotate in an opposed manner respective to each other, meaning that when one of the rocker goes up, the other goes down. Each rocker has the bogie component attached to one end and the drive wheel attached to the other end.

The bogie component has a drive wheel attached to each of the ends and has a free pivot point in the middle that connects to an end of the rocker component.

Some of the main advantages of the mechanism are:

- Ability to withstand a tilt of 50 degrees in any direction without overturning.
- Ability to climb over obstacles with vertical heights up to twice of its wheel diameter.
- Thanks to the differential, when one leg climbs an obstacle that is higher than the one the other leg is climbing, the leg climbing this lower obstacle is pressed downwards, ensuring that all the 6 wheels are touching the ground at all times and therefore ensuring traction and control.
- A complex transmission system is unnecessary, as each wheel has its own motor, simplifying the power transfer mechanism.

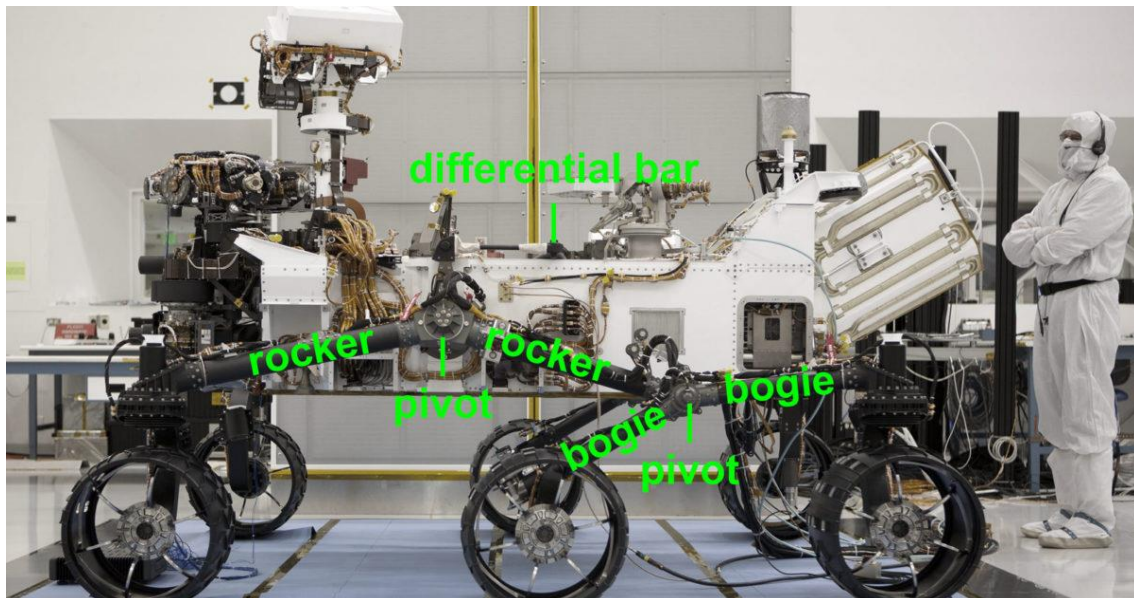


Figure 3. Rocker-bogie mechanism [5]

### 3.1.3 Ackermann steering

The Ackermann steering geometry [6] [7] is a method of steering that is implemented in almost every vehicle that has steerable wheels, from a common car to a Mars rover, and helps avoid tire slipping. Since implementing Ackermann steering is one of the software requirements of the project, it was interesting to do some research on this subject.

When making a turn, the wheels on the inside of the turn travel a shorter distance than the wheels on the outside, so if we want to avoid tire slipping, the inside wheels have to be placed at a tighter angle. In the case of cars and other common vehicles, we only have to worry about the two steerable wheels, but in Mars rovers we have four steerable wheels, making the steering geometry a bit more complicated.

Focusing on how Ackermann steering works on Mars rovers, there are two issues that need addressing: the direction and the speed of each of the corner wheels. As for the direction, each turning wheels needs to align their center axis towards the desired turning point of the whole rover, as seen in **Figure 4**.

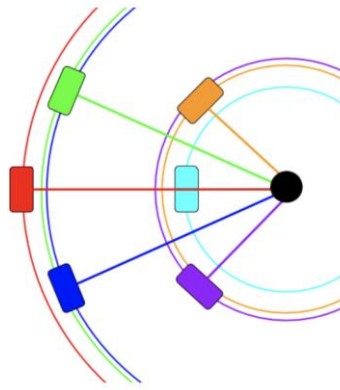


Figure 4. Ackermann steering turning point [7]

Variables need to be created to represent the different angles, distances and turn radius of the platform. In **Figure 5** these variables can be seen.

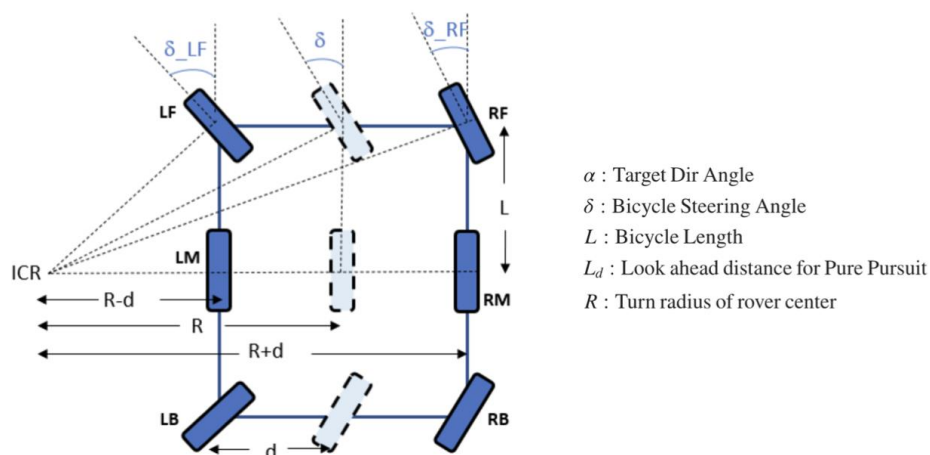


Figure 5. Ackermann steering rover variables [6]



Given a turning radius and the fixed rover's dimensions, to calculate the steering angle of each of the wheels some equations are obtained based on the Ackermann steering geometry.

$$\delta_{LF} = \tan^{-1}\left(\frac{L}{R-d}\right) \quad \delta_{LB} = -\delta_{LF}$$

$$\delta_{RF} = \tan^{-1}\left(\frac{L}{R+d}\right) \quad \delta_{RB} = -\delta_{RF}$$

Where ( $\delta_{LF}$ ,  $\delta_{LB}$ ,  $\delta_{RF}$ ,  $\delta_{LB}$ ) are the steering angles for each of the turning wheels. In **Figure 5** a term named ICR (Instantaneous Center of Rotation) appears, which indicates the center of the imaginary turning circle of the rover.

As for the speed, since the difference in the speeds of the outer wheels when turning are minimal, given a turning radius and the fixed rover's dimensions, only three equations are obtained based on the Ackermann steering geometry.

$$V_{innerFrontBackWheels} = V_{rover} * \frac{(\sqrt{L^2 + (R-d)^2})}{R+d}$$

$$V_{innerMiddleWheel} = V_{rover} * \frac{R-d}{R+d}$$

$$V_{outerWheels} = V_{rover}$$

Where ( $V_{outerWheels}$ ) is the speed of all the wheels outside the turn, ( $V_{innerMiddleWheel}$ ) is the speed of the middle wheel inside the turn, ( $V_{innerFrontBackWheels}$ ) is the speed of the front and back wheels inside the turn and ( $V_{rover}$ ) is the speed of the whole rover.

Using these speed equations, assuming a forward right turn at full speed and with a low turning radius, each of the wheel speeds would approximately be:

- Left wheels (LF, LM, LB): 100% speed
- Right-Front and Right-Back wheels (RF, RB): 60% speed
- Right-Middle wheel (RM): 50% speed

As seen in the previous example, the inner wheels will always be slower than the outer wheels. Regarding the angles, the angles of the inner rotating wheels will always be higher than the angles of the outer rotating wheels.

It is also interesting to mention that the distance between the middle wheels of a rover is often a bit larger than the distance between the front or back wheels. This means that a new distance variable would need to be added on **Figure 5**, and the equations would have to be slightly modified if it was necessary to be very precise. A similar thing happens with the "L" distance, as the distance between the front and middle wheels may be a bit different than the distance

between the back and middle wheels. Despite these facts, as the differences in the wheel angles and speeds calculations would be minimal, the explanation of this chapter has been simplified without taking this into account.

All of the equations and images shown in this Ackermann section have been extracted from a sample project on MATLAB's website [6] and an educational document from NASA's JPL (Jet Propulsion Laboratory) [7], with some modifications and improvements to make them clearer. In turn, these two pages have obtained the equations from the original Ackermann steering equations but adapting the original equations to the concept of a Mars rover with a six-wheeled platform and four steerable wheels.

## 3.2 Open-source definition

One of the objectives of the project is to make it open-source [8], so it is important to research and explain what open-source really means.

Open-source projects are the ones that people can freely modify and share because their inner designs are publicly available and no restrictive licensing is used. Although the term open-source was originally used just for software, nowadays it is a much broader term and it also includes other fields like the more hardware related ones.

Projects where its inner contents (e.g., code, architectural designs...) are not publicly available are called "proprietary" or "closed-source" projects. In these types of projects, often only one individual or organization has access and control over the inner contents, and a very restrictive license is used to ensure that users of the project will not do anything (e.g., modify the project) that the original creators of the project have not expressly permitted.

It is important not to always associate the term open-source with the fact that some project or software is free of charge. The implications of open-source go beyond the economic factor, and there can be open-source projects that require monetary contributions in order to be used, especially if technical support or other similar services are required.

Some of the benefits usually associated with open-source projects [8], especially software ones, are:

- **Control:** Experienced users often like to access the code to ensure that it is not doing anything that is not supposed to do (e.g., gathering excessive user data) and to change or improve parts they do not like.
- **Security:** As anyone can view the source code, these types of projects are considered more secure because vulnerabilities are often quickly identified by the community and fixes are also usually quicker. In lots of cases, the original creators of the project do not have to fix the vulnerabilities themselves, as the community build around the project takes care of this.
- **Stability:** Many proprietary projects are killed after a few years because they stop being profitable or because the original company that created them was closed. With open-

source projects, users can rely on the community to maintain the software long-term, as long as it is deemed useful.

- **Community:** As stated in many of the previous points, useful or interesting open-source projects are easily able to build a community of developers and users around them. In proprietary projects this community may also exist, but they are usually not able to make contributions to the project, as is the case with open-source projects.

There are several websites to host the projects so that they can be accessible to the rest of the world. One of the most known and used websites for this purpose, especially in the software world, is GitHub [18] . Besides being a platform where users can host their code and other project contents, GitHub allows other users to actively collaborate in the projects; improving the code, reporting bugs and creating new functionalities suggestions. Projects uploaded to GitHub are stored in repositories using Git, which is a Version Control System (VCS). This means that if changes are made to the project files, a “Git commit” can be made to start tracking and managing the changes.

### 3.3 Similar existing projects

Once the project’s general idea was defined, one of the first steps consisted in doing an exhaustive exploration of existing open-source DIY robotic projects, with the aim of identifying the most valuable ideas they offered and the limitations they presented. During this exploration, several projects that developed Mars rover-like platforms were found. Although sharing a common theme, each of these projects had different set of functionalities and a very different approach for developing the robotic platform, mechanical structure, hardware, and software.

The most interesting open-source projects that also developed a Mars rover platform are listed below. For each project, a list of valuable contributions and identified shortcomings is provided.

#### **DIY Mars Perseverance Rover Replica [9]**

- **Valuable contributions:**
  - A very detailed, 3D printable, functional and modular design.
  - Detailed documentation and videos for mechanical assembly.
  - Published design files that allow an easy modification of the 3D printable structure.
  - Detailed BOM (Bill Of Materials).
- **Shortcomings:**
  - Very limited functionalities, only basic motor and servomotor functions are implemented.
  - Does not have a functional robotic arm.
  - Remote control that only uses a generic transmitter and that has no telemetry.
  - Low quality camera that uses an analog FPV module as a transmitter.
  - Only one type of development board is used (Arduino).
  - Poor exterior wire management.

- Inexistent code modulation.
- No sensors.

### **Jakob Krantz Mars Rover [10]**

- Valuable contributions:
  - 3D printable functional design.
  - Custom 3D printed remote control with an ESP32 that is able to connect via WiFi or LORA to the rover.
  - The RC generates an AP where a phone can connect to view telemetry from the Rover on a website.
  - GitHub with code and documentation.
  - Detailed BOM (Bill Of Materials).
  - Sensors for temperature, acceleration and gyroscope.
  - Good code modulation.
  - 4-axis foldable robotic arm.
- Shortcomings:
  - Functional, but not very detailed 3D design.
  - No documentation for mechanical assembly.
  - Lack of humidity, pressure, and altitude sensors.
  - Very basic UI for displaying all the telemetry and camera view. Webserver has very few resources as it is being hosted on an ESP32.
  - Instead of having a fixed screen on the RC, there is the need of having a mobile phone screen to display the telemetry and camera view.
  - Only one type of development board is used (ESP32).
  - Website is hosted on the remote control, which is small and has a tiny battery. This limits the webserver capabilities and the battery life of the RC.
  - RC commands are limited by the physical switches and joysticks (no customizable touchscreen for control, which would allow easy expansion of functionalities).

### **NASA's JPL Open-Source Rover [11]**

- Valuable contributions:
  - Sturdy aluminum construction, without 3D printed parts.
  - GitHub with code, very detailed documentation, and an assembly manual.
  - Detailed BOM (Bill Of Materials).
  - RaspberryPi as the rover's brain, allowing versatility as it is quite powerful.
  - Several ways of communication with the robot (Bluetooth, WiFi, Serial...).
  - Strong community built around the project, with several people having constructed the rover model and many more making code contributions.
  - Use of ROS (Robot Operating System), making the project very educational as it is a very powerful library.

- Shortcomings:
  - Functional design, but does not replicate the look of a real rover model.
  - Due to the aluminum construction, it ends up being a bit expensive, with an estimated cost of over 1500€.
  - Only one type of development board is used (RaspberryPi).
  - No mention of a physical RC.
  - No camera, although it could be easily added thanks to the RaspberryPi board.
  - No robotic arm.
  - No sensors.

### 3.3.1 Distinctive characteristics of OpenRover

Having explored and analyzed several of these open-source robotics projects, it was then possible to point out what valuable contributions could differentiate the OpenRover platform developed in this final degree project from other apparently similar robotic projects.

The OpenRover is a complex project that has a broad scope, so only its key distinctive characteristics will be highlighted here:

- Use of different development boards among its main functional modules and interactions between these modules: Many of the explored projects were very complex but only used a single type of developing board, reducing the educational value of the project. By using several development boards that interact with each another, the optimal usecase for each board can be appreciated, and more functionalities can be added.
- Custom Remote Control with touchscreen that is expandable and is not limited by the hardware: Some of the explored projects used a physical RC with switches and joysticks that could easily become obsolete if the project functionalities wanted to be expanded. By adding a touchscreen to the RC, more controls and monitorization parameters can be easily added without the need for more hardware.
- Visually appealing web monitoring dashboard that is self-hosted on the rover and where users can monitor many status parameters of the rover, as well as take a look at the camera feed: Although some of the projects generated websites using development boards like the ESP32, these websites were very basic as they were restricted by the limited resources available in these type of development boards.

For more detailed information about the expected contributions of the project, refer to the **Objectives** and **Requirements definition** sections.

## 4 Practical framework

Once the initial research phase was concluded, it was then possible to clearly define how the solution would be implemented. Among other things, this practical phase includes a definition of the solution, with the selected components, and a detailed explanation of all the mechanical, hardware and software work carried out throughout the project.

### 4.1 Solution proposition

As stated in the **Objectives** section, *“The main objective is to develop an open-source robotic platform build upon the 3D printed model of the Mars rover that was developed as a personal project”*. Although the mechanical aspect is not fully inside the scope of this final degree project, it is still interesting to mention what resources were used to build the mechanical platform and what additions were made.

The OpenRover’s 3D design of the main mechanical platform originates from the *DIY Mars Perseverance Rover Replica* [9] project, and the 3D design of the on-board robotic arm originates from the *Jakob Krantz Mars Rover* [10] project. These two projects are some of the ones listed in the **Similar existing projects** section of this document, where a list of shortcomings and valuable contributions was made for each of them. Several modifications were made to this original 3D designs, specially to integrate the robotic arm into the main body of the rover. These modifications will be discussed in the **Mechanical structure implementation** section of the project.

The *DIY Mars Perseverance Rover Replica* [9] project was selected as the starting point for the 3D design primarily due to its highly detailed, functional, and modular design. Moreover, the original project offered very limited hardware and software capabilities, making the project an interesting foundation to build upon, creating all the hardware and software aspects from scratch.

In the subsections inside this **Solution proposition** section, the different selected hardware and software components for OpenRover will be detailed and justified. These components are some of the ones presented in the **Hardware components analysis** and **Software technologies and libraries analysis** sections, so a more in-detail explanation for each of them can be found in those sections.

#### 4.1.1 Selected hardware components

For each hardware category present in the **Hardware components analysis** section, the selected hardware for OpenRover will be listed. It is important to remember that simpler electronic components categories were not included in the hardware analysis and therefore will also not be included here. In the annexes of this document, there will be a list with most of the components and their cost.

- Development boards: Arduino Mega for rover’s body, controlling all moving elements, sensors, and radio transceiver. ESP32S3 for embedded screen. Arduino Nano for RC PPM value processing and controlling radio transceiver. Raspberry Pi Zero 2W for rover’s head camera and webserver.
- Touchscreen: ESP32S3-8048.

- Camera: RaspberryPi Camera Module 3 Wide.
- Battery: 11.1V LiPo battery with 6000mAh and 50C.
- Wireless communication module: NRF24L01+.
- Environmental sensor: BME280.
- Motors: 12V 30RPM Brushed motors with 11 kgf·cm and 2.2A of current draw. DRV8871 motor driver.
- Servomotors: MG996R/MG995.
- Stepper motor: 12V bipolar stepper with 45 N·cm, 1.5A of current draw and NEMA17. faceplate. A4988 stepper driver.
- Generic remote control: Spektrum DX8.
- Wired communication protocol between development boards: UART.

### 4.1.2 Selected software technologies and libraries

For each software category present in the **Software technologies and libraries analysis** section, the selected software technologies and libraries for OpenRover will be listed.

- Development environment: PlatformIO IDE with Arduino framework.
- Operating system: Raspberry Pi OS (headless) for the Raspberry Pi Zero 2W board.
- Wireless communication: Arduino RF24 library.
- Wired communication: Arduino SerialTransfer and Python SerialTransfer libraries.
- Touchscreen: Arduino GFX and LVGL libraries. SquareLine Studio platform.
- Camera: Python Picamera2 library.
- Motors: Arduino PalatisSoftPWM library.
- Servomotors: Arduino Servo library.
- Stepper motor: Arduino StepperDriver library.
- Environmental sensor: Adafruit/Arduino BME280 library.
- Generic remote control: Arduino RcTrainer library.
- Website: Flask and Bootstrap.

## 4.2 Hardware and software selection rationale

In this section, the selected hardware, software technologies and libraries will be justified. Comparisons will be made with alternatives listed on the **Software technologies and libraries analysis** and **Hardware components analysis** sections, highlighting the advantages and disadvantages of the chosen selection.

### 4.2.1 Hardware components rationale

#### Development boards

The development board present in the main rover's body has to be responsible for managing all the moving elements (motors, servomotors, stepper) as well as receiving data from the sensors and using a radio module for communication, among other things. This means that a huge number of pins will be necessary to connect all this hardware. Therefore, the Arduino Mega is an excellent candidate, as it offers 54 digital and 12 analog pins, and was selected as the development board for this module. Another candidate could be the Arduino Due, which has the same amount of pins and comes with higher processing power, but it is a little more

easier to maintain and expand, while not posing many limitations for the specific project's use case.

- Although Express.js is known for being easy to use, Flask offers an even simpler setup process, especially for web applications that do not require sophisticated features.

To complement Flask, which helps with the backend, Bootstrap was selected as a frontend framework. It provides a large set of styled UI components and a responsive grid system, allowing for the quick development of visually appealing and responsive interfaces. This is particularly valuable in a project like OpenRover, as the web dashboard is just one of multiple components. By using Bootstrap, the frontend development time is reduced without sacrificing a clean interface, enabling more focus on other functionalities that are more logic related.

### 4.3 Mechanical structure implementation

To implement the mechanical structure, for each of the rover's modules the same procedure was followed:

- 1) 3D designing: Starting from scratch or modifying an existing 3D design, the mechanical components that are going to be 3D printed are designed using apps like Fusion360. In order to do the designs, accurate measurements are taken of other components (e.g., electronic boards, bolts, motors) that will use the 3D printed parts.
- 2) Slicing: The 3D design of a component is exported to an STL format and is imported into a slicing app like UltiMaker Cura. Depending on the shape of the design, strength needs, and visual appearance wanted, different configuration parameters are set in the slicing app. From the design and configuration parameters, the app generates a GCODE file, which is directly used by 3D printers to print the design. This GCODE file contains the 3D printer extruder's movements, fan speed, filament flow... for each of the layers.
- 3) 3D printing: The GCODE file is inputted into the 3D printer using an SD card. The printing bed and extruder are heated up, the filament is fed into the extruder, and the 3D printing process starts, with a very variable duration (from a few minutes to several hours for each 3D printed component, depending on size and quality).
- 4) Assembly: Using all the 3D printed components and other mechanical components which can be purchased at hardware stores (e.g., PVC tubes, nuts, bolts, aluminum profiles), the rover's main chassis, head and the RC transmitter cases are assembled.

All the 3D printing and assembly was done in a small at-home workshop, using a consumer-grade 3D printer and common and inexpensive tools that are often used for many DIY projects. Although the exact number of 3D printing hours was not recorded, it is estimated that it took approximately 400 hours to 3D print all the rover's components. As a matter of fact, if sample 3D printed pieces used for testing the designs, and final 3D printed pieces that failed mid print were considered, approximately 100 hours would need to be added to the printing hours count.



Since the mechanical aspect of the rover is not fully inside the scope of this final degree project, which puts more focus on the hardware and software aspects, only one image of the modified rover 3D design and of the final 3D printed and assembled mechanical structure will be shown here, without detailing the designing, 3D printing and assembling steps.



**Figure 6.** Rover modified design and assembled mechanical structure (own elaboration of images and original design extracted from [9] )

The original rover's central mechanical platform design is from the *DIY Mars Perseverance Rover Replica* [9] . The design shown in **Figure 6** is a customized version of this design, with some improvements that are detailed in **Annex I**.

An explanation of the different designing, 3D printing, and assembling steps of the rover's mechanical structure is present in **Annex I**. This explanation is organized on a module-by-module basis and is complemented with several images of the rover's mechanical platform implementation.

## 4.4 Hardware implementation

This section explains how the selected electronic hardware components interact with each other and describes the process followed to design and create all the custom prototyping PCBs and electrical connections for the rover. The hardware design was made taking into account the specifications defined on the **Requirements definition** section of the project, with the aim of achieving the desired set of functionalities for the rover.

The implementation of the hardware was done on a module-by-module basis, starting with the central rover module, continuing with the RC and touchscreen module, and ending with the camera and webserver module.

For each of the modules, some initial tests were carried out using prototyping boards and simple connections between the electronic hardware components, usually using DuPont jumper wires. Many of these hardware tests also required software test programs, therefore some of the conducted tests related to hardware will be detailed in the **Software implementation** section.

The hardware schematics, which detail the components and its connections, will also be shown in this section. Every time a hardware test for a module was successfully completed, prior to

disassembling the used prototyping board and/or electrical connections, a design that replicated the used connections was added to the schematics. These schematics could then be used as a guide to create the definitive connections between the components. The components that did not have accessible male/female pin headers, or that required a compact size with close proximity and connections to other components, were soldered onto prototyping PCBs.

#### 4.4.1 Central rover module

The hardware present on the central rover module includes an Arduino Mega development board that is responsible for managing all the moving elements (e.g., servomotors, motors, stepper motor), reading the environmental sensor's values and sending/receiving communications via serial (to/from RaspberryPi Zero board) and via radio (to/from custom RC).

##### 4.4.1.1 Schematic

In the schematic seen in **Figure 7**, all the hardware present on this module can be seen. Besides the components listed in the **Selected hardware components** section, some other components are present, which have more basic functions but are still important:

- Buck converters: Used to transform the rover's battery output voltage to a specified lower and more stable voltage level. Essential if only a single battery wants to be used but there are electronic components with different voltage ratings.
- Switches: Allow easy startup/shutdown. Also used to change between operating modes (e.g., debugging/normal mode) and as a travel limit sensor.
- Diode: Ensures that current flows only in one direction, protecting from reverse polarity.
- Buzzer: Used to indicate audible alerts.
- Resistors: Used to divide voltages.
- JST connectors: Standard type of Male/Female connectors, used to allow an easy plugging/unplugging and ensuring the correct polarity of the connections.
- Relay: Used as an electronic switch to connect/disconnect power of some of the circuits.

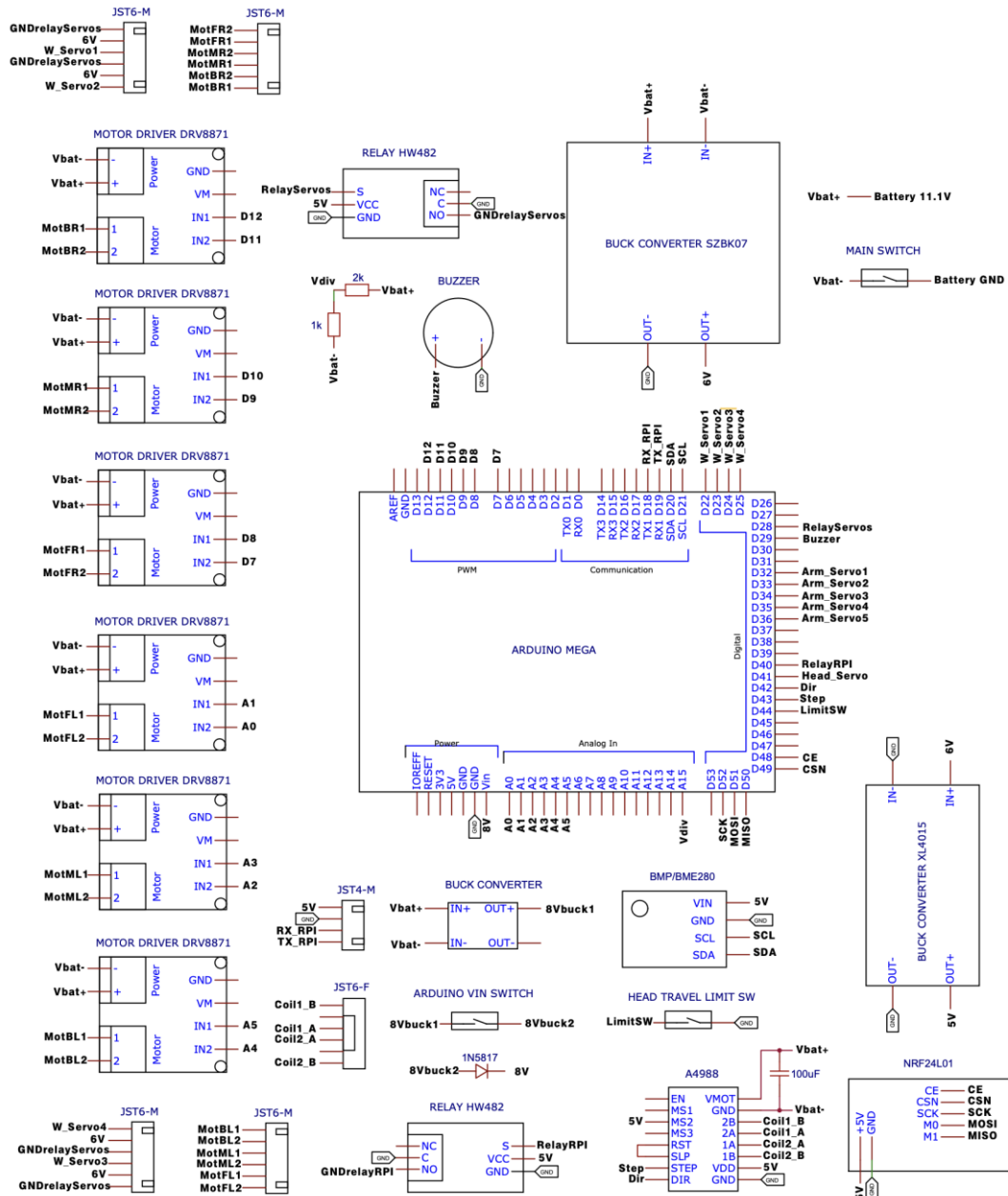


Figure 7. Central rover module schematic (own elaboration)

### Schematic component remarks

The single 11.1V LiPo battery of the rover is connected to the input of the main buck converter (SZBK07), which lowers the voltage to 6V. The servomotors are directly connected to this main buck converter output as they need 6V to operate at full power. As most of the other hardware components of the rover work below 6V and receive power from this main buck converter, it has a high current rating of 20A to support the load without overheating. It is also important to mention that the power wires had to be dimensioned to support the required maximum currents, using 22AWG wires (max. 5A) for the motors/servomotors and 14AWG (max. 20A) wires for the input/output of the main buck converter. The only components that do not receive power from the main buck converter are the motor drivers, voltage divider and the buck converter used to power the Arduino, which are directly connected to the 11.1V. The buck

converter used to power the Arduino lowers the voltage to 8V, which is within the 7-12V range that the VIN pin, which has an onboard power regulator, supports. Another buck converter is used to lower the voltage from 6V (output of main buck converter) to 5V, as most of the used electronic components are rated to operate at this voltage.

The used 11.1V LiPo battery has three 3.7V cells connected in series. These cells do not stay static at the rated 3.7V, but at a slightly higher/lower voltage value depending on the charge level (4.2V when full, 3.2V when empty). A voltage divider is used to divide the battery's voltage by three, so that the cell voltage is directly available for reading. As this cell voltage reading will always stay under 4.2V (cell fully charged), it can be read directly using the Arduino ADC, which can read up to a 5V analog value. The combination of the voltage divider and the Arduino's ADCs acts as a sensor to monitor the battery level, which was one of the hardware requirements.

As the rover's mechanical platform is quite large, disassembly of some of the parts (e.g., mast with head, rocker-bogie arms with the wheels) from the main body section is necessary to facilitate the rover's transportation. To easily connect/disconnect the hardware wiring of the different sections, JST connectors with power and control signal wires are used for motors, servomotors, stepper motors and also for the connection between the Arduino and the head's RaspberryPi Zero 2W board.

For some of the components of the rover, like the servomotors and the RaspberryPi, it is necessary to have the ability to decide via software if they want to be powered or not. Relays are used to implement this functionality, connecting or disconnecting the ground signal of the components on command. They can be set to a Normally Open/Closed (NO/NC) configuration.

#### 4.4.1.2 Wiring connections and soldering

Before implementing any electronics inside the rover's body, the wiring of the chassis was done, with all the wires of the motors and servomotors ending up inside the body. This was a crucial first step, as it laid the foundation for the electrical system and made it possible to perform initial functionality tests on the motors and servomotors. All the chassis wiring was wrapped using a black braided sleeved that help group all the cables and improved overall cable management.

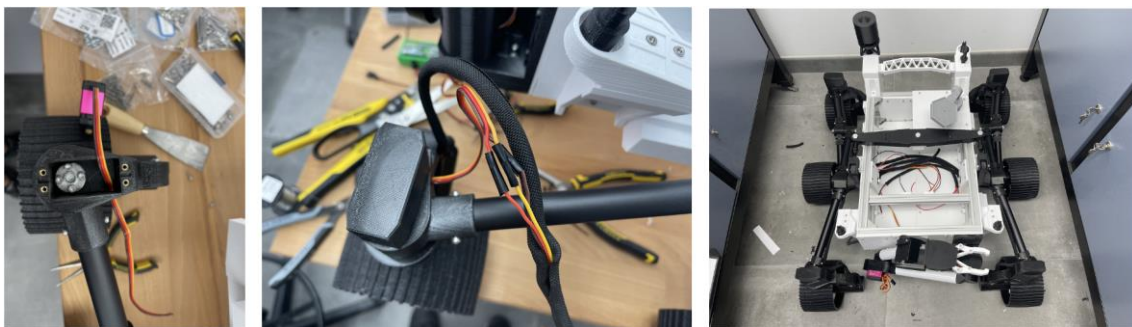
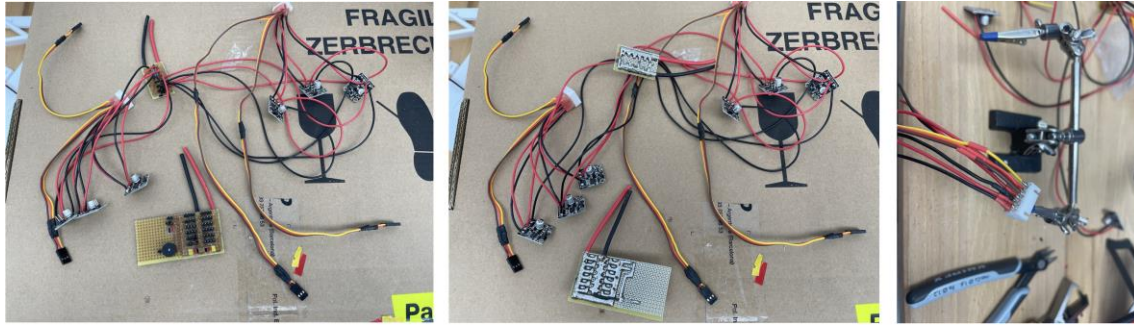


Figure 8. Central rover module chassis wiring (own elaboration)

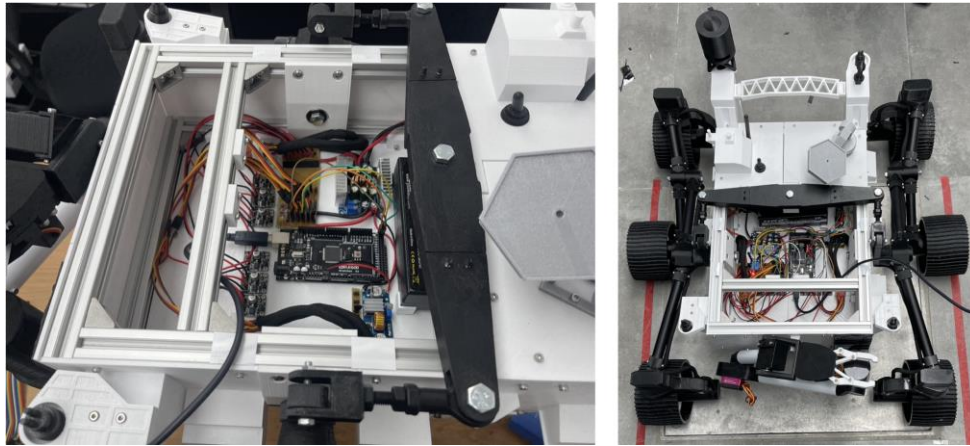
The next step was soldering all the power-related PCBs. This custom PCBs helped distributing the power from a single battery to a lot of components. In **Figure 9**, the soldered PCBs that distribute power to the motor drivers and servomotors are shown, along with an image of the custom JST connectors that were soldered to facilitate connections for the motors and servos.





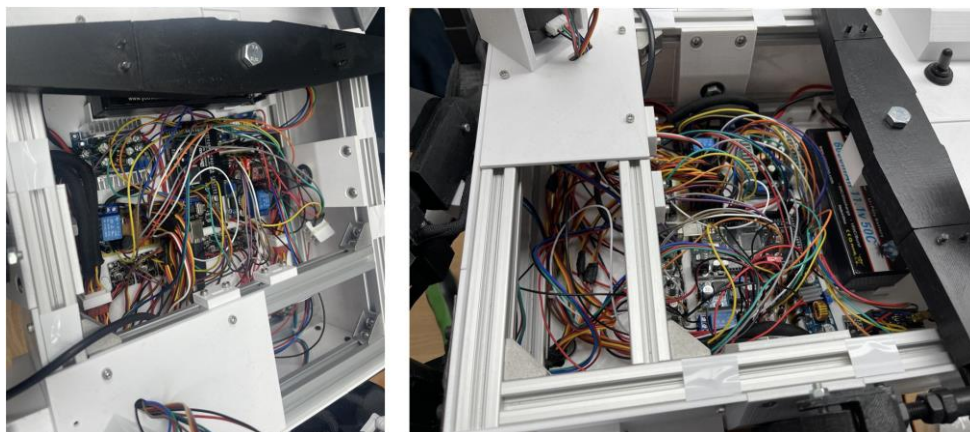
**Figure 9. Rover power distribution PCBs (own elaboration)**

Once the power distribution boards were completed, they were placed inside the body of the rover, along with the connected motor drivers. Other essential components like buck converters, Arduino Mega, NRF24 radio module and battery were then added in order to start testing, with basic software programs, the motors and servomotors sending remote commands.



**Figure 10. Initial rover component connections (own elaboration)**

Finally, the remaining hardware components of the central rover module were added, like the environmental sensor, stepper motor, stepper driver and relays. Connections between these components and the existing components were made, mainly using DuPont jumper cables, as many of the newly added components had accessible pin headers. In **Figure 11**, the final component connections are shown, which are the same ones detailed on **Figure 7** schematic.



**Figure 11. Final rover component connections (own elaboration)**

#### 4.4.1.3 Problems and solutions

During the hardware design and implementation of this module, some issues were encountered. Below are some of the key problems and the corresponding solutions that were applied:

- In some cases, while the rover's battery is disconnected, it is necessary to connect the Arduino development board to a computer via USB for debugging (e.g., reading a sensor's values and printing them on computer's serial) or programming purposes. In this scenario, instead of the Arduino receiving power from the battery through the power wires (which are connected to the VIN/GND Arduino pins), it receives power from the USB and outputs some of it via the VIN/GND pins. As this may cause problems, a diode is used to ensure no power is outputted from the VIN pins. Another possible scenario is when all the components, including the Arduino, are receiving power from the battery but we also want to connect the Arduino USB to a computer to debug. In this scenario, an external switch in the VIN pin is used to ensure that, while debugging, the Arduino is being powered by the USB and still has a common ground with the rest of the components, which are being powered up by the battery.
- Most servomotors need to constantly receive PWM signals to maintain their position and behave properly. When the rover's switch is turned on and power is given to the Arduino Microcontroller and servomotors, there is a lapse of time where the servomotors are receiving power but are not receiving PWM signals, or are receiving erratic signals as the microcontroller is booting up. In some cases, this caused the servos to jitter and to move to random positions during the firsts instants after powering the rover, which posed a problem. For this reason, a relay put on NO (Normally Open) was used to connect/disconnect the servomotors from the power source, ensuring that they were only connected when the microcontroller was ready and sent a signal to the relay.

#### 4.4.2 Remote control and touchscreen module

The hardware present on the remote control and touchscreen module includes an ESP32S3 development board that is responsible for managing the screen and an Arduino Nano board that helps processing the RC PPM signals and controlling the radio transceiver. The generic RC unit, on which the custom radio and touchscreen module are mounted, can also be considered a hardware component. However, the only modification made to it involves adapting and utilizing its internal battery to power the custom module.

##### 4.4.2.1 Schematic

In the schematic seen in **Figure 12**, all the hardware present on this module can be seen. Besides the components listed in the **Selected hardware components** section, some other components are present, which have more basic functions but are still important. As some of these other components were explained on the previous central rover module, here they will only be listed:

- Buck converter.
- Switch.
- Diode.
- Buzzer.
- Resistors.

- JST connectors.
- 3.5mm Jack: Used to connect to the trainer port of the generic transmitter.
- Level shifter: Used to convert signals from one voltage level to another, allowing devices with different operating voltages (e.g., 3.3V and 5V) to communicate with each other.

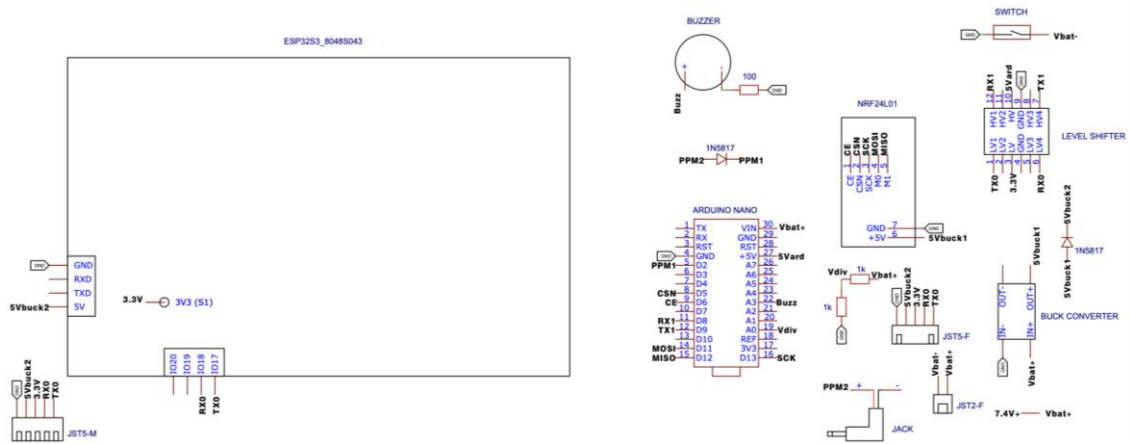


Figure 12. Remote control and touchscreen module schematic (own elaboration)

### Schematic component remarks

In the schematic shown in **Figure 12**, the generic Spektrum DX8 RC does not appear as an entity, as the only interactions with the developed custom module are through the trainer port, represented in the schematic by the 3.5mm jack that connects to the port, and the shared battery, whose voltage output is also represented in the schematic.

The Spektrum DX8 RC has an internal 7.4V Li-Ion battery [42]. This battery is directly connected to the only buck converter of the custom module, which lowers the voltage to 5V. The only component that is powered directly by the battery's 7.4V is the Arduino Nano, as it has a voltage regulator with an input range of 7-12V in the VIN pin.

The used 7.4V Li-Ion battery has two 3.7V cells connected in series and behaves similarly to a LiPo battery [36]. As it was the case with the rover's LiPo battery, this Li-Ion battery cells do not stay static at the rated 3.7V, but at a slightly higher/lower voltage depending on the battery level. A voltage divider is used to divide the battery's voltage by two, so that the cell voltage can be read. The combination of the Arduino's ADCs and the voltage divider acts as a battery level sensor, which was one of the hardware requirements for the custom RC.

In the schematic, an ESP32S3 SoC does not appear as an entity, as it is integrated in the same package as the touchscreen. Instead of showing all the typical pins of an ESP32S3, only the available pins that are not used to run the touchscreen are shown. The touchscreen came with some pre-soldered JST connectors for easy interfacing with the available GPIO pins.

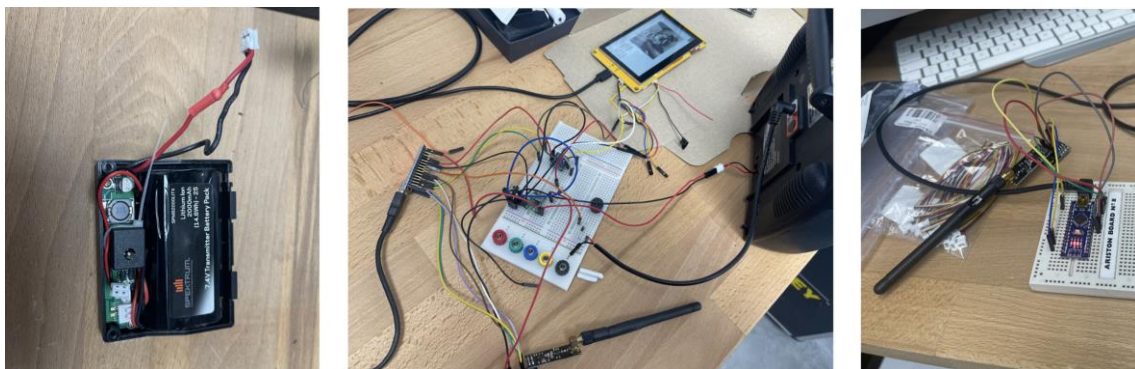
There are two diodes in the custom RC module. One diode is used simply as a reverse-polarity safety feature to protect the trainer port connection with the Spektrum DX8 RC. The other diode is used to ensure that when the ESP32S3-8048 board is being powered by the USB, no voltage is outputted from the 5V pin of the board, as it is directly connected to the output of the buck converter (which should not receive voltage).

The level shifter is used for serial communication between the Arduino Nano, which has 5V logic, and the ESP32S3, which has 3.3V logic. The LV (low voltage) and HV (high voltage) reference pins of the level shifter board are directly connected to the 3.3V and 5V pins of the ESP32S3 and the Arduino, respectively. This ensures that the voltage reference values are the same as the ones the boards use.

Although the schematic showcases all the components together, in the physical implementation they are divided into two sections: the touchscreen section and the radio control section. The touchscreen section is only formed by the ESP32S3-8048, and the radio control section is formed by the rest of components. These sections are part of the same “Remote control and touchscreen module” and are interconnected using JST connectors with power and data wires. The physical separation of the sections can be seen in **Figure 54**.

#### 4.4.2.2 Wiring connections and soldering

One of the first steps of the RC and touchscreen module implementation was creating all the necessary connections between the hardware components on a prototyping board, as seen in the central image of **Figure 13**. This way, it was possible to do various tests to check if the hardware design would work prior to soldering the components and creating the definitive connections between them. Additionally, for this first step, a second connector was soldered to the internal battery of the Spektrum DX8 RC, allowing the battery to power both the Spektrum RC and the custom developed RC and touchscreen module. Creating this second connector early in the process was essential to verify whether the internal battery could supply enough current to power everything, and to confirm that the PPM signal could be accurately read with both the Spektrum RC and the custom module sharing the same power source. To validate the correct functioning of the NRF24L01 radio module that was on the prototyping board, a second prototyping board with another Arduino Nano and NRF24L01 was used to carry out some tests. This second prototyping board, whose design is not present in the **Figure 12** schematic as it was created only for testing purposes, can be seen on the rightmost image of **Figure 13**.



**Figure 13. Remote control and touchscreen prototyping board (own elaboration)**

Once the correct functioning of the hardware was validated and the final schematic was created, a prototyping PCB that would be used to solder all the components except the screen was cut to size. This custom created PCB can be seen in **Figure 14**. For the remote control and touchscreen module, it was important to have as many connections made with soldering as possible. These types of connections take up less space and are more resistant to movements,



which is a big advantage considering that the custom RC module must be very compact and will be subject to a lot of movements while it is being used and handled.



Figure 14. Remote control custom PCB (own elaboration)

The only soldering that had to be done to the touchscreen module was related to creating the custom JST connectors with the appropriate wire length to connect the touchscreen, which is at the front of the Spektrum RC, with the custom remote control PCB which is at the back. In **Figure 54**, the connection between these two sections can be seen, as well as how the final result of the whole module looks like when the 3D printed cases are used.

#### 4.4.2.3 Problems and solutions

During the hardware design and implementation of this module, some issues were encountered. Initially, this module's design did not include the Arduino Nano board, as all the components were supposed to be connected directly to the ESP32S3 board, which on paper is capable of handling all of them. The touchscreen used up nearly all the ESP32S3 GPIOs, leaving only five free. However, eight available GPIOs were required for all the module's functionalities (five for NRF24 handling, one for PPM signal reading, one for battery level reading and one for the buzzer). As the ESP32S3 could not accommodate all these inputs and outputs, a solution to the problem was found, which consisted in using an additional Arduino Nano board to handle all I/O-related tasks. This Arduino Nano would bidirectionally communicate with the ESP32S3 via the serial interface, resolving the GPIO number limitation and offloading I/O responsibilities.

### 4.4.3 Camera and webserver module

The hardware present on the camera and webserver module includes a RaspberryPi Zero 2W board that is responsible for receiving camera input, creating a WiFi network with a self-hosted website available in it and sending/receiving communications via serial (to/from Arduino Mega).

#### 4.4.3.1 Schematic

In the schematic seen in **Figure 15**, all the hardware present on this module is shown. Besides the components listed in the **Selected hardware components** section, some other components are present, which have more basic functions but are still important:

- USB HUB HAT [89] : Connects to RaspberryPi Zero's Pogo pins to extend the single micro-USB of the board to four functional USB A ports.
- USB to TTL [90] : Helps connecting devices with TTL logic levels, like microcontrollers, with PCs or other devices that use USB, enabling data communication between them.

- Fans: Used for active cooling, reducing the RaspberryPi's CPU temperature.

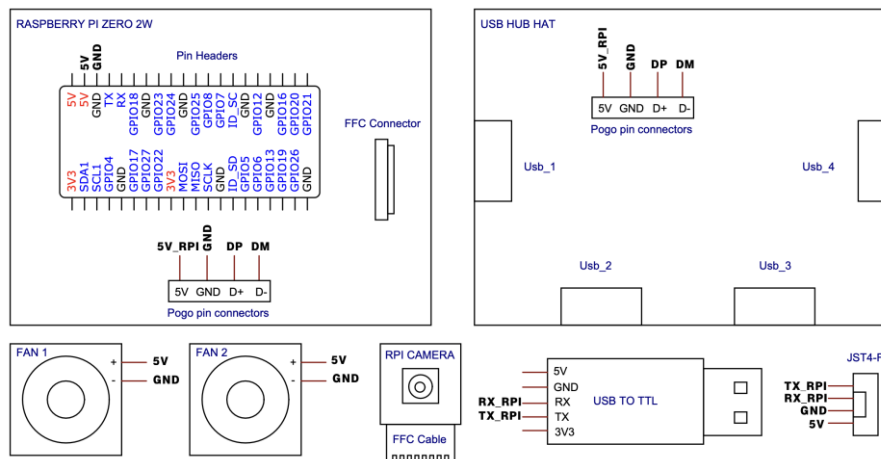


Figure 15. Camera and webserver module schematic (own elaboration)

### Schematic component remarks

The four-pin female JST connector that is shown in **Figure 15** is connected directly to the four-pin male JST connector that is present in the central rover module. Using this connection, the camera and webserver module receives 5V power and can bidirectionally communicate with the Arduino Mega of the central rover module via serial.

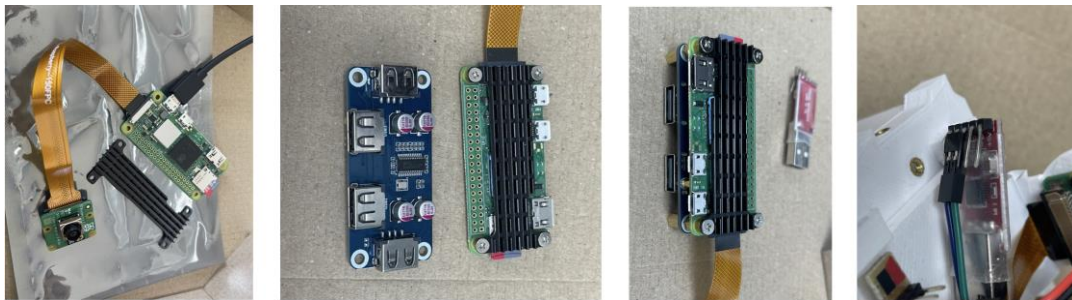
For the serial communication, a USB to TTL converter is used to connect a USB interface of the Raspberry Pi Zero to one of the TX/RX serial interfaces of the Arduino Mega. This converter is necessary because USB and TTL serial (UART, TX/RX) are different standards. Instead of using this converter, the TX/RX pins of the Arduino Mega could be directly connected to the RX/TX pins that are available on the RaspberryPi Zero, but this approach is usually less optimal. One of the reasons behind this is that the voltage levels of the RaspberryPi and the Arduino are different (3.3V and 5V respectively) so an additional “level shifter” board would also be needed. Another reason is that there is only one TX/RX interface available on the RaspberryPi Zero, whereas multiple USBs can be available if a USB Hub is used, making it possible to have several communication interfaces.

The RaspberryPi Zero board only features a single micro-USB port, but it includes some pads at the bottom side that expose the D+ and D- USB connections. These allow for the connection of USB Hub boards, which expand the number of available USB ports. These USB Hubs come in various formats, with some designed to match the RaspberryPi Zero form factor, offering an integrated and compact solution for expanding USBs. This is the case of the selected USB HUB HAT for the project, which sits beneath the RaspberryPi and adds four USB-A ports. HAT stands for “Hardware Attached on Top”, which means that the board is an add-on for a specific RaspberryPi model and easily connects to the GPIO or bottom pads layout of the board. This USB HUB HAT is essential to the project since USB to TTL converters generally use USB-A rather than micro-USB. Additionally, it provides flexibility to connect other USB peripherals to the RaspberryPi Zero, such as USB memory sticks or USB antennas to increase WiFi range.

As the selected camera is fully compatible with the Zero board, a FFC (Flat Flexible Cable) is used to directly connect the camera to the RaspberryPi Zero board.

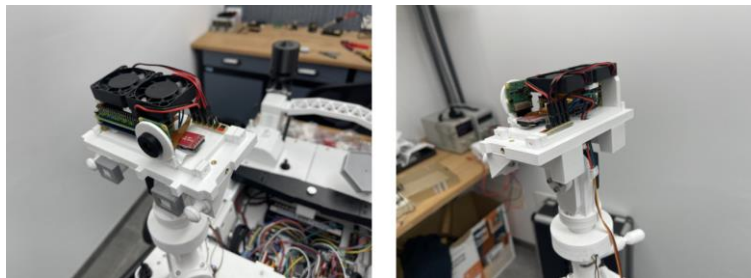
#### 4.4.3.2 Wiring connections and soldering

The wiring connections and soldering for this module were rather straightforward, as little soldering was needed and there were not many connections between the components. That said, the main challenge was managing the limited space available; wires had to be cut to precise lengths to minimize clutter and the components needed to be distributed following an optimal layout. The first steps of the module's wiring process consisted in connecting the FFC between the camera and the RaspberryPi board, coupling the USB HUB HAT to the board using its Pogo pins, and connecting RX/TX from the JST connector to RX/TX of the USB to TTL converter. This process is shown in **Figure 16**.



**Figure 16.** Initial camera and webserver module connections (own elaboration)

Once these initial connections were made, the USB to TTL was plugged into one of the ports of the USB HUB HAT and both the fans and the RaspberryPi board were connected to the 5V power source that the JST connector provided. All the definitive connections are shown in **Figure 17**.



**Figure 17.** Final camera and webserver module connections (own elaboration)

In **Figure 56**, more images of the component's placement and wiring can be seen.

#### 4.4.3.3 Problems and solutions

During the implementation of this module, some issues were encountered. When the RaspberryPi Zero was powered up and the first testing programs were executed, after some time the performance of the board started to decrease. Doing some research on the internet [91] and conducting tests running programs with different processor needs, it was clear that this performance decline was caused by the CPU reaching its maximum temperature, which caused the board to throttle down. To solve this problem, a heatsink was added for passive cooling and two fans were added for active cooling, ensuring that the CPU did not reach its maximum temperature no matter the load.

## 4.5 Software implementation

This section explains how, using the selected software technologies and libraries, the different software modules of the OpenRover project were implemented, and how they interact with each other. The software design was made considering the specifications defined on the **Requirements definition** section of the project, with the aim of achieving the desired set of functionalities for the rover.

As the project modules are fairly independent, the implementation of the software was also done on a module-by-module basis, starting with the central rover module, continuing with the RC and touchscreen module, and ending with the camera and webserver module.

The codebase of the project is very large, and it would be impossible to show and explain each of the implemented individual functions in this document. Instead, diagrams are presented to illustrate the flow of the different programs, what files are they composed of and the interactions between these files.

One of the requirements of the OpenRover project was having a modular software design, organizing the implementation of functionalities into separate files, each responsible for a specific aspect of the system. This was considered for the software development of all modules.

The main programming languages used for the modules are C++ and Python. Many of the used libraries follow OOP (Object-oriented programming), requiring the creation of objects to access their functionalities, but most of the code written for the modules does not follow OOP. For the camera and webserver module, HTML, CSS and JavaScript are also used.

Since the bulk of the code does not follow OOP, class diagrams illustrating all the classes and their relations would not be the best option. Instead, diagrams detailing the created files are presented. In the case of C++, all the created header (.h), which have the functions headers that other files can call, and implementation (.cpp) files, are presented. In the case of Python, all the “.py” files, which include function headers and implementation, are presented. The diagrams also illustrate the relations between these files, indicating which files invoke functions from other files. Each of the “.cpp” and “.py” files implement the functions that are responsible for a specific aspect of the system (e.g., serial communication, managing motors, radio communication). In the case of C++, each “.cpp” file usually includes a “.h” file with the same name, which has the headers of functions present in the “.cpp” that can be invoked from other files that include this “.h” file.

### 4.5.1 Central rover module

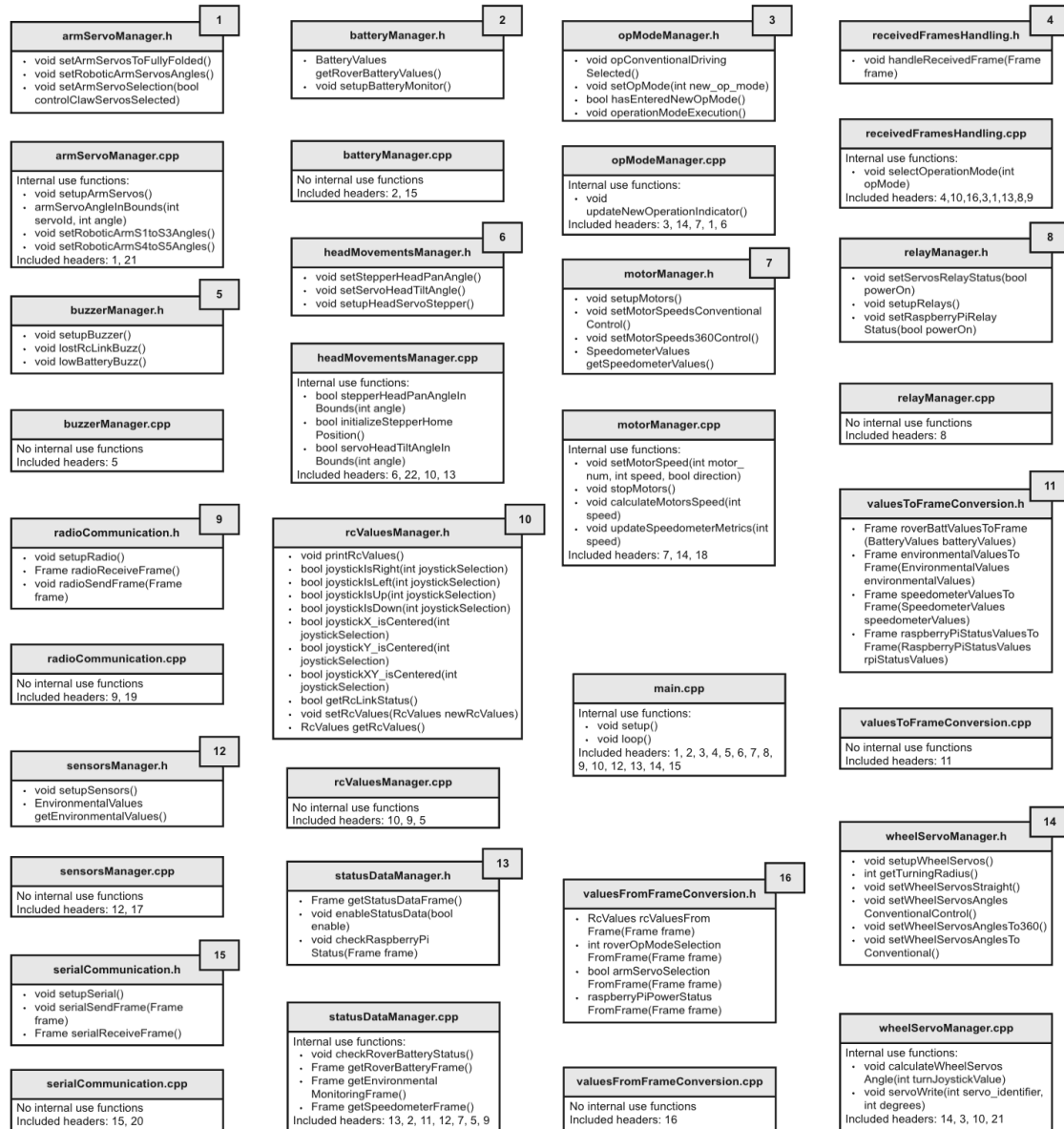
The software present in the central rover module is responsible for controlling all the moving elements (e.g., servomotors), reading the environmental sensor’s values, sending/receiving communications via serial and via radio and implementing the logic for many of the rover’s functionalities (e.g., controlling robotic arm, 360° turn control), among other things.

The software of this module has been developed using C++ and PlatformIO IDE [14]. All the code has been designed to run and has been tested on an Arduino Mega microcontroller board.

#### 4.5.1.1 Program structure

In **Figure 18**, a diagram detailing the most important created files for the module and its relations is shown. The function headers present in these files are also detailed in this diagram. For the function implementation files (.cpp), only the headers of the internal use functions are detailed, which are not present in the header files (.h) with the same names as the “.cpp” files. Internal use functions are all those functions which cannot be invoked from other files, as their purpose is to implement parts of the logic behind the functions present in the header files.

##### Key header (.h) and implementation (.c) files developed



##### Key library header (.h) files used



**Figure 18. Key header and implementation files for central rover module (own elaboration)**

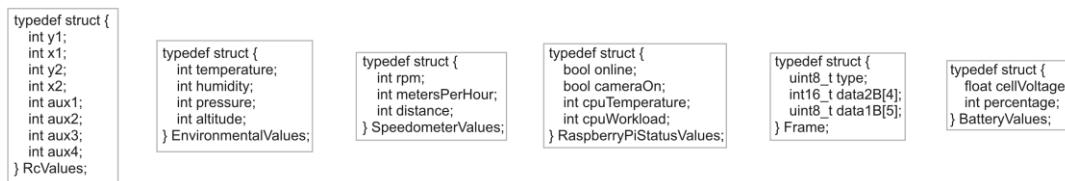
Each developed “.cpp” file implements functions dedicated to a specific aspect of the system. Below is a description of the main responsibilities assigned to each of these files:

- `armServoManager.cpp`: Facilitates the control of the robotic arm servomotors, modifying the angles at a constant speed depending on joysticks positions. Makes sure that the angles of each of the axis are within the limits.
- `batteryManager.cpp`: Implements calculations to convert the read analog values from the voltage divider to numeric cell voltage and percentage values.
- `buzzerManager.cpp`: Provides a set of buzzes with different tones and durations.
- `headMovementsManager.cpp`: Facilitates the control of the head's stepper motor and servomotor, modifying the pan and tilt angles at a constant speed depending on joystick positions. Makes sure that the pan and tilt angles are within the limits. Initializes the stepper's home position by reading a limit switch that indicates the maximum pan angle.
- `motorManager.cpp`: Facilitates the control of the wheel motors, modifying the individual speed and direction depending on joystick positions and selected operation mode. Implements Ackermann steering calculations to adjust each speed depending on the desired turning radius.
- `opModeManager.cpp`: Facilitates the change of the current operation mode and calls required procedures at the appropriate intervals, depending on operation mode.
- `radioCommunication.cpp`: Facilitates receiving and sending custom frames via radio.
- `rcValuesManager.cpp`: Provides information of the joysticks position, translating received joystick numeric values to known positions (e.g., up, down, centered). Implements calculations to detect if the remote-control link is lost.
- `receivedFramesHandling.cpp`: Handles the operations that need to be done for each type of received radio or serial frame.
- `relayManager.cpp`: Simplifies the state selection (on/off) of all the relays.
- `sensorsManager.cpp`: Facilitates the reading of sensor's values, like the BME280's values.
- `serialCommunication.cpp`: Facilitates receiving and sending custom frames via serial.
- `statusDataManager.cpp`: Facilitates the retrieval of status data (e.g., environmental sensors, battery values) at the appropriate intervals. Implements calculations to detect if the serial link with the RaspberryPi is lost, indicating that it probably has shut down.
- `valuesFromFrameConversion.cpp`: Translates the frames of the custom communication protocol to known data structures, which are more easily used.

- valuesToFrameConversion.cpp: Translates known data structures to frames of the custom communication protocol, which are optimized for data transfer.
- wheelServoManager.cpp: Facilitates the control of the wheel's turn servomotors, modifying the angles at a constant speed depending on joysticks positions. Makes sure that the angles of wheel are within the limits. Implements Ackermann steering calculations to adjust each wheel's angle depending on the desired rover turning radius.

The “main.cpp” file does not contain any complex logic; its primary responsibility is handling the initial setup and making recurring calls to functions defined in the developed header files. This approach aligns with the requirement of using a modular software design.

Several of the function headers shown in **Figure 18** utilize custom data structures. These structures facilitate the storage, transfer, and manipulation of data throughout the different parts of the system. All the structures used in this module are defined on a header file, which is included by the files that require them. In **Figure 19**, these custom structures are detailed.

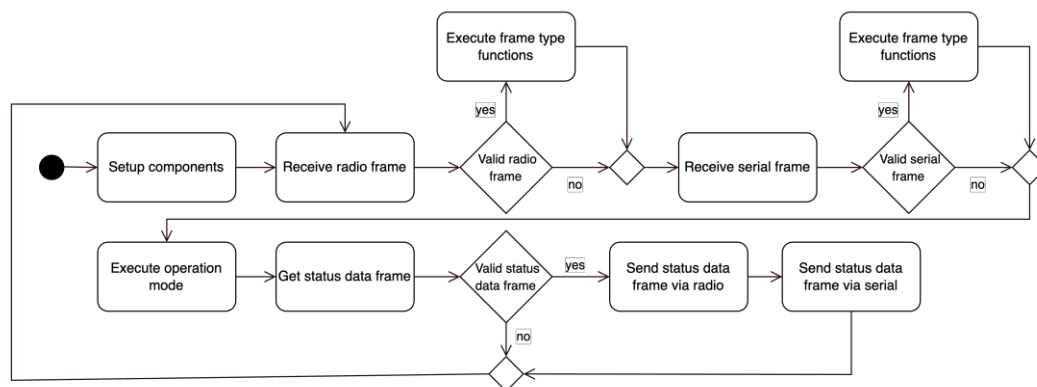


**Figure 19. Central rover module data structures (own elaboration)**

The structure names are self-explanatory, with the exception of the “Frame” structure, which would require a deeper explanation. This structure is integral to the custom communication protocol used to communicate between the OpenRover’s modules. Later in the document, a dedicated **Communication protocol** section will provide an explanation for this structure and the communication protocol.

#### 4.5.1.2 Program flow

In **Figure 20** an activity diagram of the module is shown. This diagram is only a general representation of the flow that the program follows, without explaining in detail how each of the steps in the flow are internally implemented.



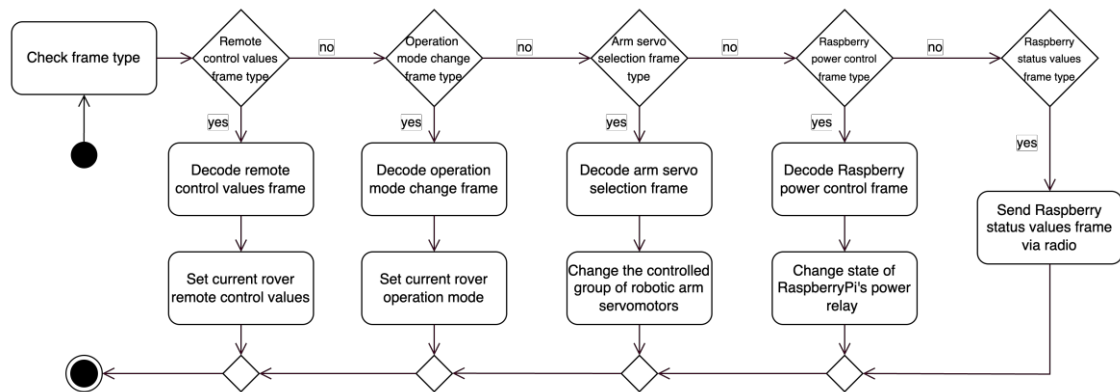
**Figure 20. Central rover module main activity diagram (own elaboration)**

As seen in **Figure 20**, once the “Setup components” activity is finished, the program follows a never-ending loop flow that starts/finishes on the “Receive radio frame” activity. Some activities are self-explanatory and not very complex (e.g., send status data via serial) but others are very general, with a lot of logic, and require a deeper explanation.

These more complex activities are:

- **Execute frame type functions:** From a received radio or serial frame, a “type” field can be obtained. This “type” field indicates the category of the received message, and is used to execute the functions that will be able to process the type of message and, if necessary, give a response.
- **Execute operation mode:** Depending on the set rover operation mode (e.g., robotic arm control, 360 degree turn control) functions are called to calculate and set the motors speeds and servos angles, among other things. These functions need to be continuously called, as the remote control values may change and therefore the calculated and set values will also change.
- **Get status data frame:** Each of the status data values (e.g., battery values, environmental values) has to be read and calculated with a certain frequency (e.g., every 2 seconds). In this activity, the status values are returned when it is time and they are valid.

To get a better understanding of this complex activities, activity diagrams of the subactivities inside the “Execute frame type functions”, “Execute operation mode” and “Get status data frame” activities are shown in **Figure 21**, **Figure 22**, **Figure 23**.



**Figure 21. Central rover module “Execute frame type functions” activity (own elaboration)**



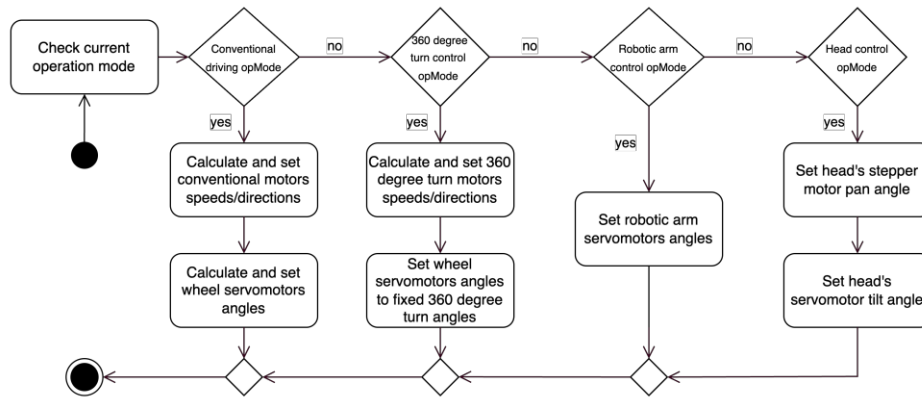


Figure 22. Central rover module “Execute operation mode” activity (own elaboration)

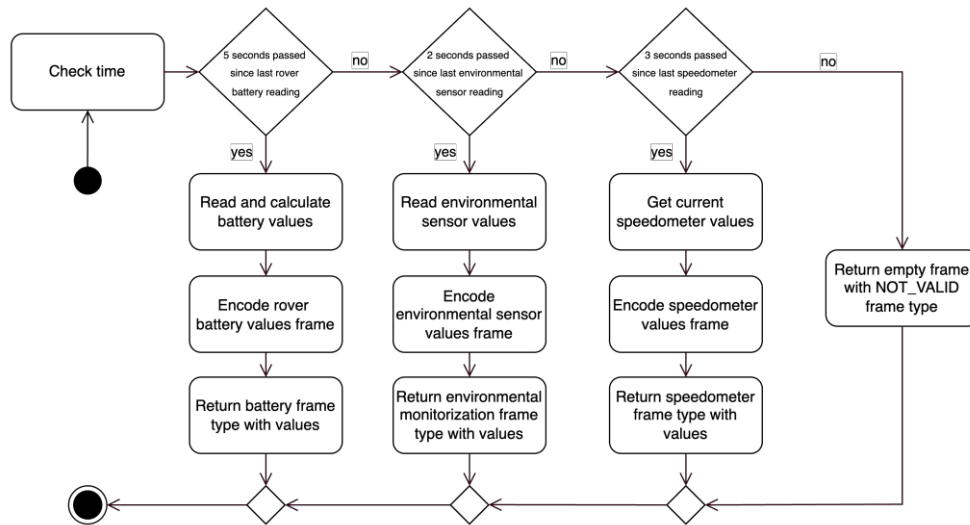


Figure 23. Central rover module “Get status data frame” activity (own elaboration)

#### 4.5.1.3 Tests performed

The main tests of this module were related to the implementation of the Ackermann steering geometry equations [6] [7] to control the speed of each of the motors depending on the desired turning radius.

The implemented equations were supposed to work perfectly on paper, but when the first practical tests were done, a slight “tire slipping” occurred on two of the wheels. This phenomenon was especially noticeable when the rover speed was high, and the desired turn was tight. Some additional tests were conducted, which consisted in sending the same PWM signal to all motor drivers, and then observing how the motors behaved with and without load. To test the motors behavior with load, the rover was placed on a flat surface and two different tire types were tried, one made with a rubber-like material and the other made with hard plastic. To test the motors behavior without load, the rover was suspended without the tires touching any surface. After these tests, it was concluded that some of the motors were slightly faster than the rest, and that physical factors like the tire’s material also had an effect on “tire slipping”, with the original hard plastic tires being more slippery.

This is why it was necessary to adjust the implemented Ackermann equations a bit and add some “physical correction factors”, which ended up reducing the “tire slipping” phenomenon significantly, but not eliminating it completely.

#### 4.5.1.4 Problems and solutions

During the implementation of this module, some issues were encountered. Below are some of the key problems and the corresponding solutions that were applied:

- Initially, pin 13 of the Arduino Mega board, which has hardware PWM, was used to control one of the motors’ speed via PWM, but whenever the board was powered up or reset, the motor started rotating for a brief moment. After doing some research, it was concluded that the board’s bootloader flashed pin 13 on boot and reset [84] , and therefore could not be used to control a motor or any other critical function. Thanks to the use of the selected “software PWM” [56] library, this motor was connected to an available non-hardware PWM pin.
- The selected “software PWM” library uses “Timer1” of the Arduino Mega to generate the PWM signals. The selected “Servo” [57] library takes control of several timers of the Arduino Mega by default (including “Timer 1”), in order to support a high number of servomotors. This posed a problem, and required the modification of some files of the “Servo” library to restrict its timer use only to “Timer 5”, which was not used by any other part of the system.

### 4.5.2 Remote control and touchscreen module

The software present in the remote control and touchscreen module is responsible for creating the embedded UI from which the rover will be monitored and controlled, reading the PPM channel values of the Spektrum DX8 RC, sending/receiving communications via serial and via radio, and implementing all the backend logic for the UI, among other things.

The software of this module has been developed using C++ and PlatformIO IDE. All the code has been designed to run and has been tested on an ESP32S3 or an Arduino Nano, depending on the specific section of the module.

As explained previously on the **Hardware implementation** section, one of the problems that were encountered while developing this module was that the ESP32S3 that manages the touchscreen does not have enough GPIOs to implement all the rest of the functionalities. Therefore, an Arduino Nano is used to help with some of these functionalities, such as reading the PPM signal’s channel values and communicating with the NRF24 radio module. For the module’s backend explanation, the explanation is divided between the “ESP32S3 and touchscreen” and “Arduino nano” sections, as they can be considered as submodules inside this module. These submodules communicate bidirectionally using serial.

#### 4.5.2.1 Embedded front-end development

Developing embedded UIs tends to be a very time-consuming and complex task. This is why, as explained in the **Software technologies and libraries rationale** section, the LVGL [15] library and the SquareLine Studio platform [54] were selected to accelerate the UI development process while maintaining a visually appealing and completely customizable design.

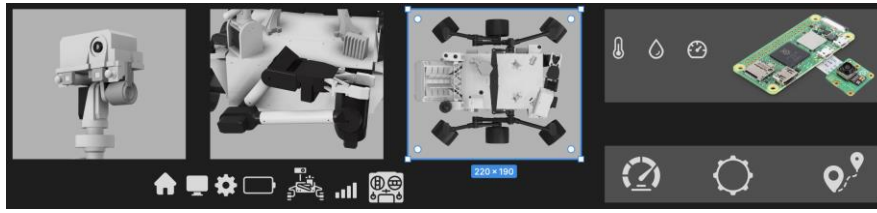
Before beginning the design process in SquareLine Studio, a conceptual planning was carried out to determine how all the necessary control and monitoring functionalities could be represented using appropriate and available UI elements (e.g., switches, status bars), as well as how these elements would be distributed in the screen or screens. After a thorough conceptual planning, and considering OpenRover's requirements as well as the available UI elements on SquareLine, it was concluded that three screens with the following purposes and elements would be created:

- **Control screen:** This screen contains all the interactive elements required to control the rover's functionalities and change between its operation modes. Toggle switches are used to enable/disable individual functionalities, while buttons allow the user to select a specific operation mode. The "conventional driving" operation mode does not have an associated button, as it is the default operation mode when no other mode is currently selected. As this is the primary interface used during rover operation, this screen is also known as the "Home screen" and is the first screen displayed when the remote controller is powered on. The speed control of the rover's motors and the angle of the servomotors are managed through the Spektrum DX8 RC joysticks, rather than via the touchscreen. Therefore, the touchscreen's "control screen" UI does not require any directional arrows or digital joysticks for these controls.
- **Monitor screen:** This screen contains all the non-interactive elements that display all the status parameters of the "Central rover" and "Camera and webserver" modules, as well as the current PPM channel values. Text boxes are used to display these channel values and other parameters like rover speed, distance, temperature, altitude, and RaspberryPi's CPU load. Visual elements like bars and arcs, which can display a value between known minimum and maximum parameter values, complement the information present in these text boxes. Other more complex visual elements like graphs are also used to display the last-minute readings of temperature and humidity.
- **Configuration screen:** This screen contains rover's functionalities which are not usually used, like trimming the wheel's servomotors angles if they are misaligned or disabling the motors to avoid involuntary activation. Many of the functionalities present in this screen are not implemented in the backend, as the main purpose of the screen is to be a place where future less commonly used functionalities can be added without needing to modify the other screens, which are tightly packed with key elements.

Even though there is a "monitor screen" where monitoring parameters are shown, there are some parameters like rover and RC battery status, radio signal, and warning messages that should be visible from other screens. To solve this problem, all the screens share a top panel where these parameters are displayed.

Once this initial conceptual planning was completed, websites such as Flaticon [85] were used to find suitable icons for the screen's elements. Figma [86] was then employed to resize these icons to the appropriate pixel dimensions, since SquareLine Studio requires images to match specific resolutions depending on the intended image size and the physical screen resolution. Additionally, Figma was also used to make minor modifications to some of the icons and to test

different screen layouts and element distributions. In **Figure 24**, the different icons and images that were resized and modified in Figma are shown.



**Figure 24. Embedded remote control UI icons in Figma (own elaboration)**

Once these icons and images were in Figma with their final size and look, it was possible to start the design process in SquareLine Studio. In **Figure 25**, all the components provided by SquareLine that were used in the OpenRover's touchscreen UIs are shown. These components are implemented using the LVGL library, and have a default visual theme and configuration. Although the components are provided, each one of them has dozens of custom configurations for changing its appearance and behavior, enabling the creation of GUIs that do not look standardized and that can meet the needs of any project. This is precisely what was done for the created touchscreen UIs, as all the default visual theme and configuration was changed, making the components and their behaviors totally different from the ones shown in **Figure 25**.



**Figure 25. SquareLine studio components with default theme and configuration (own elaboration)**

One of the used SquareLine components [92] was called “TabView”, and enabled the creation of two or more screens with a top or bottom panel for navigation between them. Additionally, this component also allows to change between screens by dragging left or right.

At the beginning of this **Embedded front-end development** section, the purpose and contents of each of the three developed screens are explained. In **Figure 26**, the final designs for these screens, developed in SquareLine Studio, are shown. Additionally, an image of the home screen in the physical touchscreen is also shown, detailing how the operation buttons change when one of them is clicked. Specifically, when one of these buttons is clicked, it is highlighted changing its color and adding a square around it, whereas the other buttons are disabled, making them darker. This highlight/darken and show/hide logic will be explained on the back-end sections.

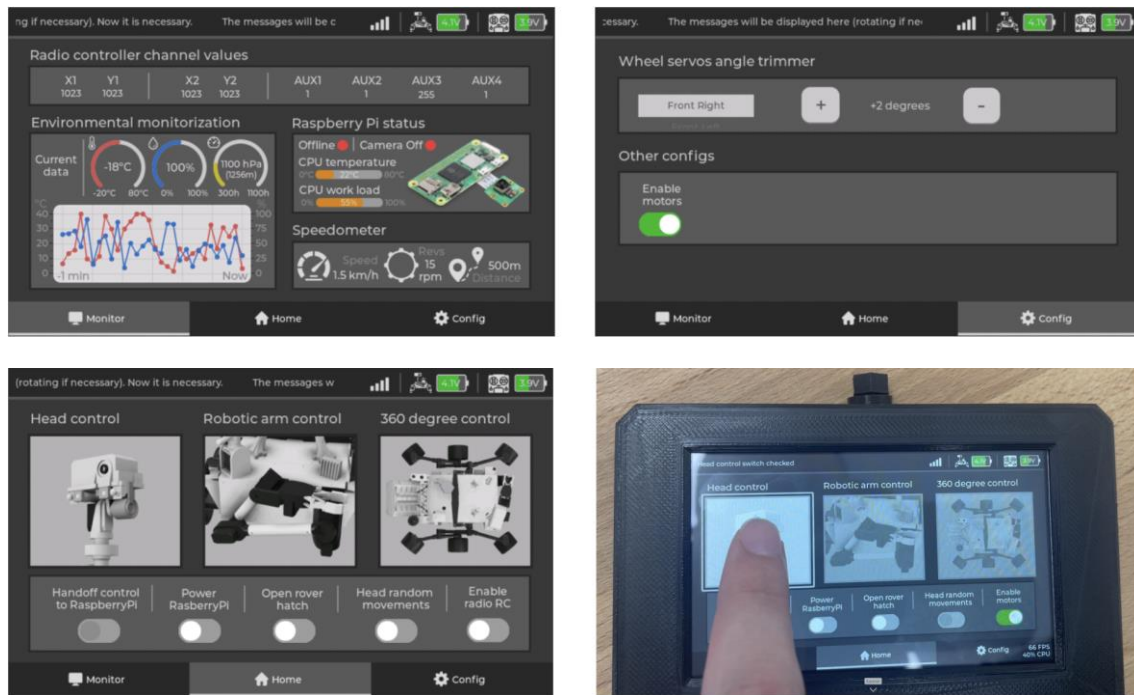


Figure 26. Final remote controller screens design (own elaboration)

It is important to mention that SquareLine Studio was only used to create the front-end, and that all the backend functionalities were coded with C++, using the LVGL library when interaction with the created UI elements was needed. In SquareLine, each UI element has a large set of parameters that can be modified, with the most significant being the “states” and “styles” parameters. The “states” parameter allows to define a current state for the element (e.g., checked/unchecked, enabled/disabled) and the “styles” parameter allows to define a custom style (e.g., colors, shapes, borders) for each of the “states”. This approach allows the backend of the application to modify an element’s appearance simply by changing its state, without needing to specify the desired individual style parameters. This can be seen in the image of the physical touchscreen shown in **Figure 26**, where if an operation mode button is pressed, it is highlighted to indicate its “checked state”. For the buttons and toggle switches, SquareLine also enabled the creation of events, where depending on a certain trigger action (e.g., checked, unchecked) a function header name could be specified. This function header name could then be used in backend to receive the events and implement the necessary code to handle them.

When the UI design in SquareLine was finished, the design was exported to a series of “.h” and “.c” files. These files contain all the styles, states, positions, shapes, LVGL components, fonts, and events of the UI elements. Additionally, for each image that was used in the design (e.g., home icon, operation mode images), a “.c” file with each of the pixel’s values is automatically generated.

All SquareLine Studio exported files were then added to a folder inside the PlatformIO project, to differentiate between the generated UI front-end files and the backend files that would be coded afterwards. One of the most important UI generated files is called “ui.h” and contains references to absolutely all the UI elements present in the design. This header file is included in every backend file that needs to interact with any UI element.

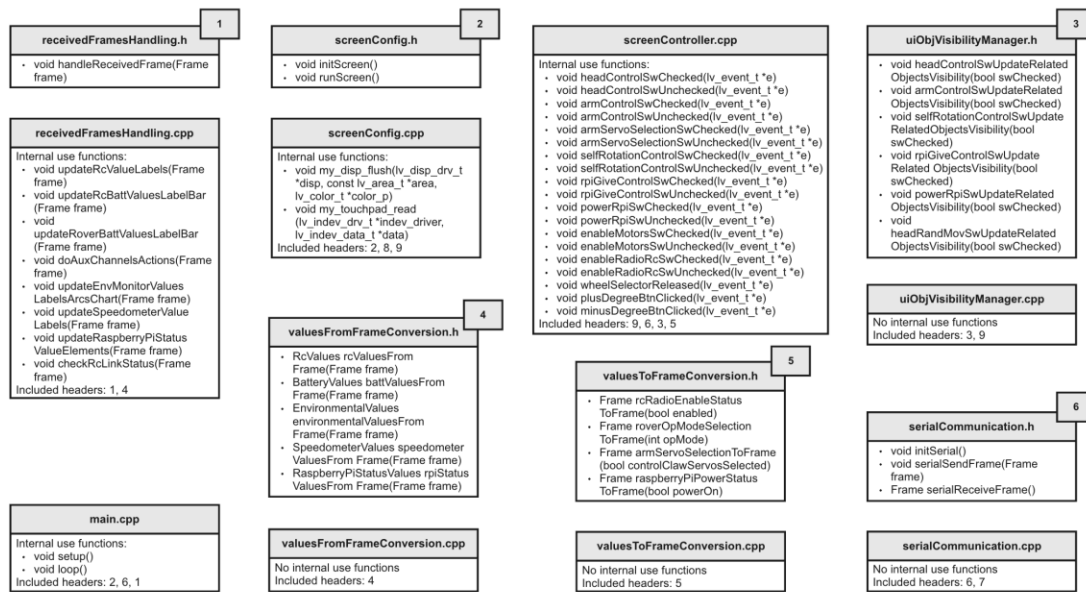
#### 4.5.2.2 Back-end program structure

The “ESP32S3 and touchscreen” and “Arduino nano” sections, can be considered as submodules inside this module, each one having a different program structure.

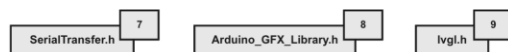
##### ESP32S3 and touchscreen section

In **Figure 27**, a diagram detailing the most important created files for the module and its relations is shown. The function headers present in these files are also detailed in this diagram. Refer to the **Program structure** section of the **Central rover module** for an explanation of what is included in this diagram, as the diagram content type is the same for both modules. The front-end “.c” and “.h” files exported from SquareLine Studio are not included in this diagram, as they have been automatically generated and do not contain any backend logic.

##### Key header (.h) and implementation (.c) files developed



##### Key library header (.h) files used



**Figure 27. Key header and implementation files for ESP32S3 and touchscreen submodule (own elaboration)**

Each developed “.cpp” file implements functions dedicated to a specific aspect of the system. Below is a description of the main responsibilities assigned to each of these files:

- receivedFramesHandling.cpp: Handles the operations that need to be done for each type of received serial frame.
- screenConfig.cpp: Has the setup code provided by the ESP32S3-8048 touchscreen manufacturer [93], which sets up the configurations needed for the screen’s driver and bus. Additionally, some initial UI values like the default “tabview” screen selection are specified in this file.

- `screenController.cpp`: Receives and handles all the events coming from the touchscreen's buttons, toggle switches and selectors. The logic to handle the events is implemented on other files, with the necessary function calls being made from this file.
- `uiObjVisibilityManager`: Handles how the visibility of various UI element changes depending on the clicked button or toggle switch (e.g., when one operation mode is selected, the other two operation modes visibility is shown as disabled).
- `valuesFromFrameConversion.cpp`: Translates the frames of the custom communication protocol to known data structures, which are more easily used.
- `valuesToFrameConversion.cpp`: Translates known data structures to frames of the custom communication protocol, which are optimized for data transfer.
- `serialCommunication.cpp`: Facilitates receiving and sending custom frames via serial.

The “`main.cpp`” file does not contain any complex logic; its primary responsibility is handling the initial setup and making recurring calls to functions defined in the developed header files.

Several of the function headers shown in **Figure 27** utilize custom data structures. These structures facilitate the storage, transfer, and manipulation of data throughout the different parts of the system. All the structures used in this module are defined on a header file, which is included by the files that require them. These structures are the same that were used in the central rover module, which are detailed on **Figure 19**.

### Arduino nano section

The Arduino Nano subsection has very minimal logic, as its only responsibilities are reading the PPM signal's channel values, communicating with the NRF24 radio module, communicating with the ESP32S3 via serial, reading the RC's battery values and managing a buzzer.

One of the goals for this submodule was to minimize the amount of logic implemented in the Arduino Nano, as the ESP32S3 is significantly more powerful, and it is more optimal and scalable to centralize the main codebase and processing on that board. Therefore, the Arduino Nano's program was designed primarily to act as a “communication relay”. In other words, most communication frames received by the Nano via radio or via serial are not processed locally but are instead directly forwarded to the ESP32S3 or the NRF24, depending on the case. This forwarding can be seen in **Figure 32**, where the frames received via serial are sent via radio and vice versa.

Given that the code for this submodule is relatively short and, more importantly, not intended to be scalable (as the Arduino Nano's main purpose is to fulfill this “communication relay” role), all the code is located on a single “`main.cpp`” file. This implies that, for this submodule, there is no file modularity. **Figure 28** shows the function headers present in this “`main.cpp`” file, as well as the headers of the libraries that are used.

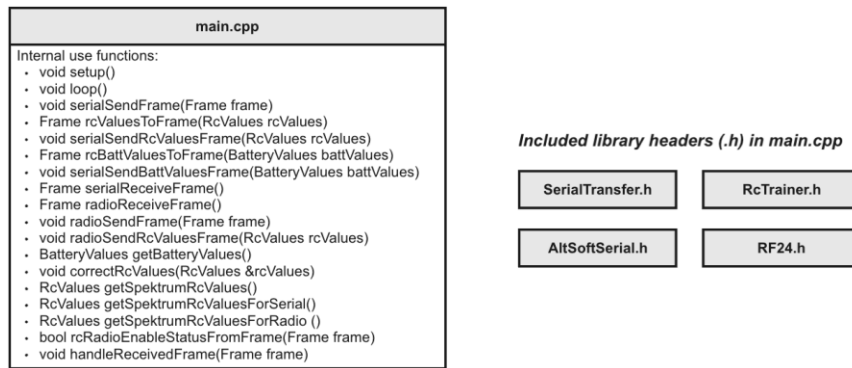


Figure 28. Library headers and main.cpp file for Arduino Nano submodule (own elaboration)

#### 4.5.2.3 Back-end program flow

The “ESP32S3 and touchscreen” and “Arduino nano” sections, can be considered as submodules inside this module, each one having a different program flow.

In **Figure 29** and **Figure 32**, an activity diagram of each submodule is shown. These diagrams are only a general representation of the flow that the programs follow, without explaining in detail how each of the steps in the flow are internally implemented.

#### ESP32S3 and touchscreen section

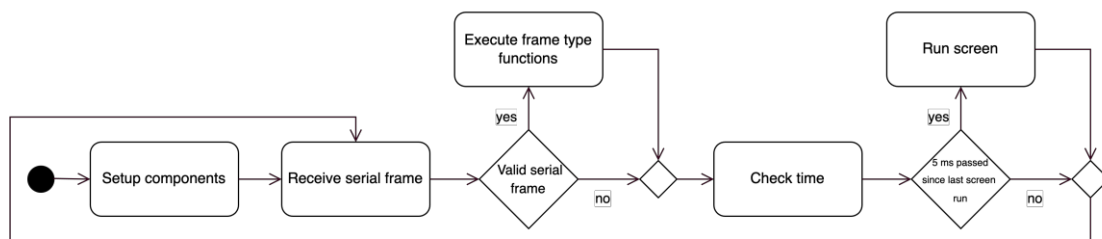


Figure 29. ESP32S3 and touchscreen submodule main activity diagram (own elaboration)

As seen in **Figure 29**, once the “Setup components” activity is finished, the program follows a never-ending loop flow that starts/finishes on the “Receive serial frame” activity. Some activities are self-explanatory and not very complex (e.g., receive serial frame) but others are very general, with a lot of logic, and require a deeper explanation:

- **Execute frame type functions:** From a received serial frame, a “type” field can be obtained. This “type” field indicates the category of the received message, and is used to execute the functions that will be able to process the type of message and, if necessary, give a response via serial.
- **Run screen:** Handles all the tasks related to LVGL, refreshing the UI and detecting clicks on some of the UI elements (e.g., buttons, toggle switches). These tasks are handled by the LVGL library, and the developers only need to call the “lv\_task\_handler()” function every 5 milliseconds or less to ensure a responsive touchscreen system.

To get a better understanding of the “Execute frame type functions” activity, in **Figure 30** an activity diagram with all its subactivities is shown.



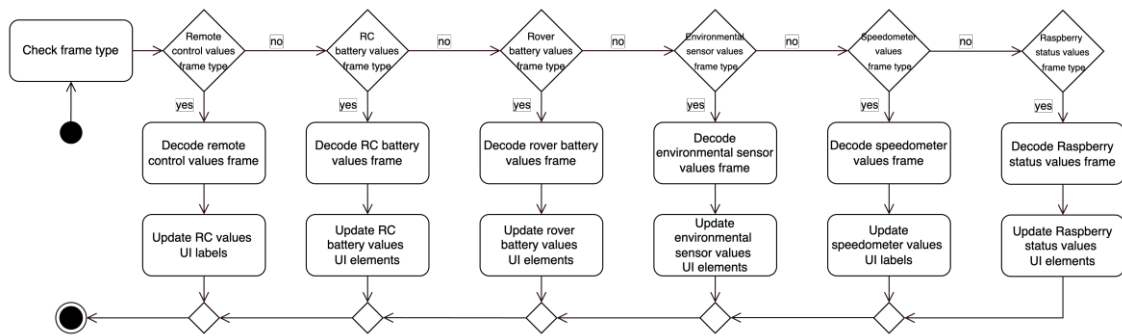


Figure 30. ESP32S3 and touchscreen submodule “Execute frame type functions” activity (own elaboration)

In **Figure 30**, the subactivities that update the “UI labels” only change the corresponding text that is displayed on the UI, while the subactivities that update the “UI elements” also change visual indicators like status bars, arcs, and charts.

As previously explained, the execution of the tasks in the “Run screen” activity of **Figure 29** is handled by the LVGL library. One of these tasks is detecting user clicks on some UI elements. When a click is detected, an LVGL “lv\_event\_t” is launched, and the functions present in the “screenController.cpp” file, seen in **Figure 27**, receive and handle these events. Although the handling of the events differs a bit depending on the specific button/switch clicked, the general flow that is followed is similar for most clicked button and toggle switches. This flow can be seen in **Figure 31**.

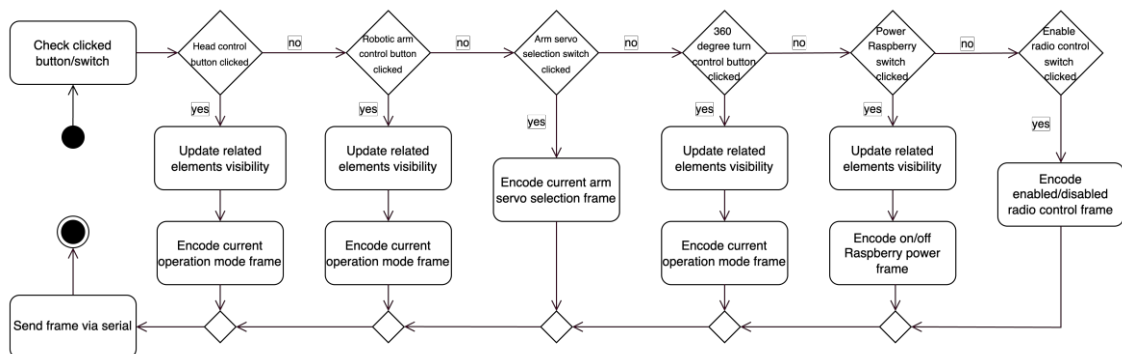


Figure 31. ESP32S3 and touchscreen submodule buttons and switches flow (own elaboration)

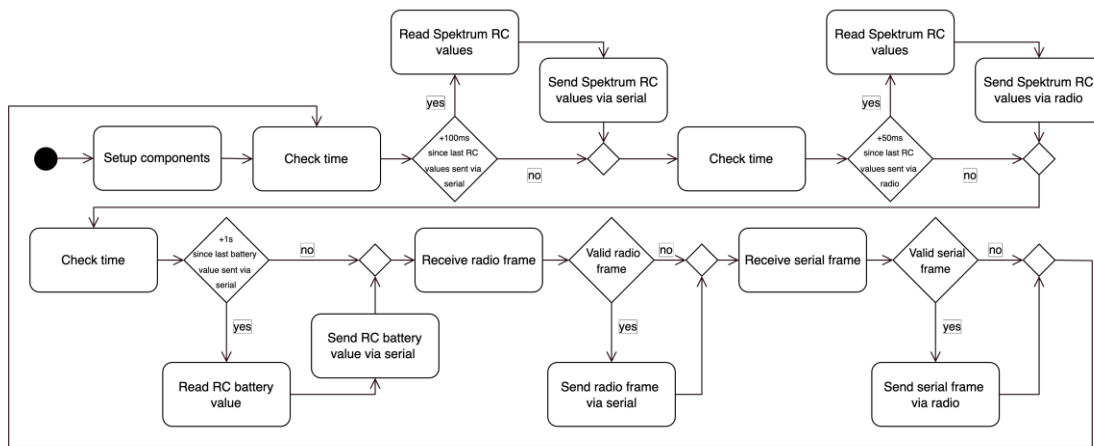
In the code, each click event is not directly a “button/switch click event” but a “button/switch checked event” or “button/switch unchecked event”. In **Figure 31**, the events are shown as a “button/switch click event” to simplify the diagram.

The logic to set the current operation mode that will be sent to the rover is specified in each “checked/unchecked” event of the operation mode buttons (head control, robotic arm control, 360 degree turn control). When one of these buttons is clicked, the corresponding operation mode is sent. When it is unclicked, the “conventional control” operation mode is sent, as it is the default one.

Only the buttons/switches where backend logic has been implemented are detailed in the **Figure 31** diagram. Other buttons/switches that are present in the UI but not implemented (e.g., open rover hatch, random head movements), are possible future functionalities and are not detailed.

As seen in **Figure 31** diagram, when certain buttons/switches are clicked, the visibility of some of the UI's elements has to be updated. To facilitate this task, one function is created for each pair of “checked/unchecked” event, allowing to specify which UI elements are affected in each case and some additional conditions. As seen on **Figure 27**, these functions are part of the “uiObjVisibilityManager.cpp” file, which contains all the functions that handle the visibility of the UI elements.

#### Arduino nano section



**Figure 32. Arduino Nano submodule main activity diagram (own elaboration)**

As seen in **Figure 32**, once the “Setup components” activity is finished, the program follows a never-ending loop flow. Although a lot of activities are present in this diagram, as explained previously, the logic of the submodule is very minimal. As seen in the diagram, the RC values and the RC battery value are sent via serial to the ESP32S3 at certain frequencies, so that they can be displayed on the touchscreen. The frequency of the sent RC values is much higher than the battery value’s one, as there is no point in continuously checking the battery since its value will not rapidly change. A similar thing happens with the frequency of the sent RC values via serial and the sent RC values via radio, where the frequency of the sent RC values via radio is higher (every 50ms compared to every 100ms) since the RC values sent via radio are used for rover control, and sudden changes to them need to be transmitted to the rover as soon as possible. The specification of “sending frequencies” for serial and radio communications is essential as the boards and the NRF24 have limited resources; if a continuous stream of data were to be sent, they could saturate and create delays in the transmissions.

Another thing that can be appreciated in **Figure 32** is how the Arduino Nano fulfills its main “communication relay” role. All the received radio frames, if valid, are sent via serial, and all the received serial frames, if valid, are sent via radio. This means that the received frames, except in a couple of specific scenarios, are not processed and are only forwarded.

#### 4.5.2.4 Tests performed

For this module, several test programs were created to study the data transmission limits of the serial and radio communications of the rover, with the main objective of determining if the developed communication protocol with a “frame” structure could send and receive “frames” at an acceptable rate.

These tests consisted in bidirectionally communicating an Arduino Mega and an Arduino Nano sending a continuous stream of “frames” between the two at a specified frequency (e.g., 1 frame every 200ms). In the different carried tests for serial and radio, this frequency was reduced until it was detected that several frames were being lost. With these tests, it was very noticeable that the limiting factor in the rover communications would be radio communications, as serial communications had a significantly higher maximum frame transmission rate. Therefore, all further tests were focused only on radio communication, trying different “frame” formats to see which one was more suitable.

The main “frame” format candidates used for testing were a frame structure with numeric “frame type” field and a 40-character string “message” field, and a frame structure with numeric “frame type” field and several numeric “data” fields using arrays. These two candidates are shown in **Figure 33**.

```
typedef struct {
    uint8_t type;
    int16_t data2B[4];
    uint8_t data1B[5];
} Frame;
```

```
typedef struct {
    uint8_t type;
    char message[40];
} Frame;
```

**Figure 33. Frame structure candidates (own elaboration)**

The frame structure with the string “message” field had the advantage that it was easier to send human-readable messages (e.g., message field with sensor values like “temp=27&hum=50&press=1050”). The frame structure with the numeric “data” fields had the advantage of having a significantly lower size, but at the cost of having to “decode” the field to understand the meaning of the values it contained. Moreover, sending numeric values with the first structure format was far more costly than sending numeric values with the second structure format (e.g., sending a 255 as chars occupies 3 bytes, while sending it as a uint8 occupies 1 byte).

In a system with lots of resources, the frame structure with the string “message” field would most likely be selected. However, in a constrained system with low resources, as is the case with the Arduino microcontrollers and the NRF24 radio module, it was necessary to check if this frame structure could be used to send and receive frames via radio at an acceptable rate.

After some tests, the use of the frame structure with the string “message” field yielded very poor frame transmission rate results, with a maximum bidirectional transmission rate of about 1 frame every 200ms. On the other hand, the use of the other frame structure accomplished a maximum bidirectional transmission rate of about 1 frame every 50ms.

For a reliable communication, a minimum of 1 frame every 100ms was needed to ensure that the remote control values that were sent to the rover for the motors and servomotors control resulted in fluid movements of these motors and servomotors. Therefore, the frame structure with the numeric “data” array fields was selected.

#### **4.5.2.5 Problems and solutions**

During the implementation of this module, one issue was encountered. To control the 4-axis robotic arm with a gripper, a total of 5 joystick axis would be needed for the individual control of the 5 robotic arm servomotors, but only 4 joystick axis were available in the Spektrum remote control [42] .

To solve this problem, a “control claw servos” button was added to the embedded UI, which is only visible when the button for the “robotic arm control” operation mode is activated. This “control button” can be activated by touching it or by changing the value of one of the RC auxiliary channels (which is mapped to a physical switch), and allows to control a different set of robotic arm servos depending if its checked or unchecked.

### 4.5.3 Camera and webserver module

The software present in the camera and webserver module is responsible for creating the web dashboard from which the rover will be monitored, adding a camera view to this dashboard and communicating via serial with the central rover’s module Arduino Mega to receive/send status values.

The software of this module has been mainly developed using Python for most of the backend logic, HTML and CSS for the dashboard UI, and JavaScript for some UI related logic and to handle socket events. Bootstrap CSS classes are used to create styled UI components with ease.

As explained in the **Software technologies and libraries rationale** section, the chosen OS for the Raspberry Pi Zero 2W is the headless version of RaspberryPi OS [46] , selected for its lower resource usage. This version lacks a graphical UI and can only be operated through the command line. Consequently, the most efficient way to develop and test programs on the RaspberryPi is by connecting to it via SSH. This method eliminates the need to attach peripherals such as keyboard, monitor and mouse to the board while developing and testing, which is ideal.

Developing complex programs directly through the terminal is not very optimal, as command-line text editors such as Vim or Emacs have to be used, which can slow down the developing process for users that have not mastered them. To make the development process more efficient, an SFTP plugin [94] for the Visual Studio Code editor is employed. This allows to easily set up and modify the project’s file structure and file’s contents remotely from a standard PC, offering a far more efficient developing workflow.

#### 4.5.3.1 Back-end program structure

In **Figure 34**, a diagram detailing the most important created back-end files for the module and their relations is shown.

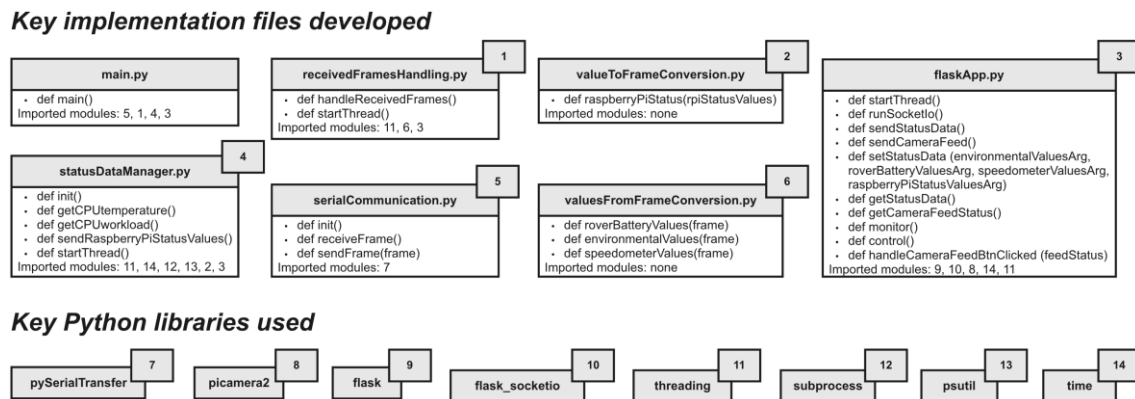


Figure 34. Key implementation files and libraries for camera and webserver module (own elaboration)

Each developed “.py” file implements functions dedicated to a specific aspect of the system. Below is a description of the main responsibilities assigned to each of these files:

- receivedFramesHandling.py: Handles the operations that need to be done for each type of received serial frame.
- statusDataManager.py: Facilitates the retrieval of status data (e.g., CPU temperature, CPU workload) and sends it via serial with the appropriate time frequency.
- serialCommunication.py: Facilitates receiving and sending custom frames via serial.
- valuesToFrameConversion.py: Translates known data structures to frames of the custom communication protocol, which are optimized for data transfer.
- valuesFromFrameConversion.py: Translates the frames of the custom communication protocol to known data structures, which are more easily used.
- flaskApp.py: Starts the built-in development server. Creates threads to send the status data and the camera feed to the front-end JavaScript section via sockets. Handles the routing of the web dashboard and the incoming socket events of UI button clicks.

The “main.py” file does not contain any complex logic; its primary responsibility is handling the initial setup and starting the necessary threads.

Several of the function headers shown in **Figure 34** utilize custom data structures. These structures facilitate the storage, transfer, and manipulation of data throughout the different parts of the system. All the structures used in this module are defined on a “.py” file, which is included by the files that require them. These structures are conceptually the same as the ones that were used in the central rover module, which are detailed on **Figure 19**.

#### 4.5.3.2 Back-end program flow

In **Figure 35** an activity diagram of the module is shown. This diagram is only a general representation of the flow that the main program follows, without detailing how each of the steps in the flow are internally implemented.

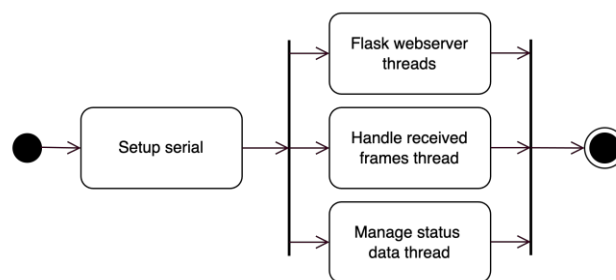
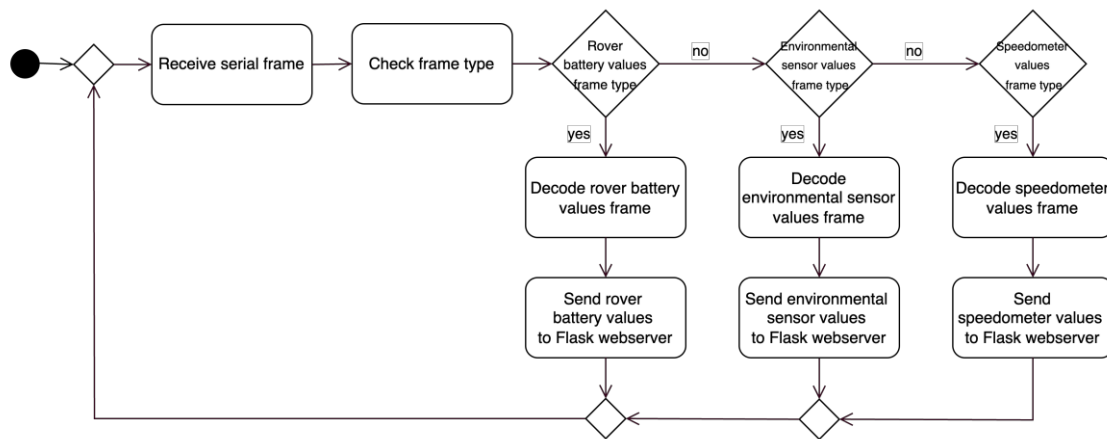


Figure 35. Camera and webserver module main activity diagram (own elaboration)

As seen in **Figure 35**, once the initial “Setup serial” activity is finished, three concurrent activities are started. These activities are very general and have a lot of logic, therefore requiring an explanation:

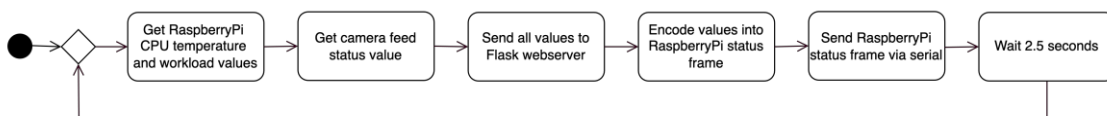
- **Flask webserver threads:** In this activity, three threads are started. The first thread runs the webserver using the “flask\_socketio” library [95] . The second thread sends status data updates to the website’s JavaScript via sockets with a certain time frequency, so that the new status data values can be displayed in the UI. Similar to the second thread, the third thread sends camera images to the website’s JavaScript via sockets with a high time frequency (30 images per second), simulating a continuous video feed.
- **Handle received frames thread:** From a received serial frame, a “type” field can be obtained. This “type” field indicates the category of the received message, and is used to execute the functions that will be able to process the type of message. Once processed, if the message is a status value, it is sent to the Flask webserver to be displayed in the UI.
- **Manage status data thread:** In this activity, each of the RaspberryPi’s status data values (e.g., CPU temperature and workload) are read and calculated with a certain frequency (e.g., every 2 seconds). These status values are then sent to the Flask webserver with the appropriate frequency, to be displayed in the UI.

To get a better understanding of the “Handle received frames thread” activity, in **Figure 36** an activity diagram with all its subactivities is shown.



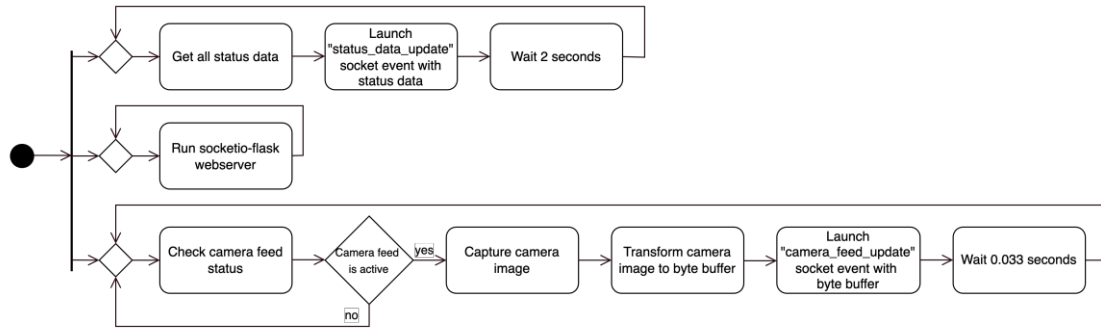
**Figure 36. Camera and webserver module “Handle received frames thread” activity (own elaboration)**

To get a better understanding of the “Manage status data thread” activity, in **Figure 37** an activity diagram with all its subactivities is shown.



**Figure 37. Camera and webserver module “Manage status data thread” activity (own elaboration)**

To get a better understanding of the “Flask webserver threads” activity, in **Figure 38** an activity diagram with all its subactivities is shown.



**Figure 38.** Camera and webserver module “Flask webserver threads” activity (own elaboration)

The activities of **Figure 36**, **Figure 37** and each of the three branches in **Figure 38**, are internally implemented using threads, which are continuously running and follow a never-ending loop flow.

In the “camera feed” branch of **Figure 38**, the followed loop flow has a wait time of 0.033 seconds between interactions so that 30 socket events are launched every second (1 second / 1 image every 0.033 seconds = 30 images in one second). This is known as the FPS (frames per second) rate, and 30 FPS is usually considered an acceptable rate for live video transmission.

#### 4.5.3.3 Front-end development

Developing a custom web dashboard UI can be a very time-consuming task, especially when aiming for a visually appealing and responsive design where only raw HTML, CSS and JS is used. For this reason, as explained in the **Software technologies and libraries rationale** section, the Bootstrap [17] front-end framework was selected to accelerate the dashboard’s development process and enhance the overall UI. In addition to Bootstrap, a JS library was used to simplify the creation of interactive charts [97] , and a CSS icon library [96] containing hundreds of icons was included to complement the titles of some of the UI elements.

Specifically, Bootstrap CSS was used for:

- Creating and styling navbar.
- Creating and styling component panels, where various status values are displayed.
- Styling a button matching the look of the rest of the UI.
- Creating an image panel to display the video feed, which scales to fit its parent element.
- Structuring and distributing all UI components using the Bootstrap grid system, which enables an organized and responsive design that adapts to different screen widths.

The use of all these resources (Bootstrap, JS chart library, CSS icon library) has allowed to focus the development efforts on the logical functionality, while still getting a clean, responsive, and customizable UI.

Before beginning the implementation of the front-end of the dashboard, a conceptual planning was carried out to determine how all the necessary control and monitoring functionalities could

be represented and distributed. After this initial conceptual planning, and considering OpenRover's requirements, it was concluded that two screens with the following purposes and elements would be created:

- **Monitor screen:** In this screen, most of the status parameters of the "Central rover" module and this "Camera and webserver" module are displayed using a panel with a title, icon, and text box for each status parameter. Additionally, for the environment temperature and humidity values, charts are used to display their evolution over the last minute, as was done in the embedded UI of the "Remote control and touchscreen module". A large panel is used to display the camera's live video feed, with a button to start/stop the feed also being included in this panel.
- **Control screen:** All the interactive elements required to control the rover's functionalities and change between its operation modes are included in this screen. Contrary to the embedded RC UI, where a UI control element like a joystick was not necessary as we had the physical RCs joysticks, in this web dashboard digital joysticks or directional arrows can be used to control some of the functionalities of the rover.

To navigate between the two screens, a top navigation bar with a link to each screen is used.

It is important to mention that the "control screen", primarily due to time constraints, was finally not implemented in the created dashboard UI. The link for the screen appears in the navigation bar and the backend routing is implemented, but no interactive control elements are present on the screen. In the **Fulfillment of requirements** section this will be explained further.

### Program structure

The front-end section of this module is composed of a series of HTML, CSS and JS files:

- **HTML files:** The HTML files define the content (e.g., buttons, titles, panels, and other UI components) and the structure of this content. For certain components, references are made to CSS classes (present in Bootstrap CSS or custom styles.css file), which indicate exactly how these components are styled. For the components that need to be referenced from the JavaScript section of the application (to add event listeners to them if they are buttons or modify their value if they are status data fields), unique IDs are defined.
- **script.js:** This created JavaScript file handles the socket events sent from the backend "flaskApp.py" (e.g., camera\_feed\_update) and sends socket events (e.g., camera\_feed\_btn\_clicked) to this same backend. The value of the UI components, like the status data text fields, can be dynamically changed from this file by using the unique IDs defined in the HTML files. The logic for some of the UI components, such as the temperature and humidity charts, is also located in this file.
- **styles.css:** This file contains all the custom CSS classes used to style some of the UI components. Additionally, some Bootstrap CSS classes are also included in this file, as some of their attributes are changed to modify the default Bootstrap CSS class look.



Another thing that is included in this file is a “media query”, which allows to change some CSS attributes depending on the resolution of the screen, making for a more responsive design. Specifically, when the width of the screen is small, the UI panels are stacked, and with this media query vertical space is added between these UI panels.

## Modularity

Aligning with the “modular code” requirement of the project, instead of grouping all the HTML code in a single file, it is distributed in different files, each one having a different purpose or being a different component:

- `navbar.html`: This file contains the HTML code that creates a navigation bar with links to the two screens (monitor and control). This navigation bar is styled with the Bootstrap “navbar” class, which gives it a clean look.
- `monitor.html`: This file contains a container “div” which groups all the panels that contain the icons, titles, and value of the status data. Additionally, this container includes the panels for the temperature and humidity charts, as well as the video feed panel. All these panels are organized using the Bootstrap grid system.
- `control.html`: This file contains a container “div” which, in the future, will group all the panels that contain the interactive components used to control the rover from the dashboard.
- `base.html`: This file contains the links to the Bootstrap, icon library and “styles.css” CSS files, as well as references to the chart library, socketio library and “script.js” JavaScript files. The “head” and “body” of the dashboard website is specified in this file.

For the “base.html” file, using the directive “`{% include 'navbar.html' %}`” in the body, Flask dynamically inserts the contents of the “navbar.html” component. Similarly, if the “`{% block content %} {% end block %}`” directive is used in the body, and the “monitor.html” and “control.html” extend the “base.html” and use this same directive, Flask can insert the contents of “monitor.html” or “control.html” in the body depending on template that Flask wants to render. This approach makes the HTML code very modular, scalable and avoids code repetition, as items like links, references to JS libraries and shared components (e.g., navigation bar) only have to be coded in the “base.html” file.

## Front-end and back-end interaction with socket events

For communications between the front-end “script.js” file and the back-end “flaskApp.py” file, socket events are received and emitted:

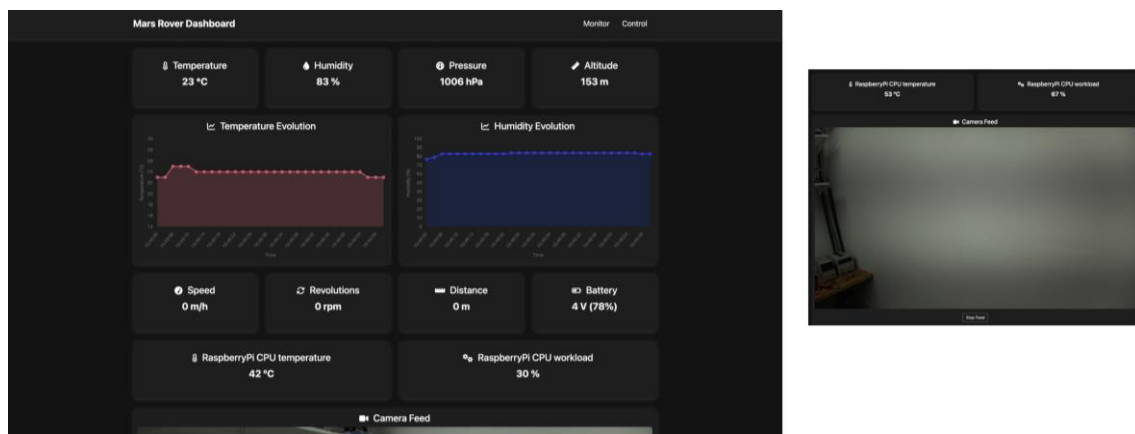
- Event “status\_data\_update”: This socket event is emitted inside “flaskApp.py” and received inside “script.js”. Thanks to this event, all the status data values (e.g., temperature, humidity, speed, Raspberry CPU workload, rover battery) which the backend has stored are sent to be displayed in the UI.

- Event “camera\_feed\_update”: This socket event is emitted inside “flaskApp.py” and received inside “script.js”. The event data contains an image byte buffer, which the JavaScript inserts as a JPEG in the HTML image panel.
- Event “camera\_feed\_button\_clicked”: This socket event is emitted inside “script.js” and received inside the “flaskApp.py”. Inside the “script.js”, an event listener is added to the “camera feed enable” button, which toggles between the enabled/disabled states for the video feed and sends an event with the changed state when the button is clicked.

The “status\_data\_update” and “camera\_feed\_update” socket events can be seen in **Figure 38**, where the flow of the “Flask webserver threads” activity is detailed. However, the “camera\_feed\_button\_clicked” socket event is not detailed in any flow diagram as it is asynchronous, with the launch of the event depending on when the user clicks the button.

### Implemented design

At the beginning of this **Front-end development** section, the purpose and contents of each of the developed screens and components are explained. In **Figure 39**, the implemented UI for the monitor screen, as well as the navigation bar, is shown.



**Figure 39. Web dashboard monitor screen (own elaboration)**

As explained previously, the “control screen” was finally not implemented in the created dashboard UI, primarily due to time constraints. In the **Fulfillment of requirements** section this will be explained further.

Thanks to the use of Bootstrap and media queries, the rover’s dashboard design is responsive and adapts to different screen sizes. In **Figure 40**, the look of the web dashboard when it is accessed from a mobile browser is shown.

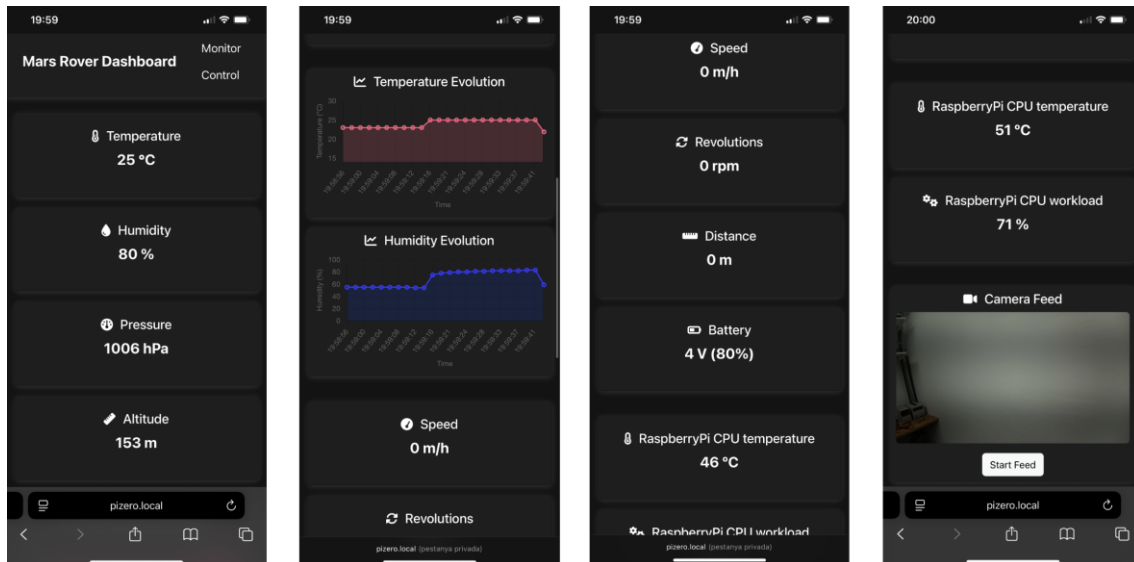


Figure 40. Web dashboard in mobile browser (own elaboration)

The use of the web dashboard to monitor the rover's status parameters is compatible with the concurrent use of the embedded UI touchscreen to monitor and control the rover. The values of the status parameters (e.g., temperature, battery) are updated at the same pace in both scenarios. This can be seen on **Figure 41**, where temperature and humidity graphs have the same shape in the web UI and the embedded UI, and the displayed status values are the same.



Figure 41. Web dashboard and embedded UI comparison (own elaboration)

#### 4.5.3.4 Automated webserver start

In the “Central rover” and “ESP32S3 and touchscreen” modules, microcontroller boards were used, which have fast boot times and the ability to automatically execute the uploaded code upon power-up, without requiring any additional configurations. In contrast, this module uses a RaspberryPi Zero 2W running RaspberryPi OS, which behaves like a traditional computer. Therefore, upon startup, by default the system does not automatically launch the custom programs developed for OpenRover.

This is why additional configurations are required to ensure that the desired software programs and actions are initiated at boot. These configurations are specified in a custom Bash script called “startupTasks.sh”, which uses a series of shell commands, as seen in **Figure 42**.

```
#!/bin/bash
1) sudo nmcli device wifi hotspot ssid RoverWiFi password marsRover
2) cd /home/polv && source my-rover-venv/bin/activate
3) cd /home/polv/RoverRaspberryPi && python -m logic.main &
```

**Figure 42. RaspberryPi startup bash script (own elaboration)**

The first command in the script uses the “nmcli” utility to create a WiFi hotspot with the specified name and password [83] . This WiFi hotspot is generated from the onboard WiFi module of the Raspberry Pi Zero 2W, and is where the clients will connect to in order to access the web dashboard that is hosted on the Raspberry.

The second command in the script is used to access and activate the virtual environment for the OpenRover applications. This virtual environment contains all the Python libraries that are imported in this module’s developed python files. The key libraries that are present in the virtual environment can be seen in **Figure 34**. Some of these have been installed using the “Pip” packet manager, while other libraries came by default and were automatically included in the virtual environment when it was created.

The third command in the script is used to access and execute the “main.py” program file, which calls functions present in other developed python files. From these calls, all the necessary threads are created and the whole back-end and front-end of this module’s program starts running. The “&” at the end makes the command run in the background, ensuring that the terminal does not wait for the Python script to end.

The **Figure 42** script must be executed at startup to ensure that the WiFi hotspot is generated, the webserver is started and serial communication with Arduino Mega is begun as soon as the RaspberryPi receives power and finishes its initial booting sequence. To execute the script at startup, the script is called from the “/etc/rc.local” file, which is executed at startup after all usual system services are started.

#### 4.5.3.5 Problems and solutions

During the hardware design and implementation of this module, some issues were encountered. Below are some of the key problems and the corresponding solutions that were applied:

- As explained on the previous **Automated webserver start** section, a script is launched as soon as the RaspberryPi board power up, which creates a WiFi hotspot and starts the main program execution. Several attempts were necessary before the contents of this script could be correctly executed, since the initial commands that the script included had some errors (e.g., did not include “sudo” at the beginning when it was necessary) and there was no direct way to see the exception/errors that the commands of the script produced, as the script was launched automatically at power up and did not display the exception/errors on any visible terminal. To understand and solve these minor errors, an additional command was included in the script, “exec > /home/polv/startup.log 2>&1”, which created a log file that contained all the prints that the commands below this command made when the script was automatically executed at start up. This log file was essential to understand and correct the errors that impeded the correct creation of the WiFi hotspot and the correct initiation of the main program execution.
- The “SerialTransfer” library for Arduino (C++) allowed the specification of the baudrate, but mirror “SerialTransfer” library for Python did not. This caused some problems as the initial serial communications between the Arduino and the RaspberryPi could not be correctly read. After doing some research on the internet, it was concluded that the RaspberryPi had a default baudrate of 115200 [87] , and once this baudrate was also specified in the Arduino code, the communication between the two boards was successful.
- As explained on the previous **Automated webserver start** section, a “virtual environment” is used to install some of the Python libraries that this module’s code uses. To install libraries on the virtual environment, only “Pip” packet manager can be used. The encountered problem was that some libraries were not able to be installed through “Pip”, as they were only available through “Apt” packet manager. To solve this problem, these packages were installed system-wide through “Apt” and were then manually copied to the already created virtual environment. Before reaching this solution, several unsuccessful attempts were made to install the packages directly in the virtual environment using “Apt”, trying different configurations, but this was not possible.
- As explained on the **Front-end development** section, resources like Bootstrap and libraries for icons, charts and sockets were used. Usually, these resources and libraries are included using a CDN (Content Delivery Network) link, but this approach only works if the user that connects to the webserver or the webserver itself has access to the internet. As the RaspberryPi generates its own WiFi network, which is disconnected from the internet, and the users connect to this network to access the web dashboard, if the resources and libraries were to be included as links, they would not load. To solve this problem, the files that were available in the CDNs were downloaded and saved into the “/static/resources/” server directory.

#### 4.5.4 Handling of concurrent activities

One of the software requirements for all the modules was having concurrency support, being able to execute multiple tasks apparently at the same time. The way concurrency is implemented depends heavily on the hardware platform where the software is being run:

- Development boards such as the RaspberryPi Zero 2W are very powerful, having several CPU cores [26] and a computer-like behavior, even running complex OSs like the Raspberry Pi OS or Ubuntu. In such hardware platforms, tasks concurrency can be implemented using threads or separate processes.
- In microcontroller development boards such as the Arduino Mega and Nano, sequential programming is used, since multithreading and separate processes are not available due to hardware limitations and lack of an OS. Instead, to simulate concurrency, “non-blocking programming” is implemented, where a high frequency loop is used to execute tasks sequentially and repeatedly, with each task being very lightweight to ensure that the program execution is not being excessively delayed. Timing mechanisms (e.g., “`millis()`” arduino framework function) are used to schedule these lightweight tasks at the appropriate intervals.
- In development boards such as the ESP32S3, despite being microcontrollers, powerful CPUs with more than one core are used and lightweight OSs like “FreeRTOS” [22] are supported. Thanks to this OS, multiple “loops” can run in parallel, while abstracting the coder of the complexity involved in managing a parallel execution. Sequential programming can also be used in ESP32 boards. This was the case for this project, where “non-blocking programming” was implemented for the ESP32S3 touchscreen section, allowing to replicate part of the execution flow that was already implemented in the Arduino boards, and reuse some of the created functions.

##### Concurrency on RaspberryPi Zero 2W

For the branches seen in **Figure 35** and **Figure 38** activity diagrams, threads were used for a concurrent execution of the branches. To launch the threads, the “`threading`” Python module was used, which allows to set a target function for a thread and then start the thread, which will execute the function in a separate thread until it finishes. Many of the functions that are set as thread targets do not end by themselves, meaning that they keep executing in a separate thread until the main thread is manually shut down or the RaspberryPi is powered off.

The use of threads was chosen over launching separate processes mainly due to the fact that threads share the same memory space, simplifying communication between them, while processes have separate memory spaces [98]. Having separate memory spaces requires inter-process communication (IPC) mechanisms (e.g., queues, pipes), which adds some complexity and overheads.

##### Concurrency on Arduino Mega, Nano and ESP32S3

Using threads on RaspberryPi Zero 2W, if a certain wait time was needed before executing an action, it was possible to simply call a “`time.sleep(X)`” function to delay the execution of a thread

by “X” seconds, without affecting any of the other threads. In the Arduino framework, there is a similar function called “delay(X)” [12] which delays the execution of the program by “X” seconds, but as there is only one thread, the whole program is delayed.

Therefore, if timing an action is required (e.g., sending a sensor value over serial every 2.5 seconds), the “delay(X)” function should not be used. Instead, the “millis()” function of the Arduino framework, which returns the milliseconds since the board’s startup, is used to time actions/tasks and functions. With “millis()”, the current time is compared with previously stored timestamps to check if a certain task has to be executed.

```
int interval = X; //We want to execute a task every X seconds

void loop () {
    static unsigned long previousMillis = 0;

    if (millis() - previousMillis >= interval) {
        previousMillis = millis();
        //Execution of scheduled task (e.g., function call, procedure)
    }

    //Other tasks can run here, without being blocked
}
```

**Figure 43. Tasks timing using non-blocking “millis()” template (own elaboration)**

This non-blocking “millis()” programming approach, shown in **Figure 43**, is used in many sections of the Arduino Mega, Nano, and ESP32S3, as many concurrent actions are needed.

#### 4.5.5 Communication protocol

The OpenRover’s communication protocol was designed with the primary objective of enabling an easy transmission of structured data between modules, regardless of the specific physical communication method (e.g., serial or radio) or the specific development board in use (e.g., Arduino or Raspberry) in each module. This is why libraries like the “NRF24” and “SerialTransfer” were selected, as they allowed the transmission of structured data without many complications and had mirror python libraries that could be used for RaspberryPi.

A common data structure, shared among the development boards of all the modules, was created. This structure was called “Frame”, which was initially presented in **Figure 19** when detailing the data structures used in the “central rover” module. In **Figure 44**, this C++ “Frame” structure and its counterpart Python “Frame” structure are shown.

|                                                                                                      |                                                                                              |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <pre>typedef struct {     uint8_t type;     int16_t data2B[4];     uint8_t data1B[5]; } Frame;</pre> | <pre>class Frame:     fType = 0     data2B = [0, 0, 0, 0]     data1B = [0, 0, 0, 0, 0]</pre> |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|

**Figure 44. C++ (left) and Python (right) frame structure (own elaboration)**

When there is the need to communicate two development boards via radio or via serial, this frame structure is always used to send the data. Each frame contains data related to a specific message type (e.g., “remote control values”, “change RaspberryPi power relay status” message types). The message type is specified in the “type” field of the frame structure, and the data is located in one or several positions of the “data2B” and “data1B” array fields, depending on the needs or the defined positions of the specific message type.

The frame “type” field contains a numeric value, which takes up very little space (1 byte) but is not very human-readable. In order to facilitate the development of the programs and the creation of frames, each numeric frame type is associated with a definition string. In C++, this is accomplished using “#define” directives, which are written in a “frameTypesDefinition.h” header file that is included in all files that receive or send frames. This can be seen in **Figure 45**, which details all the possible frame/message types that can be sent or received. A similar approach is used for Python, where a “frameTypesDefinition.py” also contains all the possible message types and their associated numeric values.

```
#define NOT_VALID 0
#define INF_F_RC_VALUES 1
#define INF_F_RC_BAT_LEVEL 2
#define CMD_F_RC_BUZZER 3
#define CMD_F_RC_RADIO_ENABLING 4
#define CMD_F_ROVER_OP_MODE_SELECTION 5
#define INF_F_ROVER_BAT_LEVEL 6
#define CMD_F_ARM_SERVO_SELECTION 7
#define INF_F_ENVIRONMENTAL_MONITORIZATION 8
#define INF_F_SPEEDOMETER_VALUES 9
#define INF_F_RASPBERRYPI_STATUS 10
#define CMD_F_RASPBERRYPI_POWER 11
#define CMD_F_TEST 250
```

**Figure 45. Frame types definition C++ header file (own elaboration)**

The selected definition strings that can be seen in **Figure 45** indicate the message type (e.g., RC\_VALUES) and whether the data that the frame contains is informative (INF prefix) or is a command (CMD prefix). For example, a frame that contains a boolean value that is used to decide if a relay is powered on can be considered a “command frame”, whereas a frame that contains the values of an environmental sensor can be considered an “information frame”. These prefixes do not have any effect on the program’s logic and are only used as a way for the developers to better understand the purpose of the frame that is being sent or received.

As frames are used for serial and radio communications, ensuring a small frame size is essential to maximize the number of frames that can be sent in a certain time window. To dimension the frame structure, the type of message with the biggest data content was taken into account. This message type with the biggest data content corresponds to the “remote control values”, for which it was necessary to send the following data:

- Joystick channel values: Each joystick axis has a range of (-255, 255), requiring two data bytes. There are two joysticks, with two axis (X, Y) each.
- Auxiliar channel values: Each AUX value has a range of (0, 255), requiring exactly one data byte. There are four AUX channels.

This is why an array of two-byte signed integers with four positions was used to be able to store the joystick channel values, and an array of one-byte unsigned integers with five positions was used to be able to store the AUX channel values and an additional one-byte value just in case. Considering the “type”, “data2B” and “data1B” frame fields, each frame has a size of 14 bytes. It is also important to mention that the “type” and “data1B” fields were declared as unsigned integers to support a range of (0, 255) instead of the (-127, 127) range, as negative values do not make sense in the “type” field and will not be saved in the “data1B” field.

As seen in **Figure 44**, the frame’s field types (e.g., int16\_t) are specified in the C++ struct but not in the Python struct, where only the length of the fields is specified. In Python, it is not possible



to directly specify the byte size and signage for an integer value; therefore, the byte size and signage constraints for each field are specified inside the “sendFrame()” and “receiveFrame()” functions using “format characters”, which allow conversions between C and Python types.

For every message type detailed in **Figure 45**, two functions to encode and decode the specific message type are implemented in each development’s board code. The encoding functions convert the values of a message type into a frame structure, specifying the message type into the frame “type” field and storing the message values on the suitable array positions of the “data1B” and/or “data2B” fields. This frame structure has already been detailed in **Figure 44**. Once the encoding function for a certain message type is created, the decoding function can be implemented, which converts the data present in a frame structure to a more manageable data format, such as one of the data value structures seen in **Figure 19**. The development board that sends a serial or radio message will use the frame encoding function, while the one that receives the serial or radio message will use the analogous frame decoding function.

## 4.6 Making the project open-source

One of the objectives of the OpenRover project was making the project open-source, enabling other people to replicate the platform to learn more about the software and hardware aspects of DIY robotic projects, and to have the possibility of adding more software or hardware features to the platform.

Aligning with this objective, all the developed hardware design and software files were published on a GitHub repository<sup>1</sup>, making them publicly available. To ensure that the created repository was usable and easily understandable by other people, a detailed README.md file was created. This file provides essential information about the project, including:

- General overview, detailing context, motivation, objectives, key contributions and disclaimers.
- Features description.
- Project modules description.
- Expected mechanical and hardware build time.
- Getting started steps.
- Possible improvements to work on.
- How to contribute description.
- Licensing information.

The OpenRover repository project files are organized aligning with the project’s modular design, with a separate folder for each module:

- central\_rover: Contains the PlatformIO project with all the code and the hardware schematics of the “Central rover module”.
- remote\_control\_touchscreen: Contains the two PlatformIO projects with all the code and the hardware schematics of the “ESP32S3 and touchscreen” and “Arduino Nano” submodules. These submodules are part of the “Remote control and touchscreen”

module. Additionally, the SquareLine Studio project with the UI design is also included in this folder.

- `web_dashboard`: Contains the project files that run in the RaspberryPi Zero 2W and the hardware schematics of the “Camera and webserver” module.

Other folders containing additional resources were also added to the repository:

- `images`: This folder contains images about the mechanical assembly, wiring and hardware implementation process.
- `main_parts_list`: This folder contains an excel sheet detailing the most important materials and hardware and mechanical parts with their respective costs.
- `3d_designs`: This folder contains the custom 3D designs (e.g., remote control cases).

The selected GitHub repository name is “openrover-robotic-platform”, detailing the project name and the nature of the project, which is developing a robotic platform for software and hardware development.

---

<sup>1</sup> OpenRover repository: <https://github.com/pol-valero/openrover-robotic-platform>

## 5 Fulfillment of requirements

In this section, the fulfillment of the requirements that were set at the beginning of the project will be discussed. For those requirements that have been either partially or completely not implemented, an explanation of the causes is also included.

### 5.1 Mechanical structure requirements

Even though the mechanical aspect of the rover is not fully inside the scope of this final degree project, it is still interesting to point out that most of the requirements for the mechanical platform have been met.

The mechanical structure upon which the project has been built is mostly 3D printed and employs a 6-wheel drive platform with 4 steerable wheels and a rocker-bogie mechanism. Additionally, a 4-axis foldable robotic arm and a tiltable and rotary head with enough space inside for a development board and a camera are included.

However, once all the hardware components were fitted in the rover, due to the final weight and the mostly 3D printed structure, the platform was not able to withstand a lot of mechanical stress and the rover was only able to travel around relatively flat surfaces. This implies that the “all-terrain capabilities” requirement has not been completely achieved.

### 5.2 Hardware requirements

All the requirements related to the hardware components that were set at the beginning of the project have been achieved, as it will be explained below.

The OpenRover project has employed different development boards that adapt to each of the module’s needs. A custom remote control module with an integrated touchscreen and an ESP32S3 has been created, which is capable of booting within time constraints to allow an immediate operation of the rover. This module includes battery monitoring for the Spektrum DX8 RC used for the joysticks and switches input and is designed to be easily detachable using JST connectors.

For communication, an NRF24 radio module was employed, which provides low latency bidirectional data exchange between the custom RC and the rover. This module, together with the software, was able to deliver a bidirectional data transmission rate of about 1 frame (custom structure) every 50ms, ensuring fluid movements of the motors and servomotors when controlled, which were the most critical remote-operated components.

Thanks to the use of different “buck converter” boards, a single battery was used to power all the components of the main rover body, which delivers a stable discharge rate and has sufficient capacity more than 1 hour of continuous operation.

The rover employs a single environmental sensor capable of measuring temperature, humidity, pressure, and altitude, as well as a voltage sensor to monitor the battery level. To support the use of a camera and the creation of the monitoring and control web dashboard, a RaspberryPi Zero 2W board, which has an onboard WiFi module, was employed.

Motors, servomotors and stepper motors, along with the necessary drivers, were used for the different moving parts of the rover. These components were handled by an Arduino Mega microcontroller, which had plenty of available pins and nearly instant booting time, ensuring quick control over all the elements after power-up. To interconnect all the hardware components, modularity and cable management were carefully considered, using standard connectors like JST to facilitate assembly/disassembly and possible repairs.

Additionally, all the boards present in the different modules of the OpenRover project supported serial communication, ensuring a consistent data exchange mechanism across the different parts of the rover's system.

## 5.3 Software requirements

Most of the software-related requirements that were set at the beginning of the project have been achieved, but some were not, as it will be explained below.

Concurrency was effectively implemented on each of the rover's modules, enabling multiple tasks to run apparently simultaneously. This was achieved via the use of non-blocking programming techniques in the case of microcontrollers, and the use of threads in the case of the RaspberryPi board.

Four rover operation modes (conventional driving, 360° turn control, robotic arm control, head control) were implemented in the central rover module and the touchscreen's module software, allowing each mode to be easily selected from the embedded UI of the custom RC's touchscreen. This RC module is able to read the channel values from a generic RC (Spektrum DX8) thanks to decoding the PPM signal that is outputted from the RC's trainer port. A custom communication protocol has been created to transmit these RC values as well as any other information between two development boards. This protocol works both with wired serial communications and wireless communications with the NRF24 module.

The RC touchscreen and the web dashboard display a large set of monitoring and status data, with some graphs detailing the evolution of the temperature and humidity readings. In both the embedded UI and the web dashboard UI, separate screens have been created for the monitoring functionality that was just described and the control functionality, allowing to easily change between the screens using a menu. The web dashboard runs on a lightweight web server hosted on the Raspberry Pi Zero 2W and incorporates a live camera feed panel with a button that can be toggled to start or stop the feed.

Related to the web control screen, one of the requirements specifies that this screen should display some controlling elements, as digital joysticks, which allow to manage some of the rover's functionalities. This requirement was not achieved, since the control screen has not been fully implemented. The web dashboard control screen was planned as one of the last things to implement in the project, as it could not be done until the central rover module was finished and priority was given to the physical remote control module, which had a faster setup time that allowed an immediate operation of the rover. Since the creation of the interactive control elements of the web dashboard (e.g., joysticks, sliders) was more complex than anticipated, the limited amount of time left made the abandonment of the implementation of this control screen

necessary. In the **Future lines of work** section of this document, the implementation of this screen will be proposed as one of the possible future lines of work.

To enhance the mobility of the rover and avoid tire slipping, one of the requirements was implementing Ackermann steering equations to calculate the optimal speeds and turning angles of the wheels. However, this requirement has only been partially achieved, as some tire slipping still occurs when a rover performs a turn at its maximum speed and turning angle. This problem has been detailed in the **Software implementation** section, specifically in the **Problems and solutions** section inside the **Central rover module** explanation and is caused by unknown physical factors that require slight modifications of the Ackermann steering equations.

Some audible alerts using buzzers, and visible alerts using visual components of the embedded UI, have been incorporated. These alerts are used for the remote control link loss detection and the critically low warnings of the rover's and RC's batteries.

Lastly, a modular software design was followed throughout the project, organizing the general functionalities in different "modules", and for each of these modules having the specific functionalities clearly divided across different files. This not only simplified many development and debugging aspects, but also made the whole codebase more scalable and maintainable.

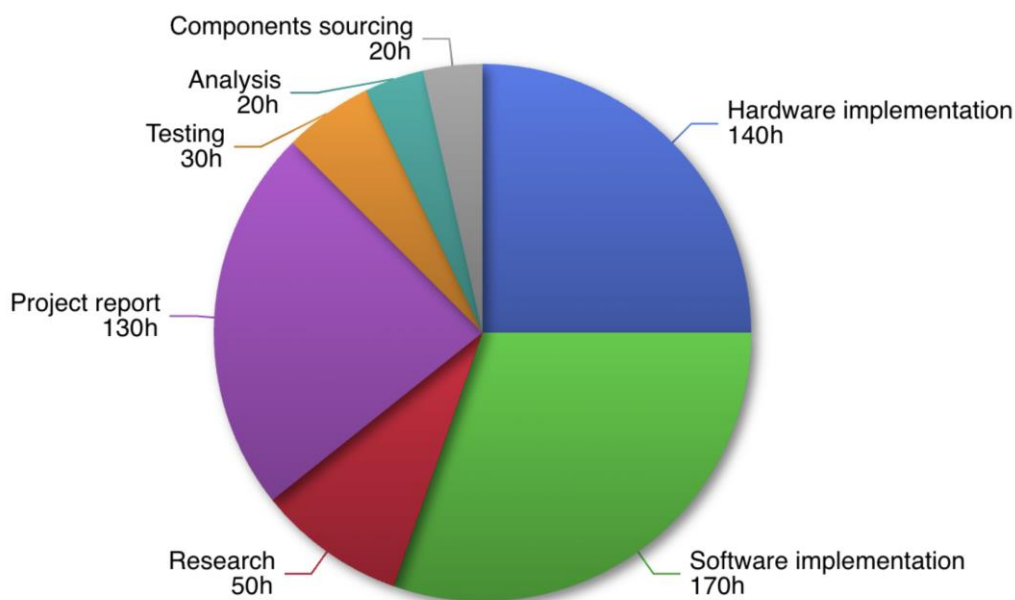
## 6 Cost analysis

In this section the temporal and economic costs of the project will be detailed.

### 6.1 Temporal costs

The project's time distribution pie chart is detailed in **Figure 46**. Below is a brief description of each of the project's sections that appear in this pie chart:

- Hardware implementation: Time spent doing hardware related work (e.g., soldering, wiring, PCB schematics creation), which is detailed on the **Hardware implementation** section of this document.
- Software implementation: Time spent doing software related work (e.g., programming, embedded UI creation), which is detailed on the **Software implementation** section of this document.
- Research: Time spent doing research related to the **Theoretical framework** section of this document (e.g., analysis and comparison of suitable hardware and software components).
- Project report: Time spent structuring and editing this report.
- Testing: Time spent conducting tests on each implemented hardware and software section.
- Analysis: Time spent designing the rover's system from a conceptual point of view.
- Components sourcing: Time spent finding and ordering the selected hardware components.



**Figure 46. Project's time distribution pie chart (own elaboration)**

Since the mechanical aspect of the rover is not fully inside the scope of this final degree project, the time spent for creating and modifying the 3D designs, handling the 3D printer, and assembling the mechanical structure is not included in the pie chart of **Figure 46**.

The project’s sections that appear in the pie chart can also be used to detail the evolution of the project by incorporating them in a Gantt diagram, as shown in **Figure 47**.

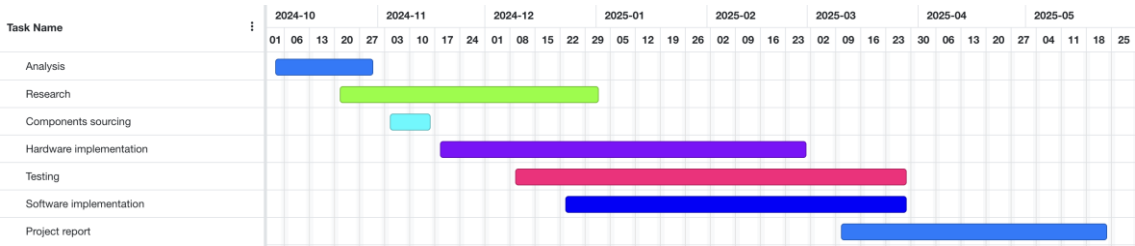


Figure 47. Project’s Gantt diagram (own elaboration)

## 6.2 Economic costs

Since OpenRover is not meant to be produced and sold at a mass scale, the economic costs are not a big point of focus in this project. However, OpenRover aims to be a platform that can be replicated by other interested people, so it is still interesting to detail the costs.

As the table with the costs of each item is very large, it has been included in **Annex II**. The final cost of the rover, which can also be seen in this table, is 887€. This cost does not include the tools that are necessary to build the rover (e.g., soldering iron, power supply for tests), some materials that are not in the rover itself (e.g., tin) and some hardware components that we already had at our disposal (e.g., jumper wires).

Being just under a 900€, it is not a cheap project to replicate, but this approximated price point puts the project at the reach of many robotics enthusiasts who may want to replicate the project and build upon the OpenRover platform.

## 7 Conclusions

The OpenRover project was started with the main objective of developing an open-source robotic platform build upon the 3D printed mechanical model of the Mars rover that was developed as a personal project. Upon completion of the project, it can be concluded that this main objective was achieved, with the resulting platform fulfilling most of the specific objectives detailed on the **Objectives** section of this document as well. The only specific objective that was not completely achieved was the implementation of the control page of the web dashboard, as explained previously in the **Fulfillment of requirements** section.

From the set of objectives, a detailed requirements list was created for each of the mechanical, hardware and software sections of the project. These requirements detailed the expected technical outcomes of the OpenRover robotic platform, and were very helpful to examine at the end of the project whether each requirement was fully, partially, or not fulfilled. This examination of the fulfillment extent of each requirement is detailed in the **Fulfillment of requirements** section of this document.

From the beginnings of the project, work was carried out, simultaneously at times, on two different fronts: the theoretical and the practical frameworks. On the theoretical framework side, an overview of how real Mars rovers operate and what components they include was detailed, existing open-source DIY robotic projects were explored, and a thorough analysis of potential hardware and software components that could be employed for the rover was conducted. This analysis was essential to select the most suitable components for each component category, keeping in mind the objectives and requirements of the OpenRover project. On the practical framework side, various experiments were performed using prototyping boards and test programs to assess the limitations and functionalities of the components. Then, on a module-by-module basis, the final hardware was assembled and wired, and all the software for the development boards was implemented.

During the practical implementation, several problems arose, which required additional research and tests. These problems, their solutions, and the whole implementation process is detailed on the **Hardware implementation** and **Software implementation** sections of this document. Additionally, throughout the hardware and software development process, some potential improvements and new functionalities which could add value to the project were identified. However, primarily due to time constraints, they were not added to the project's scope and were not implemented. These new functionalities and possible improvements are detailed in the following **Future lines of work** section of this document, so that they can be taken into account by anyone who reads the project and may want to replicate it.

After finishing the project, some conclusions about the key contributions of the OpenRover project can be drawn. Firstly, the created robotic platform has incorporated several interesting functionalities using different development boards (Arduino, ESP32, RaspberryPi) that interact with each another, allowing to see the optimal usecase for each board. Secondly, the developed remote control module has a touchscreen interface that allows it to be easily expandable, as more controls and monitorization parameters can be added without the need for more hardware (e.g., physical switches). Thirdly, the project has followed a modular approach, both from a conceptual and technical point of view. From the software technical point of view,



modularity has been implemented by distributing functionalities into different code files, each responsible for a specific aspect of the system, and implementing functions that fulfill a specific purpose and are not too general. From the hardware technical point of view, standard connectors like JST have been used to separate the modules and submodules. This modularity allows the project to be more easily maintainable and scalable, with the possibility of adding more features in the future. All these contributions make this project a very useful resource with a lot of educational value for hobbyists and robotics enthusiasts wanting to learn more about the DIY robotics field.

As a final and more personal conclusion, this project has allowed me to combine my existing knowledge in some fields not typically associated with Computer Engineering (e.g., 3D design, 3D printing, mechanical assembly) with the more electronics and software related knowledge that I have acquired during my degree. Not only have I been able to apply practically this acquired knowledge, but I have expanded the knowledge in some of the fields, such as the ones related to microcontrollers and web technologies. Additionally, besides learning about the most technical aspects of the project and putting them into practice, this project has allowed me to gain lots of knowledge in the more organizational aspects of implementing complex projects. Specifically, I was able to learn about the importance of having a good and detailed conceptual design and plan before the technical implementation, and also to learn about how a suitable solution could be developed within the constrained timeframe that this final degree project provided.

## 8 Future lines of work

Although the main objectives that were set for the OpenRover project have been met, during the development of the project some points of improvement were detected, and additional functionalities were considered, such as:

- **Communications robustness:** During some of the testing of the communications between development boards using the custom “Frame” structure, it was detected that some of the frames were not received in some scenarios. This happened mainly on wireless communications using the NRF24 module, when the distance between the custom RC and the rover was more than 20 meters or there were some obstacles (e.g., walls) in between. To minimize this problem, ACKs could be implemented in the custom communication protocol, so that every time a frame is sent, an ACK frame is expected to be received. If this ACK frame is not received, then the original frame is sent again. This process can be repeated to ensure a minimum number of retries before the connection is considered as lost.
- **Computer vision:** Thanks to the use of the RaspberryPi Zero 2W and the RaspberryPi Camera, on-board computer vision can be implemented to add some additional functionalities. For example, computer vision can be used to track a certain object (e.g., a colored ball) and make the rover’s head rotate and tilt to keep this object in focus. For a more advanced behavior, instead of only tracking the object with head movements, the whole rover could automatically move to keep this object in focus.
- **Automatic movements:** Currently, some automatic movements are performed for some of the functionalities (e.g., automatically adjusting the wheel’s servomotors to perform a 360-degree rotation), but most of the functionalities are manually controlled using the received RC values. Additional automatic movements could be implemented, such as a movement to automatically fold and unfold the robotic arm of the rover.
- **Manufacturable PCBs:** As it can be seen in **Figure 11** and **Figure 14**, even though detailed schematics are provided, the wiring that is required for the main rover and for the custom RC module is quite complex and could be a bit difficult to replicate for beginners. To make it easier to replicate the project, manufacturable PCBs designs could be created so that the rover PCBs could be ordered from suppliers like JLCPCB. This way, people who replicate the project would only have to solder all the necessary components into this manufactured PCB boards, without needing to do most of the wiring.

Additionally, as explained in the **Fulfillment of requirements** section, some requirements have not been fully achieved, so they could also be considered as future lines of work to improve the OpenRover platform. This is the case of web dashboard, where the “monitor” page was fully implemented but the “control” page was not. Therefore, rover control from this control page is proposed as an improvement to the platform, which would consist in adding interactive elements like draggable joysticks to the UI, and adding all the necessary backend logic to be able to operate the rover remotely using this page.

Some of these previously detailed possible improvements and additional functionalities are also proposed on the main README file of the OpenRover GitHub page, encouraging people who replicate the project to add their own upgrades to the robot, integrating new components and developing additional features.

## 9 References

- [1] NASA. (n.d.). *Mars 2020: Perseverance rover*. <https://science.nasa.gov/mission/mars-2020-perseverance/>
- [2] NASA. (2020). *Perseverance Twin Drives Into The Mars Yard*. <https://science.nasa.gov/resource/perseverance-twin-drives-into-the-mars-yard/>
- [3] Rodriguez Manfredi, Jose A. (2021). *The Mars Environmental Dynamics Analyzer, MEDA: a suite of environmental sensors for the Mars 2020 mission*. <https://upcommons.upc.edu/handle/2117/360596>
- [4] Tuppathy, N., Goud Nizampet, M. (2024). *Design, analysis, and fabrication of rocker bogie mechanism*. <https://www.ijrar.org/papers/IJRARTH00245.pdf>
- [5] The planetary society. (n.d.). *Labeled curiosity mobility system*. <https://www.planetary.org/space-images/curiosity-mobility-system>
- [6] MATLAB & Simulink. (n.d.). *Modeling and Control of a Mars rover*. Ackermann steering. [https://es.mathworks.com/help/sm/ug/mars\\_rover.html](https://es.mathworks.com/help/sm/ug/mars_rover.html)
- [7] Cox, M., Junkins, E., Lofaro, O., NASA JPL. (2018). *Open Source Rover: Software Controls. Ackermann steering*. <https://wikifactory.com/+wikifactory/jpl-rover-project/file/software/Software%20Controls.pdf>
- [8] Red Hat. (n.d.). *What is open source software?* <https://opensource.com/resources/what-open-source>
- [9] Nedelkovski, Dejan. (2021). *DIY Mars Perseverance Rover Replica*. <https://howtomechatronics.com/projects/diy-mars-perseverance-rover-replica-with-arduino/>
- [10] Krantz, Jakob. (2019). *Mars Rover*. <https://github.com/jakkra/Mars-Rover>
- [11] NASA JPL. (2019). *JPL Open Source Rover Project*. <https://github.com/nasa-jpl/open-source-rover>
- [12] Arduino. (n.d.). *Arduino framework documentation*. <https://docs.arduino.cc/language-reference/#functions>
- [13] Espressif. (n.d.). *ESP-IDF framework documentation*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/>
- [14] PlatformIO. (n.d.). *PlatformIO documentation*. <https://docs.platformio.org/en/latest/>
- [15] LVGL. (n.d.). *LVGL library documentation*. <https://docs.lvgl.io/master/index.html>
- [16] Pallets projects. (n.d.). *Flask documentation*. <https://flask.palletsprojects.com/en/stable/>
- [17] Bootstrap. (n.d.) *Bootstrap documentation*. <https://getbootstrap.com/docs/4.1/getting-started/introduction/>
- [18] GitHub. (n.d.). *About GitHub and Git*. <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>
- [19] Arduino. (n.d.). *Arduino hardware documentation*. <https://docs.arduino.cc/hardware/>
- [20] Espressif. (n.d.). *SoC modules offered by Espressif*. <https://www.espressif.com/en/products/socs>
- [21] Supermicro corporation. (n.d.). *What is a System on Chip*. <https://www.supermicro.com/en/glossary/soc>
- [22] FreeRTOS. (n.d.). *Information about FreeRTOS*. <https://www.freertos.org/Why-FreeRTOS/What-is-FreeRTOS>

- [23] PJRC. (n.d.). *Teensy development boards*. <https://www.pjrc.com/teensy/>
- [24] Tutorialspoint. (n.d.). *Advantages and disadvantages of microcontrollers*. [https://www.tutorialspoint.com/microprocessor/microcontrollers\\_advantages\\_and\\_disadvantages.htm](https://www.tutorialspoint.com/microprocessor/microcontrollers_advantages_and_disadvantages.htm)
- [25] Vemeko FPGA. (2024). *Comparison between microcontrollers and single board computers*. <https://www.vemeko.com/blog/the-comparison-between-microcontrollers-and-single-board-computer.html>
- [26] Raspberry Pi Foundation. (n.d.). *Raspberry Pi hardware*. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#introduction>
- [27] Ilitek corporation. (n.d.). *ILI9341 display driver datasheet*. <https://www.alldatasheet.es/datasheet-pdf/view/1131760/ETC2/ILI9341.html>
- [28] Sitronix corporation. (2018). *ST7262 display driver datasheet*. <https://www.orientdisplay.com/pdf/ST7262.pdf>
- [29] Campbell, Scott. (2016). *Explanation of SPI communication protocol*. <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>
- [30] Campbell, Scott. (2016). *Explanation of UART communication protocol*. <https://www.circuitbasics.com/basics-uart-communication/>
- [31] Campbell, Scott. (2016). *Explanation of I2C communication protocol*. <https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/>
- [32] Focus LCDs company. (n.d.). *RGB Parallel Interface*. [https://focuslcds.com/wp-content/uploads/LCDResources/Parallel-Interfaces-MCU-vs-RGB.pdf?srsId=AfmBOoqOw-Q9Z1\\_HKnPQt4zG4Ueev-JimZH3wMhN9Lk2CFuYDZyUz2nM](https://focuslcds.com/wp-content/uploads/LCDResources/Parallel-Interfaces-MCU-vs-RGB.pdf?srsId=AfmBOoqOw-Q9Z1_HKnPQt4zG4Ueev-JimZH3wMhN9Lk2CFuYDZyUz2nM)
- [33] ESPboards. (n.d.). *ESP32S3-8048 development board with touchscreen*. <https://www.espboards.dev/esp32/cyd-esp32-8048s050/>
- [34] ESPboards. (n.d.). *ESP32-2432 development board with touchscreen*. <https://www.espboards.dev/esp32/cyd-esp32-2432s032/>
- [35] Raspberry Pi Foundation. (2024). *Raspberry Pi Camera Module 3 datasheet*. <https://datasheets.raspberrypi.com/camera/camera-module-3-product-brief.pdf>
- [36] Jarema, Radek. (2024). *Choosing the right battery for a robot*. <https://husarion.com/blog/batteries-for-mobile-robots/>
- [37] Nordic Semiconductor. (2008). *NRF24L01+ datasheet*. [https://docs.nordicsemi.com/bundle/nRF24L01P\\_PS\\_v1.0/resource/nRF24L01P\\_PS\\_v1.0.pdf](https://docs.nordicsemi.com/bundle/nRF24L01P_PS_v1.0/resource/nRF24L01P_PS_v1.0.pdf)
- [38] Bosch corporation. (2018). *BME280 sensor datasheet*. <https://www.alldatasheet.es/datasheet-pdf/view/1132060/BOSCH/BME280.html>
- [39] Texas instruments. (2015). *DRV8871 motor driver datasheet*. <https://www.alldatasheet.es/datasheet-pdf/view/799728/TI1/DRV8871.html>
- [40] Allegro MicroSystems corporation. (n.d.). *A4988 microstepping driver*. <https://www.alldatasheet.es/datasheet-pdf/view/338780/ALLEGRO/A4988.html>
- [41] Endurance RC. (n.d.). *PPM and PWM signals in RC industry*. <http://www.endurance-rc.com/ppmtut.php>
- [42] Horizon Hobby company. (n.d.). *Spektrum DX8 instruction manual with available control elements*. [https://www.horizonhobby.com/on/demandware.static/Sites-horizon-us-Site/Sites-horizon-master/default/Manuals/SPM8000\\_Manual\\_EN.pdf](https://www.horizonhobby.com/on/demandware.static/Sites-horizon-us-Site/Sites-horizon-master/default/Manuals/SPM8000_Manual_EN.pdf)

- [43] Arduino. (n.d.). *Arduino IDE documentation*. <https://docs.arduino.cc/software/ide/#ide-v2>
- [44] Espressif. (n.d.). *ESP-IDF features and overview*. <https://www.espressif.com/en/products/sdks/esp-idf>
- [45] Microsoft. (n.d.). *Visual Studio Code documentation*. <https://code.visualstudio.com/docs>
- [46] Raspberry Pi Foundation. (n.d.). *Raspberry Pi Operating Systems*. <https://www.raspberrypi.com/software/operating-systems/>
- [47] RF24 project community. (n.d.). *RF24 library documentation*. <https://nrf24.github.io/RF24/>
- [48] Espressif. (n.d.). *ESP-NOW documentation with Arduino framework*. <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnw.html>
- [49] Espressif. (n.d.). *ESP-NOW overview*. <https://www.espressif.com/en/solutions/low-power-solutions/esp-now>
- [50] Arduino. (n.d.). *Arduino framework included communication protocol libraries*. <https://docs.arduino.cc/learn/communication/wire/>
- [51] Arduino. (n.d.). *Arduino SoftwareSerial library*. <https://docs.arduino.cc/learn/built-in-libraries/software-serial/>
- [52] SerialTransfer project community. (n.d.). *SerialTransfer library and documentation*. <https://github.com/PowerBroker2/SerialTransfer>
- [53] ArduinoGFX project community. (n.d.). *ArduinoGFX library and documentation*. [https://github.com/moononournation/Arduino\\_GFX](https://github.com/moononournation/Arduino_GFX)
- [54] SquareLine Studio. (n.d.). *SquareLine Studio documentation*. <https://docs.squareline.io/docs/squareline>
- [55] Raspberry Pi Foundation. (2025). *Picamera2 library documentation*. <https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf>
- [56] SoftPWM and PalatisSoftPWM project community. (n.d.). *PalatisSoftPWM library and documentation*. <https://github.com/per1234/PalatisSoftPWM>
- [57] Arduino. (n.d.). *Arduino Servo library*. <https://docs.arduino.cc/libraries/servo/>
- [58] StepperDriver project community. (n.d.). *StepperDriver library and documentation*. <https://github.com/laurb9/StepperDriver>
- [59] Adafruit company. (n.d.). *Adafruit/Arduino BME280 library and documentation*. [https://github.com/adafruit/Adafruit\\_BME280\\_Library](https://github.com/adafruit/Adafruit_BME280_Library)
- [60] McCauley, Mike. (2011). *RcTrainer PPM library and documentation for Arduino*. <https://www.airspayce.com/mikem/arduino/RcTrainer/>
- [61] ESPAsyncWebServer project community. (n.d.). *ESPAsyncWebServer library for ESP32*. <https://github.com/ESP32Async/ESPAsyncWebServer>
- [62] OpenJS Foundation. (n.d.). *Introduction to Node.js*. <https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>
- [63] OpenJS Foundation. (n.d.). *Express.js guide*. <https://expressjs.com/en/guide/routing.html>
- [64] OmniVision company. (2011). *OV5640 datasheet containing key features*. [https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/OV5640\\_datasheet.pdf](https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/OV5640_datasheet.pdf)
- [65] DFROBOT company. (n.d.). *ESP32-CAM product details and key features*. [https://media.digikey.com/pdf/Data%20Sheets/DFRobot%20PDFs/DFR0602\\_Web.pdf](https://media.digikey.com/pdf/Data%20Sheets/DFRobot%20PDFs/DFR0602_Web.pdf)

- [66] Arduino. (n.d.). *Guide to LoRa and LPWAN technologies*.  
<https://docs.arduino.cc/learn/communication/lorawan-101/>
- [67] Elecrow company. (2012). *HC-12 Wireless communication module datasheet*.  
<https://www.elecrow.com/download/HC-12.pdf?srsId=AfmBOopKj-hdz8VNYNb-AkJp-ZnnbrfcG1jtsQhpguSB2xuCdifMaoAG>
- [68] Aosong Electronics company. (n.d.). *Humidity and temperature sensor datasheet*.  
<https://www.alldatasheet.es/datasheet-pdf/view/1132459/ETC2/DHT22.html>
- [69] ST microelectronics company. (2000). *L298N dual motor driver datasheet*.  
<https://www.alldatasheet.es/datasheet-pdf/view/22440/STMICROELECTRONICS/L298N.html>
- [70] Waveshare company. (n.d.). *MG90S micro servo product specifications*.  
<https://www.waveshare.com/mg90s-servo.htm>
- [71] Handson technology company. (n.d.). *MG996R servo product specifications*.  
[https://www.handsontec.com/dataspecs/motor\\_fan/MG996R.pdf](https://www.handsontec.com/dataspecs/motor_fan/MG996R.pdf)
- [72] Texas instruments. (2010). *DRV8825 stepper motor controller*.  
<https://www.alldatasheet.es/datasheet-pdf/view/835822/TI1/DRV8825.html>
- [73] Arduino LoRa project community. (n.d.). *Arduino LoRa library and documentation*.  
<https://github.com/sandeepmistry/arduino-LoRa>
- [74] AdafruitGFX project community. (n.d.). *AdafruitGFX graphics library and documentation*. <https://github.com/adafruit/Adafruit-GFX-Library>
- [75] TFT eSPI project community. (n.d.). *TFT eSPI graphics library and documentation*.  
[https://github.com/Bodmer/TFT\\_eSPI](https://github.com/Bodmer/TFT_eSPI)
- [76] Envoy Experimental Zone company. (n.d.). *EEZ Studio documentation*.  
<https://www.envoy.eu/eez-studio-docs/>
- [77] Espressif. (n.d.). *ESP32 camera driver library and documentation*.  
<https://github.com/espressif/esp32-camera>
- [78] ESP32Servo project community. (n.d.). *Servo library for ESP32 and documentation*.  
<https://github.com/madhephaestus/ESP32Servo>
- [79] AccelStepper project community. (n.d.). *AccelStepper library and documentation*.  
<https://github.com/waspinator/AccelStepper>
- [80] Adafruit company. (n.d.). *DHT sensor library and documentation*.  
<https://github.com/adafruit/DHT-sensor-library>
- [81] PPM reader project. (n.d.). *Arduino PPM reader library and documentation*.  
<https://github.com/dimag0g/PPM-reader>
- [82] Espressif. (n.d.). *ESP32 with Arduino framework documentation*. [https://espressif-docs.readthedocs-hosted.com/projects/arduino-esp32/en/latest/getting\\_started.html](https://espressif-docs.readthedocs-hosted.com/projects/arduino-esp32/en/latest/getting_started.html)
- [83] Raspberry Pi foundation. (n.d.). *RaspberryPi software documentation. Configuration to enable hotspot*.  
<https://www.raspberrypi.com/documentation/computers/configuration.html#enable-hotspot>
- [84] Arduino. (n.d.). *Arduino Bootloader documentation*.  
<https://docs.arduino.cc/retired/hacking/software/Bootloader/>
- [85] Flaticon. (n.d.). *Flaticon vector icons searcher*. <https://www.flaticon.com/>
- [86] Figma. (n.d.). *Figma interface design tool*. <https://www.figma.com/>

- [87] Raspberry Pi foundation. (n.d.). *RaspberryPi firmware configuration file*.  
[https://www.raspberrypi.com/documentation/computers/legacy\\_config\\_txt.html#init\\_uart\\_baud](https://www.raspberrypi.com/documentation/computers/legacy_config_txt.html#init_uart_baud)
- [88] Posch, Maya. (2024). *Benchmarking latency across common wireless links for MCUs*.  
<https://hackaday.com/2024/02/11/benchmarking-latency-across-common-wireless-links-for-mcus/>
- [89] Waveshare company. (n.d.). *USB HUB HAT for Raspberry Pi*.  
[https://www.waveshare.com/wiki/USB\\_HUB\\_HAT\\_\(B\)](https://www.waveshare.com/wiki/USB_HUB_HAT_(B))
- [90] Silicon labs company. (2007). *CP2102 USB to UART bridge*.  
<https://www.alldatasheet.es/datasheet-pdf/view/201067/SILABS/CP2102.html>
- [91] RaspberryPi foundation. (n.d.). *RaspberryPi frequency management and thermal control*. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#frequency-management-and-thermal-control>
- [92] SquareLine Studio. (n.d.). *Squareline Studio widget types*.  
<https://docs.squareline.io/docs/widgets>
- [93] Shenzhen Jingcai Intelligent company. (n.d.). *ESP32S3-8048 specifications, user manual, sample configuration code and schematic*.  
[http://pan.jczn1688.com/directlink/1/ESP32%20module/4.3inch\\_ESP32-8048S043.zip](http://pan.jczn1688.com/directlink/1/ESP32%20module/4.3inch_ESP32-8048S043.zip)
- [94] VScode SFTP project community. (n.d.). *SFTP plugin for VScode*.  
<https://marketplace.visualstudio.com/items?itemName=Natizyskunk.sftp>
- [95] Pallets projects. (n.d.). *Flask SocketIO library documentation*. [https://flask-socketio.readthedocs.io/en/latest/getting\\_started.html](https://flask-socketio.readthedocs.io/en/latest/getting_started.html)
- [96] Fonticons incorporated. (n.d.). *CSS icon library*. <https://fontawesome.com/>
- [97] Chart.js project community. (n.d.). *JavaScript charting library*.  
<https://www.chartjs.org/>
- [98] Loeber, Patrick. (2019). *Threading vs Multiprocessing. Implementation in python*.  
<https://www.python-engineer.com/courses/advancedpython/15-thread-vs-process/>



# Annex I

In this annex, the designing, 3D printing, and assembling steps of the rover's mechanical structure are detailed for each module, with images complementing the explanation.

## Central rover module

### Design

The original rover's central mechanical platform design from the *DIY Mars Perseverance Rover Replica* [9] project had several flaws, including the lack of a robotic arm, missing bottom panels for the body and inexistence of protective cases for the motors and servomotors, among other more minor flaws.

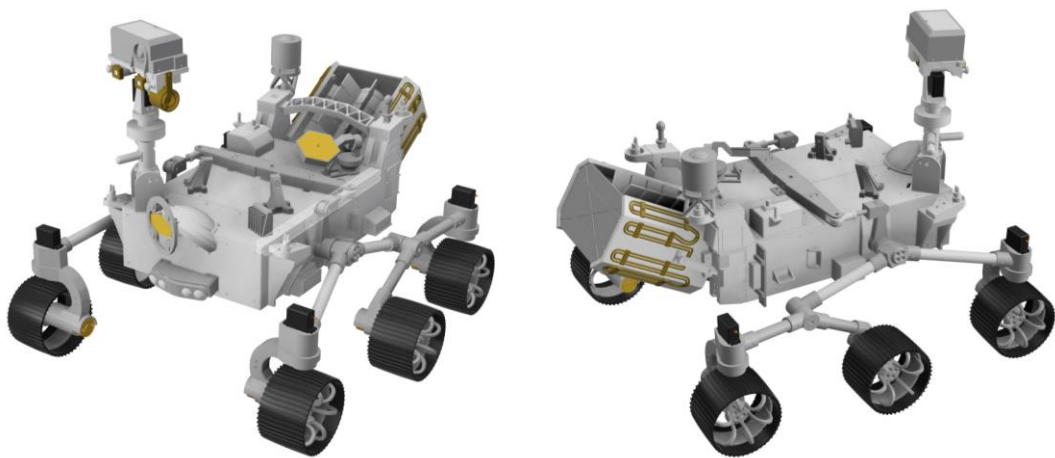


Figure 48. Original DIY Perseverance Rover Replica design (own elaboration of images and original design extracted from [9] )

To incorporate a foldable robotic arm into OpenRover, the on-board robotic arm design from the *Jakob Krantz Mars Rover* [10] project was taken as a starting point. This robotic arm's design was heavily modified so that it fit with the dimensions of OpenRover and could be mounted on the front panel. Specifically, among other things, the tubes were shortened to ensure an optimum resting position, the main mounting plate was simplified and reinforced, the gripper was changed, and casings were created to protect the gripper's gears and the servomotors.

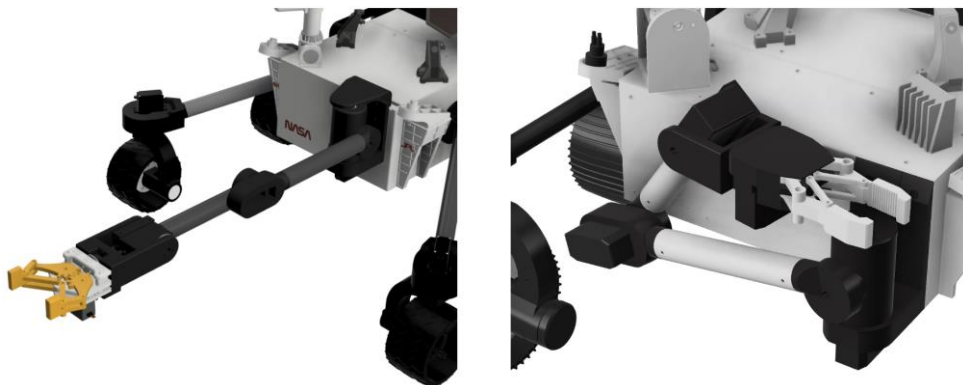
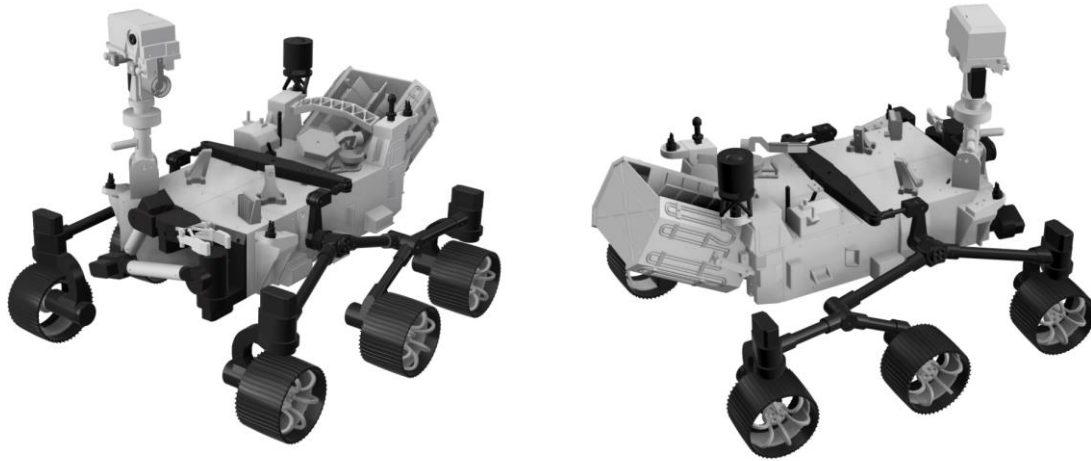


Figure 49. Original (left) and modified (right) Jakob Krantz arm design (own elaboration of images and original designs extracted from [9] [10] )

Flaws like the missing bottom panels, protective cases and other minor flaws were also corrected by doing improvements and adding new components to the design. Additionally, the color palette used for the rover's design was also changed, improving it to get a more elegant look with black, white, and silver components.



**Figure 50. Modified DIY Perseverance Rover Replica design (own elaboration of images and original design extracted from [9] )**

### 3D printing

Once these designs were modified, it was possible to 3D print all the necessary mechanical components. Many of these 3D printed components, especially the lateral panels and the back section, do not fulfill any functional purpose and are only present for aesthetic reasons. After a lot of hours of 3D printing, all the pieces of the rover's body were completed, and they were placed on a table in an organized manner to facilitate the next assembly step.



**Figure 51. Organized set of OpenRover's 3D printed pieces (own elaboration)**

## Assembly

The mechanical assembly process was long and a bit complex, with some of parts of the process being done by following the assembly guide of *DIY Mars Perseverance Rover Replica* [9] and many other parts being done in a different and original way.

As this final degree project does put a lot of focus on the mechanical aspect, only some of the most interesting parts of the assembly process will be shown and explained here. In the attached project files, more images that detail the whole assembly process of OpenRover will be shown.

The chassis of the rover uses standard 2020 aluminum profiles to create the bare cuboid shape of the body. The 3D printed panels and other elements are attached to these profiles using screws and custom nuts that fit inside the profiles. For the robotic arm and rocker-bogie structure, PVC and aluminum tubes are used to connect the different 3D printed components.



Figure 52. OpenRover body assembly (own elaboration)

The rover's mechanical platform has a size of 81x68x60 cm and an approximate weight of 7 kg.



## Remote control and touchscreen module

The custom remote control and touchscreen module is an add-on to an existing RC. To incorporate into this RC all the additional electronics and screen, 3D printed cases are created. Thanks to these cases and to the used connectors, it is very easy to add and remove the custom modules, allowing to quickly return the generic RC to its original state if needed.

The design of the radio module case is completely original, and adapts to the shape of the soldered PCB of the module, as well as the shape of the Spektrum DX8 RC. The design of the screen case is also very custom, but is based on an existing case design for the ESP32S3-8048 touchscreen found on the internet.

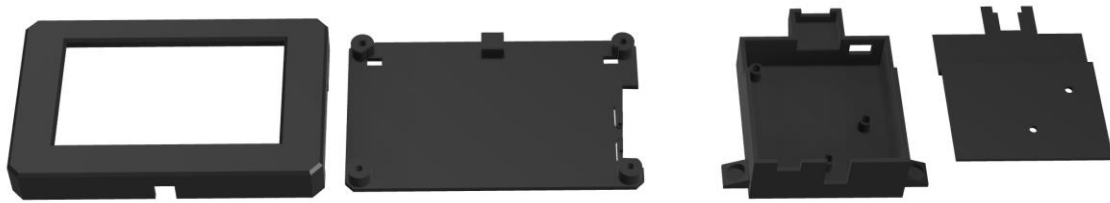


Figure 53. Custom remote control cases design (own elaboration)

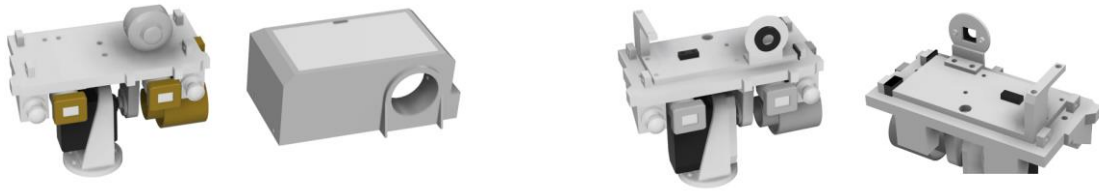
The case for the custom radio module is located at the back of the transmitter, to allow easy access to the trainer port and battery, which are also located at the back. The case for the screen is located at the front and, among other things, is used to fix the screen to the RC using a single screw.



Figure 54. 3D-printed remote-control cases (own elaboration)

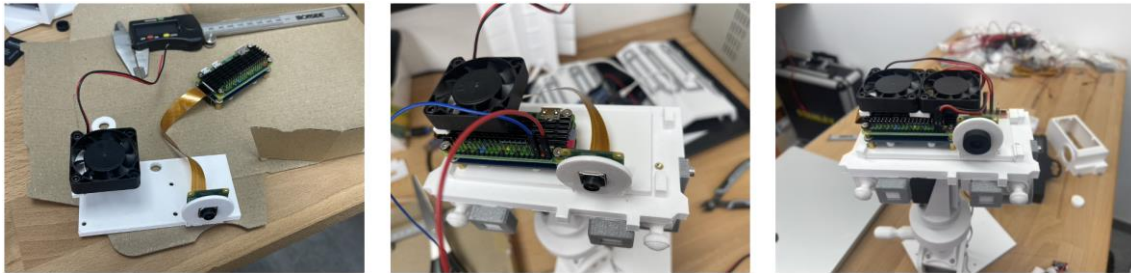
## Camera and webserver module

The original camera housing of the *DIY Mars Perseverance Rover Replica* [9] project, located on the rover's head, was designed only to accommodate a small-sized camera with a built-in analog transmission module. Therefore, to incorporate the RaspberryPi Zero 2W board, camera and other accessories (e.g., USB HAT, fans) a complete and custom redesign of the interior housing was needed.



**Figure 55. Original (left) and modified (right) interior head design (own elaboration of images and original design extracted from [9] )**

Although the changes between the interior designs of the head may seem minor, lots of precise measurements of the RaspberryPi, USB-HAT, USB-TTL, fans and camera had to be taken. All the components inside the head had to fit in a very compact form, as not much room was available.



**Figure 56. Interior head components placement (own elaboration)**